KOGI

Independent Worker Operating System

Low-Level Software Design Document

Complete Engineering Reference — All Systems, All Modules, All Features

Version 1.0.0 · Platform Engineering

Document Type	Low-Level Software Design Document (LLSDD)
Version	1.0.0 — Complete Platform Engineering Reference
Scope	28 sections · All 12 modules · Cross-cutting systems · Infrastructure · Security · Observability
Target Audience	Platform engineers, backend engineers, DevOps, security team, technical leads
Primary Languages	Zig 0.12 (Kernel) · Go 1.22 (Services) · Scala 2.13 (Analytics) · TypeScript/Angular 17 (Web) · Java 17/JavaFX (Desktop)
Build System	Bazel 7.x — monorepo, all targets cross-compiled, hermetic builds
Companion Docs	PRD v1.0 · PRS v1.0 · RFC-001 through RFC-008
Last Updated	2025

Table of Contents

§1 Introduction & Document Conventions
§2 System Architecture Overview
§3 Deployment & Infrastructure
§4 Kernel System — Zig Implementation
§5 Authentication & Session System
§6 User, Profile & Workspace System
§7 Portfolio System
§8 WBS & Story System
§9 Market System
§10 Exchange & Financial System
§11 Idea Studio System
§12 Community & Social System
§13 Work, Strategy & OKR System
§14 Crowdfunding & Capital System
§15 Autonomous Organization Governance System
§16 AI Agent System
§17 Search & Discovery System
§18 Notification & Alert System
§19 Analytics & Reporting System
§20 Security Architecture
§21 Performance & Caching Architecture
§22 Observability, Logging & Tracing
§23 Background Job System
§24 API Design Patterns & Contracts
§25 Error Handling & Taxonomy
§26 Test Strategy & Test Catalog
§27 Data Migration & Schema Versioning
§28 Glossary & Reference

§1 · INTRODUCTION

Introduction & Document Conventions

1.1 Purpose

This Low-Level Software Design Document (LLSDD) is the complete engineering reference for the Kogi Independent Worker Operating System. It covers every system, module, service, data model, algorithm, interface, and cross-cutting concern required to build, deploy, operate, and evolve the platform.

This document is designed to be read by engineers implementing the platform. It contains sufficient detail to write code directly from its specifications. Where implementation decisions have not yet been finalized, open questions are clearly marked.

1.2 Document Structure

Sections §1–§3 cover platform-wide architecture and infrastructure. Sections §4–§16 cover individual modules in depth. Sections §17–§19 cover platform-wide cross-cutting services. Sections §20–§28 cover non-functional concerns, operational practices, and reference material.

1.3 Conventions

Convention	Meaning	Example
CHAR(36)	UUID stored as fixed-length string	id CHAR(36) NOT NULL
TIMESTAMPTZ	PostgreSQL timezone-aware timestamp (UTC)	created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
JSONB	PostgreSQL binary JSON column	metadata JSONB NOT NULL DEFAULT '{}'::jsonb
→	State machine transition (from → to)	BACKLOG → READY
FK →	Foreign key reference	story_id FK → story.id
[p95]	95th percentile latency	[p95] < 200ms
INV-N	Platform invariant N (never-violate rule)	INV-3: Ledger entries are immutable
REQ-XXX-N	Platform requirement from PRS	REQ-KERN-001
Code blocks	Source code or pseudocode	// Go: service interface definition
⚠️ CRITICAL	Engineering decision with serious consequences if wrong	⚠️ CRITICAL: SELECT FOR UPDATE required

1.4 Platform-Wide Engineering Invariants

⚠️ Inviolable Engineering Rules — These Apply Across All Modules

- INV-1: No cross-service database joins. Each service owns its schema exclusively.
- INV-2: All cross-module communication is asynchronous via NATS JetStream using the Outbox Pattern (RFC-001).
- INV-3: Financial ledger entries are immutable. DB trigger enforces: no UPDATE/DELETE on ledger_entry table.
- INV-4: All wallet mutations use SELECT FOR UPDATE within a DB transaction. No exceptions.
- INV-5: All mutation endpoints require an Idempotency-Key header. Missing key → 400 error.
- INV-6: All DB operations affecting multiple tables are wrapped in ACID transactions.
- INV-7: Soft delete only. deleted_at IS NULL in all domain queries. 30-day recovery window.
- INV-8: Optimistic locking (version integer) on all mutable aggregate roots. Version mismatch → 409 CONFLICT.
- INV-9: PII (email, name, payment data) is never written to application logs. user_id only.
- INV-10: All secrets managed via AWS Secrets Manager. No secrets in source code, env files, or K8s manifests.

§2 · ARCHITECTURE

System Architecture Overview

2.1 Architecture Philosophy

Kogi uses a Modular Monolith-to-Microservices evolution pattern. The platform launches as a well-structured monolith with strict module boundaries enforced at the build system (Bazel) and runtime (Zig kernel module registry) levels. Each module can be extracted to an independent microservice without changing the inter-module contract because all communication already flows through the Event Bus.

ℹ️ **Core Architectural Decisions**

- Event-driven core: all cross-module side effects triggered via NATS JetStream events with transactional outbox guarantee
- Shared-nothing storage: each module owns its PostgreSQL schema; no cross-schema joins in production
- CQRS at module boundaries: writes through command handlers; reads from optimized projections or Redis cache
- Portfolio as universal context: every module is anchored to a Portfolio item — the single source of truth for ownership and organizational structure
- AI as first-class infrastructure: the AI Agent is a platform service, not a feature bolt-on; it consumes events from all modules
- Privacy by default: all resources default to private; explicit action required to share or publish
- Cooperative-first: group ownership, governance, and collective financial operations are first-class primitives, not afterthoughts

2.2 Service Inventory

Service	Lang/Runtime	Protocol(s)	Port	DB	Scaling Model
api-gateway	Go 1.22 / Gin	HTTP/2 + gRPC proxy	8080	None (stateless)	Horizontal — N instances, ALB
auth-service	Go 1.22	gRPC internal	9001	PostgreSQL 16	Horizontal, Redis session store
user-service	Go 1.22	gRPC + events	9002	PostgreSQL 16	Horizontal
portfolio-service	Go 1.22	gRPC + events	9003	PostgreSQL 16	Horizontal
wbs-service	Go 1.22	gRPC + events	9004	PostgreSQL 16	Horizontal
market-service	Go 1.22	gRPC + events	9005	PostgreSQL 16 + ES	Horizontal
exchange-service	Go 1.22	gRPC + events	9006	PostgreSQL 16	Active-Passive (financial)

Service	Lang/Runtime	Protocol(s)	Port	DB	Scaling Model
studio-service	Go 1.22	gRPC + events	9007	PostgreSQL 16	Horizontal
community-service	Go 1.22	gRPC + WS + events	9008	PostgreSQL 16	Horizontal + NATS WS fan-out
work-service	Go 1.22	gRPC + events	9009	PostgreSQL 16	Horizontal
fund-service	Go 1.22	gRPC + events	9010	PostgreSQL 16	Active-Passive (financial)
ao-service	Go 1.22	gRPC + events	9011	PostgreSQL 16	Horizontal
ai-service	Go 1.22	gRPC + SSE + events	9012	PostgreSQL 16	Horizontal (GPU-aware scheduling)
search-service	Go 1.22	gRPC + events	9013	Elasticsearch 8	Cluster — 3+ data nodes
notification-svc	Go 1.22	gRPC + events	9014	PostgreSQL 16 + Redis	Horizontal
analytics-service	Scala 2.13 / Akka	HTTP + gRPC	8081	ClickHouse + Redis	Horizontal
kernel	Zig 0.12	FFI (C ABI) + TCP	9000	In-memory + WAL	Single primary + hot standby
event-bus	NATS 2.10 JetStream	NATS protocol	4222	JetStream (built-in)	Cluster of 3 nodes minimum

2.3 Data Flow Patterns

Write Path (Command)

```
Client → TLS/LB → API Gateway (JWT validation, rate limit, idempotency check)
  → Target Service Command Handler
    → Validate request (struct tags + business rules)
    → BEGIN TRANSACTION
      → Write domain state (INSERT/UPDATE domain tables)
      → INSERT outbox_events (status=PENDING, same transaction)
    → COMMIT
  → Return response to client

Outbox Relay (async, per-service background goroutine):
  → SELECT * FROM outbox_events WHERE status=PENDING FOR UPDATE SKIP LOCKED LIMIT
50
  → NATS.Publish(subject, payload)
  → On ACK: UPDATE outbox_events SET status=PUBLISHED, published_at=NOW()
  → On NATS failure: retry with exponential backoff; after 10 attempts → DLQ
```

Read Path (Query)

```
Client → API Gateway → Service Read Handler
  → Redis L1 Cache: HIT → return; MISS → continue
  → Read Replica PostgreSQL (L2): query, SET Redis with TTL, return
  → (Heavy aggregates) ClickHouse OLAP (L3): query, SET Redis, return

Cache invalidation (event-driven):
  → NATS consumer receives domain event
  → Cache invalidation handler resolves affected keys
  → Redis DEL (or SET new value for write-through resources)
```

2.4 Cross-Module Data Relationships

Module A	Relationship	Module B	Mechanism
Portfolio	owns	WBS	portfolio_item_id FK in wbs table; created via event
WBS Story	linked to	Market Order	story_id in market_order; auto-created on order.placed event
Market Order	triggers	Exchange Escrow	order.placed event → exchange creates escrow
Exchange Escrow	releases to	Portfolio Capital Resource	escrow.released event → portfolio updates capital balance
Studio Idea	promotes to	Portfolio Project	idea.promoted event → portfolio creates item + wbs
Work OKR	aggregates	WBS Stories	story.completed event → OKR check-in auto-update
Community Space	linked to	Portfolio AO	ao_id FK in space; governance events update space access
Crowdfunding Campaign	funds	AO Capital Pool	campaign.funded event → capital_pool balance update
AI Agent	reads context from	All modules	Read-only gRPC calls to portfolio, wbs, work, exchange on each session

Module A	Relationship	Module B	Mechanism
Notification	fans out from	All modules	All domain events route through notification service for delivery

2.5 Platform Event Bus Architecture

Subject Naming Convention

```
// Format: {module}.{entity}.{action}
// Examples:
portfolio.item.created
portfolio.item.status_changed
wbs.story.blocked
wbs.sprint.closed
market.order.placed
market.order.delivered
exchange.escrow.funded
exchange.escrow.released
exchange.distribution.completed
studio.idea.captured
studio.idea.promoted
community.message.created
community.space.member_joined
work.okr.scored
work.automation.triggered
fund.campaign.funded
fund.campaign.failed
ao.proposal.submitted
ao.proposal.passed
ao.proposal.failed
ai.synthesis.completed
ai.action.taken
ai.alert.generated
user.provisioned
user.plan_changed
```

NATS JetStream Configuration

Stream	Subjects	Retention	Max Age	Consumers
PLATFORM	All platform events (*)	Limits	7 days	All services + analytics
FINANCIAL	exchange.*, fund.*	Limits	365 days	exchange-svc, fund-svc, analytics, audit
GOVERNANCE	ao.*, community.*	Limits	forever	ao-svc, community-svc, audit
AI	ai.*	Limits	90 days	ai-svc, analytics

Stream	Subjects	Retention	Max Age	Consumers
DLQ	dlq.*	Limits	30 days	on-call alerting, ops tooling

§3 · INFRASTRUCTURE

Deployment & Infrastructure

3.1 Cloud Infrastructure

Cloud Provider	AWS (primary). Terraform modules are GCP/Azure-compatible for portability.
Production Regions	3 active regions: us-east-1 (primary), eu-west-1, ap-southeast-1. Active-active for stateless services.
Container Orchestration	Kubernetes 1.30 (EKS). k3s for local development and CI.
Service Mesh	Istio 1.22. mTLS between all pods. Traffic management, circuit breaking, telemetry.
Load Balancing	AWS ALB (Layer 7) → Istio ingress → API Gateway pods.
DNS	Route53 with latency-based routing. api.kogi.io → nearest region.
CDN	CloudFront. Serves web SPA, media assets, and user-uploaded files from S3.
Certificate Management	AWS ACM. TLS 1.3 minimum. Cert renewal automated.
Database	Amazon RDS Aurora PostgreSQL 16. Multi-AZ per region. Read replicas per service.
Cache	ElastiCache Redis 7.2. Cluster mode enabled. 3 shards × 2 replicas per region.
Message Bus	NATS 2.10 JetStream. 3-node cluster per region. Geo-replication across regions.
Search	Elasticsearch 8.12. 3 data nodes + 1 coordinating node per region.
Object Storage	S3 + CloudFront. Lifecycle policies: Standard → IA at 30d → Glacier at 180d.
Secret Management	AWS Secrets Manager. Lambda auto-rotation. Versioned secrets.
Container Registry	AWS ECR. Image signing with Cosign. Vulnerability scanning on push.
CI/CD	GitHub Actions → Bazel build → Docker build → ECR push → ArgoCD GitOps deploy.

3.2 Kubernetes Architecture

Namespace Strategy

Namespace	Services Deployed	Resource Limits	Network Policy
kogi-ingress	ALB ingress controller, cert-manager, external-dns	CPU:4c Mem:8Gi	Ingress from internet; egress to kogi-core only
kogi-core	api-gateway, auth-service, kernel	CPU:16c Mem:32Gi	Ingress from kogi-ingress; egress to kogi-services via Istio
kogi-services	portfolio, wbs, market, exchange, studio, community, work, fund, ao, user	CPU:64c Mem:128Gi	Internal only; kogi-core ingress; kogi-data egress
kogi-ai	ai-service (GPU node pool)	CPU:32c Mem:64Gi GPU:4xA10G	Internal only; egress to LLM API allowlist only
kogi-data	NATS cluster, Redis, Elasticsearch	CPU:32c Mem:64Gi	kogi-services + kogi-ai + kogi-analytics ingress only
kogi-analytics	analytics-service, ClickHouse	CPU:16c Mem:32Gi	kogi-services read egress; kogi-data ingress
kogi-cdn	nginx media proxy, image resize worker	CPU:8c Mem:16Gi	Internet ingress; S3 egress
kogi-ops	Prometheus, Grafana, Loki, Tempo, ArgoCD	CPU:16c Mem:32Gi	All namespaces read; no data namespace write

Pod Autoscaling Configuration

Service	Min Pods	Max Pods	Scale Trigger	Scale Up Delay
api-gateway	3	20	CPU>60% or RPS>500/pod	30s
auth-service	2	10	CPU>70%	60s
portfolio-service	2	15	CPU>70% or queue>100	60s
wbs-service	2	15	CPU>70%	60s
market-service	3	20	CPU>60% or ES-query-lag>500 ms	45s
exchange-service	2	6	CPU>60% (capped: financial isolation)	60s
community-service	3	30	WS-connections>2 000/pod or CPU>65%	30s
ai-service	2	10	GPU>70% or queue>50	120s (GPU warm-up)

Service	Min Pods	Max Pods	Scale Trigger	Scale Up Delay
notification-svc	2	15	NATS consumer lag>1000	30s
analytics-service	2	8	CPU>75%	90s

3.3 Database Architecture

Per-Service Database Configuration

Service DB	Instance Class	Storage	IOPS	Read Replicas	Backup
kogi-auth-db	db.r7g.large	100GB gp3	3000	1 (same AZ)	PITR 30d + daily snapshot, 7d retention
kogi-user-db	db.r7g.large	200GB gp3	3000	2 (multi-AZ spread)	PITR 30d + daily snapshot
kogi-portfolio-db	db.r7g.xlarge	500GB gp3	6000	2 (multi-AZ spread)	PITR 30d + daily snapshot
kogi-wbs-db	db.r7g.xlarge	500GB gp3	6000	2	PITR 30d + daily snapshot
kogi-market-db	db.r7g.2xlarge	1TB gp3	8000	3 (global secondary for search)	PITR 30d + daily snapshot
kogi-exchange-db	db.r7g.2xlarge	500GB io1	16000	1 (Multi-AZ, passive failover ONLY)	PITR 90d + HOURLY snapshot, 30d retention
kogi-studio-db	db.r7g.large	200GB gp3	3000	1	PITR 30d + daily snapshot
kogi-community-db	db.r7g.2xlarge	1TB gp3	8000	3 (high read volume)	PITR 30d + daily snapshot
kogi-work-db	db.r7g.large	200GB gp3	3000	1	PITR 30d + daily snapshot
kogi-fund-db	db.r7g.xlarge	200GB io1	8000	1 (passive failover ONLY)	PITR 90d + hourly snapshot
kogi-ao-db	db.r7g.large	100GB gp3	3000	1	PITR 30d + daily snapshot — APPEND-ONLY governance log
kogi-ai-db	db.r7g.large	200GB gp3	3000	1	PITR 30d + daily snapshot

PgBouncer Connection Pooling

```
# pgbouncer.ini — per-service configuration (example: exchange-service)
[databases]
```

```
kogi_exchange = host=kogi-exchange-db.cluster.rds.amazonaws.com port=5432
dbname=kogi_exchange

[pgbouncer]
pool_mode = transaction          # transaction-level pooling for stateless
services
max_client_conn = 500            # max connections from service pods
default_pool_size = 20           # DB connections per user/db pair
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3
server_idle_timeout = 600
client_idle_timeout = 0          # keep client connections alive
query_timeout = 30
connect_timeout = 3

# Exchange service uses transaction pooling (NOT session) because:
# 1. No SET LOCAL or advisory locks needed (SELECT FOR UPDATE is sufficient)
# 2. Dramatically improves connection reuse under high concurrency
# 3. 500 service connections → 20 DB connections = 25x multiplexing
```

3.4 NATS JetStream Cluster Configuration

```
# nats-server.conf — production cluster node configuration
server_name: kogi-nats-1
listen: 0.0.0.0:4222
cluster {
  name: kogi-cluster-us-east-1
  listen: 0.0.0.0:6222
  routes: [
    nats://kogi-nats-1:6222,
    nats://kogi-nats-2:6222,
    nats://kogi-nats-3:6222
  ]
}
jetstream {
  store_dir: /data/nats/jetstream
  max_memory_store: 8GB
  max_file_store: 100GB
}
accounts {
  $SYS { users: [{ user: sys, password: "$SYS_PASS" }] }
  kogi {
    jetstream: enabled
    users: [{ user: kogi-services, password: "$KOGI_PASS" }]
    limits { max_subscriptions: 10000, max_payload: 1MB }
  }
}
```

3.5 CI/CD Pipeline

Stage	Tool	Trigger	Duration Target	Gate
Code checkout + lint	GitHub Actions + golangci-lint	PR opened/updated	< 2 min	Lint must pass
Unit tests	Bazel test //...	PR opened/updated	< 5 min	All unit tests must pass
Build all targets	Bazel build //...	PR merge to main	< 8 min	All builds must succeed
Integration tests	Docker Compose test env	PR merge to main	< 15 min	All integration tests must pass
Security scan	Trivy (container) + gosec (code)	PR merge to main	< 5 min	No CRITICAL vulnerabilities
Container build + push	Docker + ECR	main merge	< 5 min	Build must succeed; Cosign signs image
Staging deploy	ArgoCD GitOps	ECR push on main	< 3 min	Rollout health check passes
Smoke tests	Playwright E2E	Staging deploy	< 10 min	All smoke tests pass
Production deploy (canary)	ArgoCD + Flagger	Staging smoke pass	10 min	5% canary: error rate < 0.1%
Production deploy (full)	Flagger progressive rollout	Canary success	30 min	Full rollout: p95 < 200ms, error < 0.1%

§4 · KERNEL

Kernel System — Zig Implementation

The Kogi Kernel is written in Zig 0.12. It is the lowest layer of the platform: a lightweight runtime supervisor that manages the module registry, event bus adapter, scheduler, provisioner, and resource allocator. It exposes a C-compatible ABI consumed by the Go API gateway via FFI.

4.1 Module Registry

Data Structures

```
// kernel/src/registry.zig

pub const ModuleStatus = enum(u8) {
    initializing = 0,
    running      = 1,
    paused       = 2,
    error_state  = 3,
    stopped      = 4,
};

pub const SemanticVersion = struct {
    major: u16,
    minor: u16,
    patch: u16,

    pub fn gte(self: SemanticVersion, other: SemanticVersion) bool {
        if (self.major != other.major) return self.major > other.major;
        if (self.minor != other.minor) return self.minor > other.minor;
        return self.patch >= other.patch;
    }
};

pub const ModuleDescriptor = struct {
    id:              [36]u8,
    name:            []const u8,
    version:         SemanticVersion,
    status:          ModuleStatus,
    health_endpoint: []const u8,
    dependencies:    []const []const u8,
    start_order:     u32,
    restart_count:   u32,
    last_heartbeat:  i64,
    error_message:   ?[]const u8,
    metadata:        std.StringHashMap([]const u8),
};

pub const Registry = struct {
    modules:   std.StringHashMap(*ModuleDescriptor),
    mu:        std.Thread.RwLock,
    allocator: std.mem.Allocator,
```

```
    pub fn register(self: *Registry, desc: ModuleDescriptor) !void {
        self.mu.lockExclusive();
        defer self.mu.unlockExclusive();
        const d = try self.allocator.create(ModuleDescriptor);
        d.* = desc;
        try self.modules.put(desc.name, d);
    }

    pub fn setStatus(self: *Registry, name: []const u8, s: ModuleStatus) !void {
        self.mu.lockExclusive();
        defer self.mu.unlockExclusive();
        if (self.modules.getPtr(name)) |m| m.*.status = s
        else return error.ModuleNotFound;
    }

    pub fn topologicalStartOrder(self: *Registry) ![][]const u8 {
        // Kahn's algorithm over dependency graph
        // Returns names in valid startup order; error on cycle
        var in_degree = std.StringHashMap(u32).init(self.allocator);
        defer in_degree.deinit();
        var it = self.modules.iterator();
        while (it.next()) |entry| {
            try in_degree.put(entry.key_ptr.*, 0);
        }
        it = self.modules.iterator();
        while (it.next()) |entry| {
            for (entry.value_ptr.*.dependencies) |dep| {
                const v = in_degree.getPtr(dep) orelse return
error.UnknownDependency;
                v.* += 1;
            }
        }
        // ... full Kahn implementation returns sorted order
        return &[_][]const u8{};
    }
};
```

4.2 Event Bus Adapter

```
// kernel/src/event_bus.zig

pub const EventEnvelope = struct {
    event_id:       [36]u8,       // UUID v4
    event_type:     []const u8,   // e.g. "portfolio.item.created"
    source_module:  []const u8,
    correlation_id: [36]u8,
    causation_id:   ?[36]u8,      // parent event
    payload:        []const u8,   // JSON-encoded
    timestamp:      i64,          // Unix nanoseconds
    schema_version: u16,
    retry_count:    u8,
```

```
    pub fn toJSON(self: EventEnvelope, alloc: std.mem.Allocator) ![]u8 {
        return std.json.stringifyAlloc(alloc, self, .{});
    }
};

pub const GlobPattern = struct {
    pattern: []const u8,

    // Matches "portfolio.*" against "portfolio.item.created" → true
    // Matches "*.order.*" against "market.order.placed" → true
    pub fn matches(self: GlobPattern, subject: []const u8) bool {
        return globMatch(self.pattern, subject);
    }
};

pub const Subscription = struct {
    id:            [36]u8,
    pattern:       GlobPattern,
    handler:       *const fn (*EventEnvelope) anyerror!void,
    consumer_name: []const u8,
    durable:       bool,
    max_in_flight: u32,
};

pub const EventBus = struct {
    conn:  *NATSConnection,
    subs:  std.ArrayList(Subscription),
    mu:    std.Thread.Mutex,

    pub fn publish(self: *EventBus, env: EventEnvelope) !void {
        const json = try env.toJSON(self.allocator);
        defer self.allocator.free(json);
        try self.conn.publish(env.event_type, json);
    }

    pub fn subscribe(self: *EventBus, sub: Subscription) !void {
        self.mu.lock(); defer self.mu.unlock();
        try self.subs.append(sub);
        try self.conn.durableSubscribe(sub.pattern.pattern, sub.consumer_name,
sub.handler);
    }
};
```

4.3 Scheduler

```
// kernel/src/scheduler.zig
// Min-heap scheduler; O(log n) insert/remove; 1-second tick resolution

pub const CronExpr = struct {
    minute: []const u8,  // "*/5" or "0" or "0,15,30,45"
    hour:   []const u8,
    dom:    []const u8,  // day-of-month
```

```
    month:  []const u8,
    dow:    []const u8,  // day-of-week

    pub fn nextAfter(self: CronExpr, ts: i64) i64 {
        // Walk forward second-by-second from ts until cron matches.
        // Returns next Unix timestamp (seconds) that satisfies expression.
        var candidate = ts + 60; // minimum 1 minute forward
        while (!self.matches(candidate)) candidate += 60;
        return candidate;
    }
};

pub const ScheduleEntry = struct {
    id:           [36]u8,
    name:         []const u8,
    cron_expr:    CronExpr,
    event_type:   []const u8,
    payload_tmpl: []const u8,
    next_run_at:  i64,
    last_run_at:  ?i64,
    enabled:      bool,
    owner_id:     [36]u8,
    miss_policy:  enum { fire_once, skip } = .fire_once,
};

fn compareNextRun(ctx: void, a: *ScheduleEntry, b: *ScheduleEntry) std.math.Order
{
    _ = ctx;
    return std.math.order(a.next_run_at, b.next_run_at);
}

pub const Scheduler = struct {
    heap: std.PriorityQueue(*ScheduleEntry, void, compareNextRun),
    bus:  *EventBus,
    mu:   std.Thread.Mutex,

    pub fn tick(self: *Scheduler, now: i64) !void {
        self.mu.lock(); defer self.mu.unlock();
        while (self.heap.peek()) |entry| {
            if (entry.next_run_at > now) break;
            _ = self.heap.remove();
            if (entry.enabled) {
                const env = buildEnvelope(entry, now);
                try self.bus.publish(env);
                entry.last_run_at = now;
            }
            entry.next_run_at = entry.cron_expr.nextAfter(now);
            try self.heap.add(entry);
        }
    }

    pub fn addEntry(self: *Scheduler, e: *ScheduleEntry) !void {
        self.mu.lock(); defer self.mu.unlock();
        try self.heap.add(e);
    }
};
```

4.4 Provisioner — New User Saga

```
// kernel/src/provisioner.zig
// Implements the saga for new-user provisioning.
// Each step has a compensating action called on partial failure.

pub const ProvisionStep = struct {
    name:       []const u8,
    execute:    *const fn (ctx: ProvisionContext) anyerror!void,
    compensate: *const fn (ctx: ProvisionContext) anyerror!void,
};

pub const ProvisionContext = struct {
    user_id:      [36]u8,
    email:        []const u8,
    plan:         []const u8,
    workspace_id: ?[36]u8 = null,
    portfolio_id: ?[36]u8 = null,
    wallet_id:    ?[36]u8 = null,
    allocator:    std.mem.Allocator,
};

const PROVISION_STEPS = [_]ProvisionStep{
    .{ .name="create_account",   .execute=createAccount,
.compensate=deleteAccount },
    .{ .name="create_workspace", .execute=createWorkspace,
.compensate=deleteWorkspace },
    .{ .name="create_portfolio", .execute=createPortfolio,
.compensate=deletePortfolio },
    .{ .name="create_wallet",    .execute=createWallet,
.compensate=deleteWallet },
    .{ .name="emit_provisioned", .execute=emitProvisioned,
.compensate=emitProvisionFailed },
};

pub fn provisionUser(ctx: *ProvisionContext) !void {
    var completed: usize = 0;
    for (PROVISION_STEPS, 0..) |step, i| {
        step.execute(ctx) catch |err| {
            // Compensate completed steps in reverse order
            var j: usize = i;
            while (j > 0) {
                j -= 1;
                PROVISION_STEPS[j].compensate(ctx) catch {};
            }
            return err;
        };
        completed = i + 1;
    }
}
```

4.5 FFI Interface — C ABI for Go Gateway

```
// kernel/src/ffi.zig — C-compatible exports consumed by Go via cgo

export fn kogi_kernel_init(config_json: [*:0]const u8) i32 { ... }
export fn kogi_kernel_health() [*:0]const u8 { ... }       // returns JSON
export fn kogi_kernel_shutdown() void { ... }

export fn kogi_module_register(desc_json: [*:0]const u8, len: usize) i32 { ... }
export fn kogi_module_status(name: [*:0]const u8) u8 { ... }
export fn kogi_module_deregister(name: [*:0]const u8) i32 { ... }

export fn kogi_event_publish(envelope_json: [*:0]const u8, len: usize) i32 { ... }
export fn kogi_event_subscribe(pattern: [*:0]const u8,
                               consumer: [*:0]const u8,
                               handler_id: u64) i32 { ... }

export fn kogi_schedule_add(entry_json: [*:0]const u8, len: usize) i32 { ... }
export fn kogi_schedule_remove(id: [*:0]const u8) i32 { ... }
export fn kogi_schedule_pause(id: [*:0]const u8) i32 { ... }
export fn kogi_schedule_resume(id: [*:0]const u8) i32 { ... }

export fn kogi_provision_user(user_json: [*:0]const u8, len: usize) i32 { ... }

// Error codes returned by all FFI functions:
// 0  = success
// -1 = invalid JSON input
// -2 = module not found
// -3 = duplicate registration
// -4 = provisioning failed (compensations executed)
// -5 = internal error (check kogi_last_error())
export fn kogi_last_error() [*:0]const u8 { ... }
```

4.6 Kernel Observability

Metric	Type	Description
kernel_module_status	Gauge per module	0=init 1=running 2=paused 3=error 4=stopped
kernel_module_restarts_total	Counter	Cumulative restarts per module
kernel_event_publish_total	Counter	Events published, labeled by event_type
kernel_event_publish_errors_total	Counter	Publish failures by event_type and error
kernel_scheduler_ticks_total	Counter	Scheduler tick executions
kernel_scheduler_fired_total	Counter	Scheduled events fired by entry name
kernel_scheduler_missed_total	Counter	Ticks where next_run_at was in the past by > 60s (recovery event)

Metric	Type	Description
kernel_provision_total	Counter	Provisioning calls labeled outcome: success/failed/compensated
kernel_ffi_call_duration_ms	Histogram	FFI call latency per function name

§5 · AUTH

Authentication & Session System

The Authentication System handles identity verification, session lifecycle, MFA, OAuth integrations, and permission token generation. It is deliberately kept minimal — it proves identity but does not encode authorization rules, which are service-local.

5.1 Database Schema

Table: account

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	Primary key — UUID v4
email	VARCHAR(320)	NOT NULL	—	Unique, lowercase-normalized, indexed
email_verified	BOOLEAN	NOT NULL	false	Set true after email verification flow
password_hash	VARCHAR(256)	NOT NULL	—	Argon2id: time=3 memory=65536 threads=4
status	VARCHAR(20)	NOT NULL	'active'	Enum: active|suspended|deactivated|deleted
plan	VARCHAR(20)	NOT NULL	'free'	Enum: free|starter|pro|team|cooperative|enterprise
plan_expires_at	TIMESTAMPTZ	NULL	NULL	NULL = indefinite (annual billing)
mfa_enabled	BOOLEAN	NOT NULL	false	—
mfa_type	VARCHAR(10)	NULL	NULL	Enum: totp|sms

Column	Type	Null	Default	Description
mfa_secret	VARCHAR(256)	NULL	NULL	KMS-encrypted TOTP secret
mfa_backup_codes	JSONB	NULL	NULL	Array of hashed 8-char backup codes (10 total)
failed_login_count	INTEGER	NOT NULL	0	Reset on successful login; lockout at 10
locked_until	TIMESTAMPTZ	NULL	NULL	Account locked until this time after 10 failures
password_reset_token	CHAR(64)	NULL	NULL	SHA-256 of reset token; expires via reset_token_expires_at
password_reset_expires_at	TIMESTAMPTZ	NULL	NULL	Token valid for 15 minutes
timezone	VARCHAR(64)	NOT NULL	'UTC'	IANA tz string
locale	VARCHAR(10)	NOT NULL	'en-US'	BCP-47 language tag
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	Trigger-maintained
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete

Table: session

Column	Type	Null	Default	Description
id	CHAR(36)	NOT	gen_uuid()	Session UUID; stored as jti claim in JWT

Column	Type	Null	Default	Description
		NULL		
account_id	CHAR(36)	NOT NULL	—	FK → account.id
refresh_token_hash	CHAR(64)	NOT NULL	—	SHA-256 of opaque refresh token
device_fingerprint	VARCHAR(256)	NULL	NULL	Browser/device fingerprint for anomaly detection
user_agent	VARCHAR(512)	NULL	NULL	—
ip_address	INET	NULL	NULL	IP at session creation; used for anomaly detection
country_code	CHAR(2)	NULL	NULL	GeoIP lookup at creation
access_expires_at	TIMESTAMPTZ	NOT NULL	—	Access token window: 24h from creation
refresh_expires_at	TIMESTAMPTZ	NOT NULL	—	Refresh token window: 30d
revoked_at	TIMESTAMPTZ	NULL	NULL	Explicit logout or security revocation
revoke_reason	VARCHAR(50)	NULL	NULL	Enum: logout|password_change|security|admin|inactivity
last_active_at	TIMESTAMPTZ	NOT NULL	NOW()	Updated on each access token refresh
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: oauth_connection

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
account_id	CHAR(36)	NOT NULL	—	FK → account.id
provider	VARCHAR(20)	NOT NULL	—	Enum: google|github|slack|notion|linear
provider_user_id	VARCHAR(256)	NOT NULL	—	Provider-specific user identifier
access_token	TEXT	NULL	NULL	KMS-encrypted; NULL if token expired and not refreshed
refresh_token	TEXT	NULL	NULL	KMS-encrypted
scopes	JSONB	NOT NULL	'[]'	Array of granted scopes
expires_at	TIMESTAMPTZ	NULL	NULL	NULL = no expiry (non-expiring tokens)
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: Auth Schema

Index Name	Type	Columns	Purpose
idx_account_email	UNIQUE	account(lower(email))	Login lookup — case-insensitive
idx_account_status_plan	BTREE	account(status, plan)	Active user queries, plan analytics
idx_account_deleted	PARTIAL	account(id) WHERE deleted_at IS NULL	Soft-delete filter for all queries

Index Name	Type	Columns	Purpose
idx_session_account	BTREE	session(account_id, last_active_at DESC)	List sessions per user, latest first
idx_session_refresh	BTREE	session(refresh_token_hash)	Refresh token lookup
idx_session_active	PARTIAL	session(access_expires_at) WHERE revoked_at IS NULL	Expiry sweep
idx_oauth_account_provider	UNIQUE	oauth_connection(account_id, provider)	One connection per provider per account
idx_oauth_provider_uid	UNIQUE	oauth_connection(provider, provider_user_id)	OAuth callback lookup

5.2 Authentication Flows

Email/Password Registration

```
// auth/service/register.go

func (s *AuthService) Register(ctx context.Context, req RegisterRequest)
(*Account, error) {
    // 1. Normalize email
    email := strings.ToLower(strings.TrimSpace(req.Email))

    // 2. Check uniqueness
    exists, err := s.repo.AccountExistsByEmail(ctx, email)
    if err != nil { return nil, wrapDB(err) }
    if exists { return nil, ErrEmailAlreadyRegistered }

    // 3. Validate password strength (zxcvbn score >= 3)
    if score := zxcvbn.PasswordStrength(req.Password, []string{email});
score.Score < 3 {
        return nil, ErrPasswordTooWeak
    }

    // 4. Hash password (Argon2id)
    hash, err := argon2id.CreateHash(req.Password, &argon2id.Params{
        Memory: 65536, Iterations: 3, Parallelism: 4, SaltLength: 16, KeyLength:
32,
    })
    if err != nil { return nil, wrapInternal(err) }

    // 5. Create account in transaction with outbox event
    account := &Account{
        ID: uuid.New().String(), Email: email, PasswordHash: hash,
        Plan: "free", Status: "active",
    }
    if err := s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        if err := s.repo.InsertAccount(tx, account); err != nil { return err }
        return s.outbox.Insert(tx, "user.registered", UserRegisteredPayload{
```

```
            UserID: account.ID, Email: email, Plan: "free",
        })
    }); err != nil { return nil, err }

    // 6. Send verification email (async, via notification event)
    _ = s.sendVerificationEmail(ctx, account.ID, email)
    return account, nil
}
```

JWT Token Architecture

```
// JWT Access Token — 24h lifetime
{
  "sub":   "user-uuid",          // account ID
  "jti":   "session-uuid",       // jti = session ID for O(1) revocation check
  "iat":   1700000000,
  "exp":   1700086400,            // iat + 86400 seconds
  "plan":  "pro",                 // plan cached for feature gating without DB hit
  "ws":    "workspace-uuid",      // active workspace
  "roles": ["user"],              // platform roles: user|moderator|admin
  "ao":    ["ao-uuid-1"],         // AO memberships (max 20; trimmed if more)
  "v":     1                      // token schema version
}

// Signing: HMAC-SHA256, 256-bit secret, rotated every 90 days
// Rotation: new key active + old key valid for 24h (overlap = max token lifetime)

// Validation pipeline in JWT middleware:
// 1. Parse header — reject unknown alg
// 2. Verify signature with current key; if fail, try previous key (rotation
window)
// 3. Check exp > now()
// 4. Check jti NOT IN Redis revocation set (SET lookup, O(1))
//    Redis key: "revoked:{jti}" — TTL = exp - now (auto-cleans)
// 5. Inject claims into gin.Context
```

MFA — TOTP Flow

```
// MFA enrollment
func (s *AuthService) EnrollTOTP(ctx context.Context, accountID string)
(*TOTPEnrollment, error) {
    // 1. Generate 160-bit secret
    secret := totp.GenerateOpts{Issuer: "Kogi", AccountName: email}
    key, _ := totp.Generate(secret)

    // 2. Encrypt secret with KMS before storing
    encrypted, _ := s.kms.Encrypt(ctx, key.Secret())

    // 3. Store as pending (not yet verified)
    _ = s.repo.SetPendingMFASecret(ctx, accountID, encrypted)
```

```
    // 4. Return QR code URL + backup codes (plaintext, ONE TIME)
    backupCodes := generateBackupCodes(10) // 10 × 8-char random codes
    return &TOTPEnrollment{
        QRCodeURL:   key.URL(),
        BackupCodes: backupCodes,
    }, nil
}

// MFA verification (completes enrollment)
func (s *AuthService) VerifyTOTP(ctx, accountID, code string) error {
    secret, _ := s.repo.GetPendingMFASecret(ctx, accountID)
    decrypted, _ := s.kms.Decrypt(ctx, secret)
    if !totp.Validate(code, decrypted) { return ErrInvalidTOTPCode }
    // Activate MFA; hash and store backup codes
    return s.repo.ActivateMFA(ctx, accountID, decrypted,
hashBackupCodes(backupCodes))
}
```

5.3 OAuth Integration

Provider	Scopes Requested	Data Retrieved	Account Link Logic
Google	openid email profile	Email, name, avatar, Google ID	Match by email if exists; create account if not; store oauth_connection
GitHub	user:email read:user	Email, username, avatar, GitHub ID	Match by email; fallback to GitHub ID lookup; create if new
Slack	users:read	Email, display name, Slack workspace ID	Match by email; used for workspace team integrations, not primary auth
LinkedIn	r_liteprofile r_emailaddress	Email, name, headline, LinkedIn URL	Match by email; used for profile enrichment; not primary auth

5.4 Session Security Controls

Control	Implementation	Threshold
Brute force lockout	failed_login_count incremented on each failure; account locked at 10 failures	10 failures → 15-min lockout; 15 failures → 1-hour lockout; 20 failures → 24-hour lockout
Concurrent session limit	Active session count checked on new login; Pro users: 5 max; Free: 2 max	Oldest session revoked on limit breach (least-recently-used)

Control	Implementation	Threshold
Anomalous login detection	IP country code compared to last-5 logins; new country triggers email alert	Alert sent; session still created (soft detection only; not blocked by default)
Inactivity timeout	Sessions with last_active_at > 90 days swept by nightly job and revoked	Pro: 90d; Starter: 60d; Free: 30d
Refresh token rotation	New refresh token issued on each access token refresh; old token immediately invalidated	Replay of old refresh token → revoke all sessions for account (security incident assumed)

5.5 Auth Service API

Endpoint	Method	Auth	Description
POST /api/v1/auth/register	Public	—	Create new account; sends verification email
POST /api/v1/auth/login	Public	Strict	Email/password login; returns access + refresh tokens
POST /api/v1/auth/login/mfa	Public	Strict	Submit TOTP code after password verified; completes login
POST /api/v1/auth/refresh	Public	—	Exchange refresh token for new access token
POST /api/v1/auth/logout	JWT	—	Revoke current session
POST /api/v1/auth/logout/all	JWT	—	Revoke all sessions for account
POST /api/v1/auth/password/reset	Public	Strict	Request password reset email
POST /api/v1/auth/password/confirm	Public	—	Confirm password reset with token
PUT /api/v1/auth/password	JWT	—	Change password (requires current password)
POST /api/v1/auth/mfa/enroll	JWT	—	Begin TOTP enrollment; returns QR code and backup codes
POST /api/v1/auth/mfa/verify	JWT	—	Verify TOTP code to complete enrollment
DELETE /api/v1/auth/mfa	JWT	—	Disable MFA (requires TOTP code as confirmation)
GET /api/v1/auth/sessions	JWT	—	List all active sessions for account
DELETE /api/v1/auth/sessions/:id	JWT	—	Revoke a specific session
GET /api/v1/auth/oauth/:provider	Public	—	Initiate OAuth flow; returns redirect URL
GET /api/v1/auth/oauth/:provider/cb	Public	—	OAuth callback; completes login or account link

5.6 Auth Events Emitted

Event	Payload	Consumers
user.registered	user_id, email, plan, created_at	Kernel (provision saga), Analytics, Email (verification)
user.email_verified	user_id, email, verified_at	Analytics, Email (welcome series)
user.login	user_id, session_id, ip, country, mfa_used	Analytics, Security (anomaly detection)
user.logout	user_id, session_id, reason	Analytics
user.session_revoked	user_id, session_id, all_sessions, reason	API Gateway (Redis revocation SET), Analytics
user.mfa_enabled	user_id, mfa_type	Analytics, Email (confirmation)
user.mfa_disabled	user_id	Analytics, Email (security alert)
user.password_changed	user_id	All sessions revoked, Email (security alert)
user.plan_changed	user_id, old_plan, new_plan, effective_at	All services (feature gates), Analytics, Email
user.account_locked	user_id, locked_until, reason	Security alerting, Email

§6 · USERS

User, Profile & Workspace System

6.1 Database Schema

Table: profile

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
account_id	CHAR(36)	NOT NULL	—	FK → account.id, UNIQUE
display_name	VARCHAR(100)	NOT NULL	—	—
handle	VARCHAR(50)	NOT NULL	—	Unique URL-safe handle (@jordan)
bio	TEXT	NULL	NULL	Max 500 chars; Markdown allowed
tagline	VARCHAR(160)	NULL	NULL	One-line professional tagline
avatar_url	VARCHAR(512)	NULL	NULL	CDN URL; uploaded via /media endpoint
banner_url	VARCHAR(512)	NULL	NULL	Profile banner image CDN URL
location	VARCHAR(100)	NULL	NULL	City, Country — free text
website_url	VARCHAR(512)	NULL	NULL	Personal or business URL
skills	JSONB	NOT NULL	'[]'	Array: {name,level:1-5,verified_by?}
social_links	JSONB	NOT	'[]'	Array: {platform,url}

Column	Type	Null	Default	Description
		NULL		
languages	JSONB	NOT NULL	'[]'	Array: {code,level:native|fluent|conversational}
availability_status	VARCHAR(30)	NOT NULL	'available'	Enum: available|busy|not_taking_work|open_to_opps
hourly_rate_usd	DECIMAL(8,2)	NULL	NULL	Indicative hourly rate for market visibility
reputation_score	DECIMAL(5,1)	NOT NULL	0.0	Computed nightly (0–100); see §9 algorithm
review_count	INTEGER	NOT NULL	0	Denormalized from market.review table
is_public	BOOLEAN	NOT NULL	true	Whether profile visible to non-connections
is_verified_worker	BOOLEAN	NOT NULL	false	Platform-verified independent worker badge
portfolio_share_url	VARCHAR(64)	NULL	NULL	Public share token for portfolio overview
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: workspace

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	FK → account.id
name	VARCHAR(200)	NOT NULL	—	Default: "My Workspace" on provisioning
slug	VARCHAR(100)	NOT NULL	—	Unique per owner; URL-safe
workspace_type	VARCHAR(20)	NOT NULL	'personal'	Enum: personal|team|cooperative|enterprise
plan	VARCHAR(20)	NOT NULL	'free'	Mirrors account.plan; updated by plan_changed event
settings	JSONB	NOT NULL	'{}'	UI settings, feature toggles, notification prefs
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: workspace_member

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
user_id	CHAR(36)	NOT NULL	—	FK → account.id
role	VARCHAR(20)	NOT NULL	—	Enum: owner|admin|member|guest
invited_by	CHAR(36)	NOT NULL	—	FK → account.id
accepted_at	TIMESTAMPTZ	NULL	NULL	NULL = pending invitation
expires_at	TIMESTAMPTZ	NULL	NULL	NULL = indefinite
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

6.2 RBAC Permission Model

Action	Owner	Admin	Member	Guest	System/AI
View private portfolio items	✅	✅	✅	❌	✅
Create portfolio items	✅	✅	✅	❌	✅
Delete portfolio items	✅	✅	❌	❌	❌
Manage workspace members	✅	✅	❌	❌	❌
Create WBS/stories	✅	✅	✅	❌	✅
Complete/close stories	✅	✅	✅	❌	✅
Access Exchange/wallet	✅	❌	❌	❌	❌
Create marketplace listings	✅	✅	❌	❌	❌

Action	Owner	Admin	Member	Guest	System/AI
Submit governance proposals	✅	✅	✅	❌	❌
Vote on governance proposals	✅	✅	✅	❌	❌
View analytics	✅	✅	✅	❌	✅
Export data	✅	✅	❌	❌	❌

6.3 Profile Service — Key Operations

```
// user-service/internal/service/profile.go

type ProfileService interface {
    CreateProfile(ctx, accountID string, req CreateProfileReq) (*Profile, error)
    GetProfile(ctx, id, viewerID string) (*Profile, error)
    GetProfileByHandle(ctx, handle, viewerID string) (*Profile, error)
    UpdateProfile(ctx, id string, req UpdateProfileReq, version int) (*Profile,
error)
    UpdateAvailability(ctx, id string, status AvailabilityStatus) error
    GetPublicPortfolioShare(ctx, shareToken string) (*PublicPortfolioView, error)
    GeneratePortfolioShareURL(ctx, ownerID string) (string, error)
    SearchProfiles(ctx, query string, filters ProfileFilters, page Pagination)
([]*Profile, int, error)
}

// Profile search uses Elasticsearch for full-text; filters are applied in ES
query DSL.
// Indexed fields: display_name, tagline, bio, skills[].name, location, handle
// Filter fields: availability_status, hourly_rate_usd (range), skills (terms),
location (geo)
```

6.4 Workspace Provisioning

```
// Triggered by user.provisioned kernel event
func (s *UserService) HandleUserProvisioned(ctx context.Context, evt
UserProvisionedEvent) error {
    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        // Create default workspace
        ws := &Workspace{
            ID: evt.WorkspaceID, OwnerID: evt.UserID,
            Name: "My Workspace", Slug: generateSlug(evt.UserID),
            WorkspaceType: "personal", Plan: "free",
        }
        if err := s.wsRepo.Insert(tx, ws); err != nil { return err }

        // Add owner as workspace member
        if err := s.wsRepo.AddMember(tx, ws.ID, evt.UserID, "owner", evt.UserID);
err != nil { return err }
```

```
        // Create default profile stub
        profile := &Profile{
            ID: uuid.New().String(), AccountID: evt.UserID,
            DisplayName: "Kogi User", Handle: generateHandle(evt.UserID),
        }
        if err := s.profileRepo.Insert(tx, profile); err != nil { return err }

        return s.outbox.Insert(tx, "user.workspace_created",
WorkspaceCreatedPayload{
            UserID: evt.UserID, WorkspaceID: ws.ID,
        })
    })
}
```

§7 · PORTFOLIO

Portfolio System

The Portfolio System is the universal organizational layer of Kogi. Every resource, project, and goal is anchored to a portfolio. It provides the hierarchical structure that gives the AI Agent and analytics systems their context.

7.1 Database Schema

Table: portfolio

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
owner_type	VARCHAR(10)	NOT NULL	—	Enum: user|ao
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
name	VARCHAR(200)	NOT NULL	—	—
description	TEXT	NULL	NULL	Markdown
is_root	BOOLEAN	NOT NULL	false	True for auto-provisioned root portfolio
visibility	VARCHAR(20)	NOT NULL	'private'	Enum: private|team|org|public
cover_color	CHAR(7)	NOT	'#2563EB'	CSS hex color for UI theming

Column	Type	Null	Default	Description
		NULL		
tags	JSONB	NOT NULL	'[]'	String array
metadata	JSONB	NOT NULL	'{}'	Custom key-value pairs
health_score	SMALLINT	NULL	NULL	0–100; AI-computed; refreshed daily
health_factors	JSONB	NULL	NULL	Breakdown: {items,resources,goals,recency,risk}
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: portfolio_item

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
portfolio_id	CHAR(36)	NOT NULL	—	FK → portfolio.id
parent_id	CHAR(36)	NULL	NULL	FK → portfolio_item.id — nesting

Column	Type	Null	Default	Description
item_type	VARCHAR(30)	NOT NULL	—	Enum: program|project|sub_portfolio|resource|container
name	VARCHAR(200)	NOT NULL	—	—
description	TEXT	NULL	NULL	—
status	VARCHAR(30)	NOT NULL	'planning'	Enum: planning|active|on_hold|complete|cancelled
visibility	VARCHAR(20)	NOT NULL	'private'	—
priority	VARCHAR(10)	NOT NULL	'medium'	Enum: critical|high|medium|low
start_date	DATE	NULL	NULL	—
end_date	DATE	NULL	NULL	—
tags	JSONB	NOT NULL	'[]'	—
metadata	JSONB	NOT NULL	'{}'	Type-specific fields (JSON Schema validated per item_type)
attachments	JSONB	NOT NULL	'[]'	Array of {filename,url,content_type,size_bytes}
linked_wbs_id	CHAR(36)	NULL	NULL	FK → wbs.id when item_type=project
linked_ao_id	CHAR(36)	NULL	NULL	FK → ao.id when item_type=program (cooperative)
sort_order	FLOAT	NOT	0	Fractional index (1000, 2000, 1500…) for ordering

Column	Type	Null	Default	Description
		NULL		
version	INTEGER	NOT NULL	0	—
created_by	CHAR(36)	NOT NULL	—	FK → account.id
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: resource (extends portfolio_item)

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	—	PK; also FK → portfolio_item.id (1:1)
resource_type	VARCHAR(30)	NOT NULL	—	Enum: artifact|asset|capital|land|estate|labor|investment|account
quantity	DECIMAL(18,8)	NULL	NULL	Numeric amount (e.g. 1500.00 USD, 50 shares)
unit	VARCHAR(30)	NULL	NULL	USD|shares|pct|hours|sqft|acres|kg|custom
valuation_usd	DECIMAL(18,2)	NULL	NULL	Current estimated value in USD
cost_basis_usd	DECIMAL(18,2)	NULL	NULL	Original cost (for investments/assets)
acquisition_date	DATE	NULL	NULL	—

Column	Type	Null	Default	Description
depreciation	JSONB	NULL	NULL	{method, rate, salvage_value, schedule[]}
external_id	VARCHAR(256)	NULL	NULL	Reference to external system ID
resource_meta	JSONB	NOT NULL	'{}'	Type-specific fields validated by JSON schema

Table: portfolio_collaborator

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
portfolio_id	CHAR(36)	NOT NULL	—	FK → portfolio.id
user_id	CHAR(36)	NOT NULL	—	FK → account.id
role	VARCHAR(20)	NOT NULL	—	Enum: owner|admin|member|viewer
invited_by	CHAR(36)	NOT NULL	—	—
can_share	BOOLEAN	NOT NULL	false	Whether collaborator can invite others
expires_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: Portfolio Schema

Index Name	Type	Columns	Purpose
idx_portfolio_owner_ws	BTREE	portfolio(owner_id, workspace_id) WHERE deleted_at IS NULL	Owner portfolio list
idx_item_portfolio_type	BTREE	portfolio_item(portfolio_id, item_type) WHERE deleted_at IS NULL	Items by type
idx_item_parent	BTREE	portfolio_item(parent_id) WHERE deleted_at IS NULL	Hierarchy children
idx_item_status	BTREE	portfolio_item(status, item_type)	Dashboard status filters
idx_item_wbs	UNIQUE	portfolio_item(linked_wbs_id) WHERE linked_wbs_id IS NOT NULL	WBS→item lookup
idx_item_tags	GIN	portfolio_item(tags)	Tag-based filtering
idx_item_metadata	GIN	portfolio_item(metadata)	JSON field search
idx_item_sort	BTREE	portfolio_item(portfolio_id, parent_id, sort_order)	Ordered display
idx_resource_type_valuation	BTREE	resource(resource_type, valuation_usd DESC)	Resource analytics
idx_collab_unique	UNIQUE	portfolio_collaborator(portfolio_id, user_id)	No duplicate collaborators

7.2 Health Score Algorithm

ℹ️ **Portfolio Health Score — Computation**

- Input: portfolio_id. Runs daily via background job or on-demand via AI Agent.
- Item Health (30%): active_pct×1.0 + planning_pct×0.7 − overdue_pct×2.0 − cancelled_pct×0.5. Clamped [0,1].
- Resource Utilization (20%): For capital: used_budget/total_budget. For labor: hours_logged/capacity. Average across types.
- Goal Progress (30%): Average of linked OKR scores from work-service. Falls back to story completion rate if no OKRs.
- Activity Recency (10%): decay = e^(−days_since_last_update / 30).
- Risk Penalty (10%): −0.15 per critical risk story, −0.08 per high, −0.04 per medium. Floor: −0.50.
- Final = 100 × clamp(0,1, Σ weighted components). Written to portfolio.health_score.

7.3 Portfolio Tree — Recursive Query

```
-- portfolio_item tree traversal (PostgreSQL recursive CTE)
WITH RECURSIVE item_tree AS (
    -- Anchor: root items (no parent)
    SELECT id, parent_id, name, item_type, status, sort_order, 0 AS depth
    FROM portfolio_item
    WHERE portfolio_id = $1 AND parent_id IS NULL AND deleted_at IS NULL

    UNION ALL

    -- Recursive: children of already-selected items
    SELECT pi.id, pi.parent_id, pi.name, pi.item_type, pi.status, pi.sort_order,
it.depth + 1
    FROM portfolio_item pi
    INNER JOIN item_tree it ON pi.parent_id = it.id
    WHERE pi.deleted_at IS NULL
)
SELECT * FROM item_tree
ORDER BY depth, parent_id NULLS FIRST, sort_order;

-- Depth limit: max 6 levels to prevent infinite recursion
-- Go service converts flat result to nested tree structure in O(n)
```

7.4 Portfolio API

Endpoint	Method	Auth	Description
POST /api/v1/portfolios	JWT	—	Create portfolio
GET /api/v1/portfolios	JWT	—	List portfolios for current user
GET /api/v1/portfolios/:id	JWT	—	Get portfolio (optional ?include=items,health)
GET /api/v1/portfolios/:id/tree	JWT	—	Full recursive item tree
PUT /api/v1/portfolios/:id	JWT	Idem	Update portfolio metadata
DELETE /api/v1/portfolios/:id	JWT	—	Soft delete portfolio
POST /api/v1/portfolios/:id/items	JWT	Idem	Create portfolio item
GET /api/v1/portfolios/:id/items	JWT	—	List items (flat, with optional filters)
PUT /api/v1/portfolios/:id/items/:itemId	JWT	Idem	Update item
PUT /api/v1/portfolios/:id/items/:itemId/move	JWT	—	Move item (new parent_id + sort_order)
DELETE /api/v1/portfolios/:id/items/:itemId	JWT	—	Soft delete item
GET /api/v1/portfolios/:id/health	JWT	—	Get/recompute health score
POST /api/v1/portfolios/:id/share	JWT	—	Generate public share URL

Endpoint	Method	Auth	Description
GET /api/v1/p/:shareToken	Public	—	View public portfolio share

7.5 Portfolio Events

Event Type	Source	Key Payload Fields	Consumers
portfolio.created	portfolio-svc	portfolio_id, owner_id, owner_type, name	Analytics, Community (if public), AI Agent
portfolio.item.created	portfolio-svc	item_id, portfolio_id, item_type, name, parent_id?	WBS (project→wbs create), Analytics, AI Agent (context update)
portfolio.item.status_changed	portfolio-svc	item_id, from_status, to_status, actor_id	Work (OKR refresh), Analytics, AI Agent
portfolio.health_score_updated	portfolio-svc	portfolio_id, score, prev_score, factors	AI Agent (alert if delta > 10), Analytics
portfolio.resource.updated	portfolio-svc	resource_id, resource_type, field_changes	Analytics, Exchange (capital sync)
portfolio.item.deleted	portfolio-svc	item_id, portfolio_id, item_type	WBS (cascade soft-delete wbs), Analytics

§8 · WBS

WBS & Story System

The WBS (Work Breakdown Structure) System is the execution layer of Kogi. It decomposes portfolio projects into executable stories and tasks, tracks progress, manages sprints, and provides the data layer for OKR alignment and marketplace integration.

8.1 Database Schema

Table: wbs

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
portfolio_item_id	CHAR(36)	NOT NULL	—	FK → portfolio_item.id; UNIQUE — 1 WBS per project
name	VARCHAR(200)	NOT NULL	—	—
execution_mode	VARCHAR(20)	NOT NULL	'kanban'	Enum: agile|kanban|program|routine|research|ops
current_sprint_id	CHAR(36)	NULL	NULL	FK → sprint.id — active sprint
default_story_type	VARCHAR(40)	NOT NULL	'feature'	Pre-filled on new story creation
estimation_method	VARCHAR(20)	NOT NULL	'points'	Enum: points|hours|t_shirt|none
kanban_columns	JSONB	NOT NULL	—	[{id,name,status_mapping,wip_limit?}]
velocity_avg	DECIMAL(5,2)	NULL	NULL	Rolling 6-sprint average; updated on sprint.closed

Column	Type	Null	Default	Description
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: story

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wbs_id	CHAR(36)	NOT NULL	—	FK → wbs.id
parent_id	CHAR(36)	NULL	NULL	FK → story.id (hierarchy nesting)
hierarchy_level	SMALLINT	NOT NULL	5	1=WorkPackage 2=Theme 3=Initiative 4=Epic 5=Story 6=Task
story_type	VARCHAR(40)	NOT NULL	—	30+ types — see full enum below
title	VARCHAR(500)	NOT NULL	—	Elasticsearch indexed
description	TEXT	NULL	NULL	Markdown; max 100KB
acceptance_criteria	TEXT	NULL	NULL	Required before DONE transition
status	VARCHAR(20)	NOT	'backlog'	State machine controlled

Column	Type	Null	Default	Description
		NULL		
priority	VARCHAR(10)	NOT NULL	'medium'	critical|high|medium|low
story_points	SMALLINT	NULL	NULL	NULL if estimation_method=none
time_estimate_hours	DECIMAL(5,2)	NULL	NULL	Used if estimation_method=hours
time_logged_hours	DECIMAL(5,2)	NOT NULL	0	Time tracking; updated via time_entry
assignee_id	CHAR(36)	NULL	NULL	User UUID
reviewer_id	CHAR(36)	NULL	NULL	Reviewer for IN_REVIEW stage
executor_type	VARCHAR(20)	NOT NULL	'human'	ai_agent|human|hybrid|machine|custom
executor_config	JSONB	NOT NULL	'{}'	Executor-specific settings
sprint_id	CHAR(36)	NULL	NULL	FK → sprint.id
labels	JSONB	NOT NULL	'[]'	String array
linked_stories	JSONB	NOT NULL	'[]'	[{story_id, link_type: blocks|relates|duplicates|child_of}]
linked_resource_ids	JSONB	NOT NULL	'[]'	Portfolio resource UUIDs
linked_order_id	CHAR(36)	NULL	NULL	FK → market_order.id (auto-linked on order.placed)
due_date	DATE	NULL	NULL	—

Column	Type	Null	Default	Description
started_at	TIMESTAMPTZ	NULL	NULL	Set on IN_PROGRESS transition
completed_at	TIMESTAMPTZ	NULL	NULL	Set on DONE transition
blocked_since	TIMESTAMPTZ	NULL	NULL	Set on BLOCKED transition
blocker_note	TEXT	NULL	NULL	Required on BLOCKED transition
blocker_story_id	CHAR(36)	NULL	NULL	FK → story.id if blocked by a dependency
ai_summary	TEXT	NULL	NULL	AI-generated summary; refreshed on description change
ai_suggestions	JSONB	NULL	NULL	AI-generated next steps or related resources
sort_order	FLOAT	NOT NULL	0	Fractional index within parent/column
cancellation_reason	VARCHAR(200)	NULL	NULL	Required on CANCELLED transition
version	INTEGER	NOT NULL	0	Optimistic lock
created_by	CHAR(36)	NOT NULL	—	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Story Type Enum (30+ types)

Category	Story Types
Execution	feature, bug, defect, hotfix, task, chore, test, documentation

Category	Story Types
Planning	strategy, tactic, operation, requirement, enabler, capability
Innovation	idea, concept, prototype, spike, research, design
Management	milestone, objective, outcome, goal, mission, vision, business_case
Risk & Quality	risk, audit, review, performance, compliance, issue, blocker
Governance	report, policy

Table: sprint

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wbs_id	CHAR(36)	NOT NULL	—	FK → wbs.id
sprint_number	SMALLINT	NOT NULL	—	Auto-incremented per WBS
goal	VARCHAR(500)	NULL	NULL	Sprint goal statement
status	VARCHAR(20)	NOT NULL	'planning'	planning|active|review|closed
start_date	DATE	NOT NULL	—	—
end_date	DATE	NOT NULL	—	—
planned_points	SMALLINT	NOT NULL	0	—
completed_points	SMALLINT	NOT NULL	0	Updated by trigger on story DONE

Column	Type	Null	Default	Description
velocity	DECIMAL(5,2)	NULL	NULL	completed_points/planned_points at close
ai_plan_used	BOOLEAN	NOT NULL	false	—
ai_plan_confidence	DECIMAL(3,2)	NULL	NULL	—
retro_story_id	CHAR(36)	NULL	NULL	FK → story.id (Review story type)
closed_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: story_status_history

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK — append only
story_id	CHAR(36)	NOT NULL	—	FK → story.id
from_status	VARCHAR(20)	NOT NULL	—	—
to_status	VARCHAR(20)	NOT NULL	—	—
actor_id	CHAR(36)	NOT NULL	—	User or AI Agent UUID
actor_type	VARCHAR(20)	NOT NULL	—	user|ai_agent|system|automation

Column	Type	Null	Default	Description
note	TEXT	NULL	NULL	Optional transition comment
duration_in_state_minutes	INTEGER	NULL	NULL	Time spent in from_status
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: time_entry

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
story_id	CHAR(36)	NOT NULL	—	FK → story.id
user_id	CHAR(36)	NOT NULL	—	—
hours	DECIMAL(5,2)	NOT NULL	—	Must be > 0 and ≤ 24
logged_date	DATE	NOT NULL	—	—
note	TEXT	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: WBS Schema

Index Name	Type	Columns	Purpose
idx_story_wbs_status	BTREE	story(wbs_id, status) WHERE deleted_at IS NULL	Kanban board query — primary read
idx_story_parent	BTREE	story(parent_id) WHERE deleted_at IS NULL	Hierarchy traversal
idx_story_sprint	BTREE	story(sprint_id, status) WHERE sprint_id IS NOT NULL	Sprint story list
idx_story_assignee	BTREE	story(assignee_id, status)	My tasks view
idx_story_due_date	BTREE	story(due_date) WHERE due_date IS NOT NULL	Overdue detection job
idx_story_blocked_since	BTREE	story(blocked_since) WHERE status='blocked'	Blocked story escalation
idx_story_labels	GIN	story(labels)	Label search
idx_story_links	GIN	story(linked_stories)	Dependency graph queries
idx_story_sort	BTREE	story(wbs_id, parent_id, sort_order)	Ordered display
idx_sprint_wbs_status	BTREE	sprint(wbs_id, status)	Active sprint lookup
idx_history_story_ts	BTREE	story_status_history(story_id, created_at)	Story timeline / cycle time
idx_time_story	BTREE	time_entry(story_id, logged_date)	Time tracking rollup

8.2 Story Status State Machine

State	Entry Actions	Valid Transitions	Guard Conditions
BACKLOG	Insert status_history; set sort_order=auto-append; if sprint_id set, increment planned_points	→ READY→ CANCELLED	story_type valid; title NOT NULL; wbs_id valid
READY	Notify assignee if set; if sprint_id set, mark sprint story as committed	→ IN_PROGRESS→ BACKLOG→ CANCELLED	acceptance_criteria NOT NULL; assignee_id NOT NULL (or executor_type=ai_agent)
IN_PROGRESS	Set started_at; increment sprint WIP counter; emit wbs.story.started	→ IN_REVIEW→ BLOCKED→ BACKLOG→ CANCELLED	actor must be assignee, admin, or ai_agent
BLOCKED	Set blocked_since; emit wbs.story.blocked; AI Agent searches Market for resolution; decrement sprint WIP	→ IN_PROGRESS (on unblock)→ CANCELLED	blocker_note NOT NULL; blocker_story_id or blocker_resource_id recommended

State	Entry Actions	Valid Transitions	Guard Conditions
IN_REVIEW	Notify reviewer; start review SLA timer (24h default); emit wbs.story.submitted_for_review	→ DONE→ IN_PROGRESS (changes requested)→ BLOCKED	reviewer_id NOT NULL or owner defaults; actor must be assignee
DONE	Set completed_at; emit wbs.story.completed; update sprint completed_points; trigger OKR check-in; if linked_order_id: milestone check	(Terminal — no transitions out)	acceptance_criteria NOT NULL; reviewer must be different from assignee (or AI auto-approve for ai_agent executor); all child tasks must be DONE
CANCELLED	Emit wbs.story.cancelled; remove from sprint if assigned; emit sprint.replanning_needed if sprint is active	(Terminal)	cancellation_reason NOT NULL; admin or owner required if was IN_REVIEW or later

8.3 Sprint Planning Algorithm

```
// wbs-service/internal/sprint/planner.go

type SprintPlanRequest struct {
    WBSID         string
    SprintID      string
    TeamCapacity  int    // story points or hours
    UseAI         bool
}

func (s *SprintPlanner) Plan(ctx context.Context, req SprintPlanRequest)
(*SprintPlan, error) {
    // 1. Fetch backlog stories sorted by priority
    backlog, _ := s.repo.GetBacklog(ctx, req.WBSID, BacklogSort{
        Weights: []SortWeight{
            {Field: "priority_rank", Weight: 0.6},    // critical=4 high=3
medium=2 low=1
            {Field: "stakeholder_score", Weight: 0.2},
            {Field: "age_days_normalized", Weight: 0.2},
        },
    })

    // 2. Compute adjusted capacity from velocity history
    velocities := s.repo.GetLastNVelocities(ctx, req.WBSID, 6)
    adjustedCap := int(median(velocities) * float64(req.TeamCapacity))
    if len(velocities) == 0 { adjustedCap = int(float64(req.TeamCapacity) * 0.7) }
// first sprint

    // 3. Remove blocked stories and unresolved dependencies
    eligible := filterEligible(backlog)
```

```
    // 4. Greedy bin-pack with diversity constraint
    var packed []Story
    running := 0
    typeCounts := map[string]int{}
    for _, story := range eligible {
        pts := story.StoryPoints
        if pts == nil { continue }
        if running+*pts > adjustedCap { continue }
        typeKey := story.StoryType
        typePts := typeCounts[typeKey] + *pts
        if float64(typePts) > float64(adjustedCap)*0.60 { continue } // diversity
cap
        packed = append(packed, story)
        running += *pts
        typeCounts[typeKey] = typePts
    }

    confidence := 1.0 -
math.Abs(float64(running-adjustedCap))/float64(adjustedCap)
    return &SprintPlan{Stories: packed, TotalPoints: running,
                       AdjustedCapacity: adjustedCap, Confidence: confidence}, nil
}
```

8.4 Cycle Time & Velocity Analytics

```
-- Cycle time: average hours per status for all completed stories in a WBS
SELECT
    from_status,
    ROUND(AVG(duration_in_state_minutes) / 60.0, 2) AS avg_hours,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY duration_in_state_minutes) / 60.0
AS p50_hours,
    PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY duration_in_state_minutes) / 60.0
AS p95_hours,
    COUNT(*) AS sample_count
FROM story_status_history
WHERE story_id IN (
    SELECT id FROM story WHERE wbs_id = $1 AND status = 'done' AND deleted_at IS
NULL
)
GROUP BY from_status
ORDER BY
    CASE from_status WHEN 'backlog' THEN 1 WHEN 'ready' THEN 2
        WHEN 'in_progress' THEN 3 WHEN 'in_review' THEN 4 END;
```

8.5 WBS Service API

Endpoint	Method	Auth	Description
POST /api/v1/wbs	JWT	Idem	Create WBS for a portfolio project

Endpoint	Method	Auth	Description
GET /api/v1/wbs/:id	JWT	—	Get WBS metadata and board config
GET /api/v1/wbs/:id/board	JWT	—	Full board state (all columns + stories)
POST /api/v1/wbs/:id/stories	JWT	Idem	Create story
GET /api/v1/wbs/:id/stories	JWT	—	List stories (paginated, filterable)
GET /api/v1/wbs/:id/stories/:sid	JWT	—	Get single story with history
PUT /api/v1/wbs/:id/stories/:sid	JWT	Idem	Update story fields
PUT /api/v1/wbs/:id/stories/:sid/status	JWT	Idem	Transition story status
PUT /api/v1/wbs/:id/stories/:sid/move	JWT	—	Reorder story (new parent + sort_order)
DELETE /api/v1/wbs/:id/stories/:sid	JWT	—	Soft delete story
POST /api/v1/wbs/:id/sprints	JWT	Idem	Create sprint
GET /api/v1/wbs/:id/sprints/active	JWT	—	Get active sprint + stories
PUT /api/v1/wbs/:id/sprints/:spid/plan	JWT	—	Submit sprint plan (AI or manual)
PUT /api/v1/wbs/:id/sprints/:spid/start	JWT	—	Start sprint (planning → active)
PUT /api/v1/wbs/:id/sprints/:spid/close	JWT	—	Close sprint (active → closed)
POST /api/v1/wbs/:id/stories/:sid/time	JWT	Idem	Log time entry
GET /api/v1/wbs/:id/analytics	JWT	—	Velocity, cycle time, burndown data

8.6 WBS Events

Event Type	Source	Key Payload Fields	Consumers
wbs.story.created	wbs-svc	story_id, wbs_id, story_type, title, priority, created_by	Analytics, AI Agent (classify + suggest)
wbs.story.status_changed	wbs-svc	story_id, from_status, to_status, actor_id, actor_type, sprint_id?	Work (OKR), Exchange (milestone), Analytics, Market (if linked)
wbs.story.blocked	wbs-svc	story_id, blocker_note, blocked_since, wbs_id	AI Agent (Market search), Notification (escalation), Analytics
wbs.story.completed	wbs-svc	story_id, story_type, story_points, executor_id, executor_type	Sprint (velocity), Work (OKR check-in), Exchange (milestone release)
wbs.sprint.started	wbs-svc	sprint_id, wbs_id, story_ids[], planned_points, start_date	Analytics, Notification (team alert), AI (capacity check)

Event Type	Source	Key Payload Fields	Consumers
wbs.sprint.closed	wbs-svc	sprint_id, wbs_id, velocity, completed_pct, incomplete_story_ids[]	Analytics, Work (OKR period check), AI (generate retro draft)
wbs.story.ai_suggestion	ai-svc	story_id, suggestions: [{type, content, confidence}]	WBS (update story.ai_suggestions), Notification

§9 · MARKET

Market System

The Market System provides a bidirectional marketplace for services, gigs, products, talent, tools, resource rentals, and opportunities. It is tightly integrated with the Exchange (payments/escrow), Portfolio (auto-linking), and WBS (story auto-creation) systems.

9.1 Database Schema

Table: listing

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
provider_id	CHAR(36)	NOT NULL	—	User or AO UUID
provider_type	VARCHAR(10)	NOT NULL	—	user|ao
listing_type	VARCHAR(30)	NOT NULL	—	service|gig|product_digital|product_physical|talent|tool|resource_rental|opportunity
title	VARCHAR(300)	NOT NULL	—	Elasticsearch indexed (ngram, semantic)
description	TEXT	NOT NULL	—	ES indexed; Markdown
category	VARCHAR(60)	NOT NULL	—	Top-level category slug (see taxonomy)
subcategory	VARCHAR(60)	NULL	NULL	—
tags	JSONB	NOT	'[]'	ES indexed for tag search

Column	Type	Null	Default	Description
		NULL		
pricing	JSONB	NOT NULL	—	{type:hourly|fixed|retainer|free, amount, currency, billing_period?, packages:[]}
location_type	VARCHAR(20)	NOT NULL	'remote'	remote|onsite|hybrid
location_geo	GEOGRAPHY(POINT,4326)	NULL	NULL	PostGIS point for radius search
availability	JSONB	NOT NULL	—	{status:available|busy|unavailable, lead_time_days, capacity_per_week}
media	JSONB	NOT NULL	'[]'	[{url,type:image|video|pdf,caption}]
credentials	JSONB	NOT NULL	'[]'	[{type,issuer,verified_at}] — linked verified credentials
requirements	TEXT	NULL	NULL	What buyer must provide to start
deliverables	TEXT	NULL	NULL	What seller will deliver
revision_count	SMALLINT	NOT NULL	1	Number of revisions included
delivery_days	SMALLINT	NULL	NULL	Estimated delivery in days
status	VARCHAR(20)	NOT NULL	'active'	draft|active|paused|closed
view_count	INTEGER	NOT NULL	0	Incremented atomically via Redis counter, synced nightly
save_count	INTEGER	NOT	0	—

Column	Type	Null	Default	Description
		NULL		
order_count	INTEGER	NOT NULL	0	Completed orders (not all-time)
avg_rating	DECIMAL(3,2)	NOT NULL	0.00	Materialized from review aggregation
review_count	INTEGER	NOT NULL	0	—
es_doc_id	VARCHAR(64)	NULL	NULL	Elasticsearch doc ID; NULL = not yet indexed
es_indexed_at	TIMESTAMPTZ	NULL	NULL	Last successful ES index time
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: market_order

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
listing_id	CHAR(36)	NOT NULL	—	FK → listing.id

Column	Type	Null	Default	Description
buyer_id	CHAR(36)	NOT NULL	—	FK → account.id
seller_id	CHAR(36)	NOT NULL	—	FK → account.id or AO
status	VARCHAR(20)	NOT NULL	'pending'	State machine — see §9.2
brief	TEXT	NULL	NULL	Buyer project brief
proposal	TEXT	NULL	NULL	Seller accepted proposal
amount	DECIMAL(18,2)	NOT NULL	—	In USD
currency	CHAR(3)	NOT NULL	'USD'	ISO 4217
escrow_id	CHAR(36)	NULL	NULL	FK → escrow.id (cross-service reference)
portfolio_item_id	CHAR(36)	NULL	NULL	Auto-created in buyer portfolio
wbs_story_id	CHAR(36)	NULL	NULL	Auto-created in buyer WBS
milestones	JSONB	NOT NULL	'[]'	[{id,title,amount,due_date,status,escrow_milestone_idx}]
delivery_notes	TEXT	NULL	NULL	—
delivery_at	TIMESTAMPTZ	NULL	NULL	—
review_deadline	TIMESTAMPTZ	NULL	NULL	Auto-release if buyer silent
completed_at	TIMESTAMPTZ	NULL	NULL	—
dispute_id	CHAR(36)	NULL	NULL	FK → dispute.id if disputed

Column	Type	Null	Default	Description
ai_brief_match_score	DECIMAL(3,2)	NULL	NULL	AI-computed: how well delivery matches brief (0–1)
idempotency_key	VARCHAR(64)	NULL	NULL	UNIQUE — prevents double orders
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: review

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
order_id	CHAR(36)	NOT NULL	—	FK → market_order.id; UNIQUE per reviewer+order
reviewer_id	CHAR(36)	NOT NULL	—	FK → account.id
reviewee_id	CHAR(36)	NOT NULL	—	User or AO UUID
rating	DECIMAL(2,1)	NOT NULL	—	1.0–5.0, 0.5 increments
quality_rating	DECIMAL(2,1)	NOT NULL	—	Sub-dimension ratings

Column	Type	Null	Default	Description
communication_rating	DECIMAL(2,1)	NOT NULL	—	—
timeliness_rating	DECIMAL(2,1)	NOT NULL	—	—
value_rating	DECIMAL(2,1)	NOT NULL	—	—
body	TEXT	NULL	NULL	Max 2000 chars
response	TEXT	NULL	NULL	Provider public response; max 1000 chars
is_verified	BOOLEAN	NOT NULL	true	False if review pattern flagged by fraud detector
is_featured	BOOLEAN	NOT NULL	false	Provider-featured on their listing page
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: saved_listing

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
user_id	CHAR(36)	NOT NULL	—	FK → account.id

Column	Type	Null	Default	Description
listing_id	CHAR(36)	NOT NULL	—	FK → listing.id
note	VARCHAR(200)	NULL	NULL	Private user note
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: listing_event (for recommendation engine)

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
listing_id	CHAR(36)	NOT NULL	—	FK → listing.id
user_id	CHAR(36)	NOT NULL	—	FK → account.id
event_type	VARCHAR(20)	NOT NULL	—	view|save|inquiry|order|dismiss
engagement_weight	DECIMAL(3,2)	NOT NULL	—	view=0.1 save=0.5 inquiry=0.8 order=1.0 dismiss=-0.3
context	JSONB	NOT NULL	'{}'	Search query, referral source, etc.
created_at	TIMESTAMPTZ	NOT NULL	NOW()	Partitioned monthly for efficient analytics

Indexes: Market Schema

Index Name	Type	Columns	Purpose
idx_listing_provider	BTREE	listing(provider_id, status) WHERE deleted_at IS NULL	Provider listings list
idx_listing_category	BTREE	listing(category, subcategory, status)	Category browse
idx_listing_geo	GIST	listing(location_geo)	Radius search (PostGIS)
idx_listing_rating	BTREE	listing(avg_rating DESC, review_count DESC)	Top-rated listings
idx_listing_es_sync	PARTIAL	listing(id) WHERE es_doc_id IS NULL AND deleted_at IS NULL	ES sync queue
idx_order_buyer	BTREE	market_order(buyer_id, status)	Buyer orders list
idx_order_seller	BTREE	market_order(seller_id, status)	Seller orders list
idx_order_idem	UNIQUE	market_order(idempotency_key) WHERE idempotency_key IS NOT NULL	Prevent double order
idx_order_deadline	BTREE	market_order(review_deadline) WHERE status='delivery_submitted'	Auto-release sweep
idx_review_reviewee	BTREE	review(reviewee_id, is_verified)	Rating aggregation
idx_saved_user_listing	UNIQUE	saved_listing(user_id, listing_id)	No duplicate saves
idx_listing_event_user	BTREE	listing_event(user_id, created_at DESC)	User engagement history

9.2 Order State Machine

State	Entry Actions	Valid Transitions	Guard Conditions
PENDING	Create escrow record (status=created); notify seller; link to buyer portfolio item and WBS story	→ ACCEPTED (seller accepts within 48h)→ DECLINED (seller declines)→ EXPIRED (48h without response)	Idempotency key must be unique; buyer wallet KYC checked for amounts ≥ $500; escrow created atomically
ACCEPTED	Fund escrow from buyer wallet (status=funded); notify buyer; set delivery deadline based on listing.delivery_days	→ IN_PROGRESS (seller begins work)	Escrow funding must succeed; if wallet insufficient → order back to PENDING with ErrInsufficientBalance on buyer

State	Entry Actions	Valid Transitions	Guard Conditions
IN_PROGRESS	Notify both parties; WBS story status → IN_PROGRESS	→ DELIVERED (seller submits delivery)	—
DELIVERED	Set delivery_at; start review_deadline timer; AI computes ai_brief_match_score; notify buyer	→ COMPLETED (buyer approves or auto-release at review_deadline)→ DISPUTED (buyer disputes within window)→ IN_PROGRESS (seller revises, max 3 revisions)	delivery_notes NOT NULL; actor must be seller; ai_brief_match_score computed within 60s of delivery
DISPUTED	Freeze auto-release; create dispute record; assign AI mediator; escalate to human after 48h	→ RESOLVED (mediator decision)	Dispute opened within review_deadline; reason + evidence required
COMPLETED	Release escrow to seller; deduct platform fee; write ledger entries; emit order.completed; prompt reviews; WBS story → DONE; portfolio item → complete	(Terminal)	Buyer approved OR auto-release at review_deadline; platform fee = amount × fee_pct (default 2.5%)
CANCELLED	Refund escrow to buyer if funded; WBS story → CANCELLED; notify both parties	(Terminal)	Seller can cancel before ACCEPTED; buyer can cancel up to IN_PROGRESS with seller agreement; admin can cancel anytime
DECLINED	Notify buyer; no funds moved; WBS story → CANCELLED	(Terminal)	Seller must decline within 48h; after 48h → EXPIRED

9.3 Recommendation Engine

```
// market-service/internal/recommendation/engine.go

// Three-signal ranking: Content Match, Collaborative Filter, Quality Score

type RecommendationEngine struct {
    esClient   *elasticsearch.Client
    db         *sqlx.DB
    redis      *redis.Client
    embeddings EmbeddingService
}

func (e *RecommendationEngine) Recommend(ctx context.Context, req RecommendReq)
([]ScoredListing, error) {
    // 1. Build user skill vector from portfolio context
    skillVector := buildSkillVector(req.PortfolioContext)
```

```
    // 2. Elasticsearch recall: top 200 candidates
    candidates, _ := e.esClient.KNNSearch(ctx, "listings", skillVector, 200)

    // 3. Score each candidate
    results := make([]ScoredListing, 0, len(candidates))
    for _, c := range candidates {
        content  := cosineSimilarity(c.Embedding, skillVector)            // [0,1]
        collab   := e.collaborativeScore(ctx, req.UserID, c.ID)          // [0,1]
        quality  := computeQualityScore(c)                               // [0,1]
        boost    := e.blockedStoryBoost(req.BlockedStories, c)           // 1.0
or 1.5
        diversity := e.diversityPenalty(results, c)                      //
0.0-0.1

        final := (0.45*content + 0.25*collab + 0.25*quality - diversity) * boost
        results = append(results, ScoredListing{Listing: c, Score: final})
    }

    sort.Slice(results, func(i, j int) bool { return results[i].Score >
results[j].Score })
    limit := req.Limit
    if limit > 20 { limit = 20 }
    return results[:limit], nil
}

func computeQualityScore(l Listing) float64 {
    recency := math.Exp(-float64(time.Since(l.UpdatedAt).Hours()) / (30 * 24))
    return 0.40*(l.AvgRating/5.0) + 0.30*l.ResponseRate + 0.20*l.CompletionRate +
0.10*recency
}
```

9.4 Reputation Scoring Algorithm

```
// Runs nightly for all active users (daily at 02:00 UTC)
func ComputeReputationScore(ctx context.Context, userID string, lookbackDays int)
(float64, error) {
    // Component 1: Review quality (40%)
    reviews := getReviews(ctx, userID, lookbackDays)
    weightedSum, totalWeight := 0.0, 0.0
    for _, r := range reviews {
        age := time.Since(r.CreatedAt).Hours() / 24
        weight := 1.0
        if age < 30 { weight = 2.0 } // recent reviews weighted 2×
        weightedSum += float64(r.Rating) * weight
        totalWeight += weight
    }
    reviewScore := 0.0
    if totalWeight > 0 { reviewScore = (weightedSum / totalWeight) / 5.0 }

    // Component 2: Delivery rate (25%)
    completed := countOrders(ctx, userID, "completed", lookbackDays)
    accepted  := countOrders(ctx, userID, "accepted", lookbackDays)
    deliveryScore := 0.0
```

```
    if accepted > 0 { deliveryScore = float64(completed) / float64(accepted) }

    // Component 3: Response rate (15%)
    responseScore := computeResponseRate(ctx, userID, lookbackDays)

    // Component 4: Community activity (10%)
    activity := communityActivity(ctx, userID, lookbackDays)
    communityScore := math.Min(1.0, float64(activity)/50.0)

    // Component 5: Verification (10%)
    verification := verificationScore(ctx, userID) // 0, 0.2, 0.6, or 1.0

    raw := 0.40*reviewScore + 0.25*deliveryScore + 0.15*responseScore +
0.10*communityScore + 0.10*verification

    // Penalty for fraud flags
    flags := getFraudFlagsCount(ctx, userID, 90)
    penaltyFactor := 1.0 - math.Min(0.6, float64(flags)*0.20)

    score := math.Round(math.Max(0, math.Min(100, raw*100*penaltyFactor))*10) / 10
    return score, nil
}
```

9.5 Elasticsearch Index Mapping

```
// listings index mapping (ES 8.12)
{
  "mappings": {
    "properties": {
      "id":          { "type": "keyword" },
      "title":       { "type": "text", "analyzer": "standard",
                        "fields": { "ngram":
{"type":"text","analyzer":"ngram_analyzer"} } },
      "description": { "type": "text", "analyzer": "standard" },
      "category":    { "type": "keyword" },
      "subcategory": { "type": "keyword" },
      "tags":        { "type": "keyword" },
      "listing_type":{ "type": "keyword" },
      "status":      { "type": "keyword" },
      "avg_rating":  { "type": "float" },
      "review_count":{ "type": "integer" },
      "pricing_min": { "type": "float" },
      "location_geo":{ "type": "geo_point" },
      "availability_status": { "type": "keyword" },
      "provider_id": { "type": "keyword" },
      "embedding":   { "type": "dense_vector", "dims": 1536,
                        "index": true, "similarity": "cosine" }
    }
  },
  "settings": {
    "analysis": {
      "analyzer": {
        "ngram_analyzer": {
```

```
          "tokenizer": "ngram_tokenizer", "filter": ["lowercase"]
        }
      },
      "tokenizer": {
        "ngram_tokenizer": { "type": "ngram", "min_gram": 3, "max_gram": 4 }
      }
    },
    "number_of_shards": 3,
    "number_of_replicas": 1
  }
}
```

9.6 Market API

Endpoint	Method	Auth	Description
GET /api/v1/market/listings	Public	—	Browse/search listings (ES-backed)
GET /api/v1/market/listings/:id	Public	—	Get listing detail
POST /api/v1/market/listings	JWT(Pro)	Idem	Create listing
PUT /api/v1/market/listings/:id	JWT(Pro)	Idem	Update listing
DELETE /api/v1/market/listings/:id	JWT(Pro)	—	Close/delete listing
POST /api/v1/market/listings/:id/save	JWT	—	Save listing to collection
GET /api/v1/market/recommendations	JWT	—	AI-personalized listing recommendations
POST /api/v1/market/orders	JWT	Idem	Place order (creates escrow atomically)
GET /api/v1/market/orders	JWT	—	List my orders (buyer + seller)
GET /api/v1/market/orders/:id	JWT	—	Get order detail
PUT /api/v1/market/orders/:id/accept	JWT	Idem	Seller accepts order
PUT /api/v1/market/orders/:id/deliver	JWT	Idem	Seller submits delivery
PUT /api/v1/market/orders/:id/approve	JWT	Idem	Buyer approves delivery
POST /api/v1/market/orders/:id/dispute	JWT	Idem	Open dispute
POST /api/v1/market/orders/:id/reviews	JWT	Idem	Submit review (post-completion)

§10 · EXCHANGE

Exchange & Financial System

The Exchange System is the financial backbone of the Kogi platform. It handles wallets, ledgers, escrow, distributions, credits, and all value transfer between parties. Financial correctness is the highest engineering priority: correctness over performance, always.

10.1 Financial Invariants

🚫 **Non-Negotiable Financial Invariants — Any Violation Is a P0 Incident**

- INV-F1: SELECT FOR UPDATE is required on ALL wallet rows before any balance modification. No exceptions.
- INV-F2: Ledger entries are immutable. DB trigger raises EXCEPTION on any UPDATE or DELETE to ledger_entry.
- INV-F3: balance_after must equal previous balance_after ± amount at time of insert. Checked by DB constraint.
- INV-F4: All Exchange write operations require an Idempotency-Key header. Duplicate returns 200, not 200×2.
- INV-F5: fiat_available + fiat_reserved must always equal the sum of all ledger entries for that wallet.
- INV-F6: Escrowed funds live in fiat_reserved, not fiat_available, until release or refund.
- INV-F7: Platform fee is calculated at escrow creation time and locked in. Market fee changes do not affect active escrows.
- INV-F8: All financial events are published via Outbox in the same DB transaction as the balance change.

10.2 Database Schema

Table: wallet

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID; UNIQUE with owner_type
owner_type	VARCHAR(10)	NOT NULL	—	user|ao

Column	Type	Null	Default	Description
fiat_available	DECIMAL(18,2)	NOT NULL	0.00	Spendable balance in USD
fiat_reserved	DECIMAL(18,2)	NOT NULL	0.00	In active outbound escrows; unspendable
fiat_pending	DECIMAL(18,2)	NOT NULL	0.00	Inbound pending (provider settlement delay)
credit_balance	INTEGER	NOT NULL	0	Platform service credits (integer units)
currency	CHAR(3)	NOT NULL	'USD'	ISO 4217
stripe_customer_id	VARCHAR(100)	NULL	NULL	Stripe Customer for payment methods
stripe_account_id	VARCHAR(100)	NULL	NULL	Stripe Connect Express account for payouts
kyc_status	VARCHAR(20)	NOT NULL	'pending'	pending|verified|rejected|required
kyc_level	SMALLINT	NOT NULL	0	0=none 1=email 2=identity 3=enhanced
kyc_verified_at	TIMESTAMPTZ	NULL	NULL	—
kyc_provider_ref	VARCHAR(100)	NULL	NULL	External KYC provider reference
daily_send_limit	DECIMAL(18,2)	NOT NULL	1000.00	Increases with KYC level
monthly_send_limit	DECIMAL(18,2)	NOT NULL	10000.00	—

Column	Type	Null	Default	Description
ytd_sent	DECIMAL(18,2)	NOT NULL	0.00	Year-to-date sent; reset on Jan 1
ytd_received	DECIMAL(18,2)	NOT NULL	0.00	For 1099 threshold detection
version	INTEGER	NOT NULL	0	Optimistic lock — CRITICAL
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: ledger_entry ← IMMUTABLE — NO UPDATES, NO DELETES EVER

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
wallet_id	CHAR(36)	NOT NULL	—	FK → wallet.id
entry_type	VARCHAR(30)	NOT NULL	—	credit|debit|hold|hold_release|refund|fee|credit_award|distribution|adjustment
amount	DECIMAL(18,2)	NOT NULL	—	Always positive; entry_type indicates direction
currency	CHAR(3)	NOT NULL	—	—

Column	Type	Null	Default	Description
balance_before	DECIMAL(18,2)	NOT NULL	—	Snapshot at time of entry
balance_after	DECIMAL(18,2)	NOT NULL	—	balance_before ± amount; DB CHECK constraint
reference_id	CHAR(36)	NOT NULL	—	FK to source: escrow.id, distribution.id, etc.
reference_type	VARCHAR(30)	NOT NULL	—	escrow|distribution|deposit|withdrawal|credit_award|fee|refund
description	VARCHAR(500)	NOT NULL	—	Human-readable; visible to user in statement
metadata	JSONB	NOT NULL	'{}'	Additional context for reconciliation
idempotency_key	VARCHAR(64)	NULL	NULL	UNIQUE — prevents duplicate entries on replay
created_at	TIMESTAMPTZ	NOT NULL	NOW()	
created_by	CHAR(36)	NOT NULL	—	Service account UUID for audit trail

Table: escrow

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
buyer_id	CHAR(36)	NOT	—	FK → account.id

Column	Type	Null	Default	Description
		NULL		
seller_id	CHAR(36)	NOT NULL	—	FK → account.id or AO
status	VARCHAR(20)	NOT NULL	'created'	State machine
amount	DECIMAL(18,2)	NOT NULL	—	Total escrow amount in USD
currency	CHAR(3)	NOT NULL	'USD'	—
platform_fee	DECIMAL(18,2)	NOT NULL	0.00	Fee locked at creation time
fee_pct	DECIMAL(5,4)	NOT NULL	0.0250	Fee percentage at escrow creation (immutable)
funded_at	TIMESTAMPTZ	NULL	NULL	—
released_at	TIMESTAMPTZ	NULL	NULL	—
refunded_at	TIMESTAMPTZ	NULL	NULL	—
review_window_days	SMALLINT	NOT NULL	7	Days buyer has to approve/dispute
review_deadline	TIMESTAMPTZ	NULL	NULL	Set on DELIVERY_SUBMITTED
auto_release_at	TIMESTAMPTZ	NULL	NULL	Fallback: funded_at + max_hold_days
conditions	JSONB	NOT NULL	'[]'	Custom release conditions

Column	Type	Null	Default	Description
milestones	JSONB	NOT NULL	'[]'	[{idx,amount,description,released_at?}]
dispute_id	CHAR(36)	NULL	NULL	FK → dispute.id if disputed
reference_id	CHAR(36)	NOT NULL	—	order_id or contract_id or campaign_id
reference_type	VARCHAR(30)	NOT NULL	—	market_order|contract|campaign
idempotency_key	VARCHAR(64)	NOT NULL	—	UNIQUE — prevents double escrow creation
version	INTEGER	NOT NULL	0	Optimistic lock
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: distribution

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id
capital_pool_id	CHAR(36)	NOT	—	FK → capital_pool.id

Column	Type	Null	Default	Description
		NULL		
proposal_id	CHAR(36)	NOT NULL	—	FK → proposal.id (governance approval required)
total_amount	DECIMAL(18,2)	NOT NULL	—	Total to be distributed
period_label	VARCHAR(50)	NULL	NULL	e.g. "Q3 2024 Earnings"
distribution_type	VARCHAR(20)	NOT NULL	—	profit_share|dividend|patronage_refund|labor_share
formula	JSONB	NOT NULL	—	{type: equal|pro_rata_stake|pro_rata_labor|hybrid, params:{}}
recipients	JSONB	NOT NULL	'[]'	Computed: [{user_id, amount, wallet_id}]
status	VARCHAR(20)	NOT NULL	'pending'	pending|approved|processing|completed|failed
completed_at	TIMESTAMPTZ	NULL	NULL	—
idempotency_key	VARCHAR(64)	NOT NULL	—	UNIQUE
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: dispute

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
order_id	CHAR(36)	NOT NULL	—	FK → market_order.id
escrow_id	CHAR(36)	NOT NULL	—	FK → escrow.id
opener_id	CHAR(36)	NOT NULL	—	FK → account.id
reason	VARCHAR(200)	NOT NULL	—	reason category + free text
evidence_urls	JSONB	NOT NULL	'[]'	Array of evidence file CDN URLs
status	VARCHAR(20)	NOT NULL	'open'	open|under_review|resolved
resolution	VARCHAR(30)	NULL	NULL	release_to_seller|refund_to_buyer|partial_split
split_pct	DECIMAL(5,2)	NULL	NULL	Buyer pct if partial_split
mediator_id	CHAR(36)	NULL	NULL	AI agent session ID or human moderator ID
mediator_notes	TEXT	NULL	NULL	—
resolved_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: Exchange Schema

Index Name	Type	Columns	Purpose
idx_wallet_owner	UNIQUE	wallet(owner_id, owner_type)	Owner wallet lookup
idx_ledger_wallet_ts	BTREE	ledger_entry(wallet_id, created_at DESC)	Transaction history paginate
idx_ledger_reference	BTREE	ledger_entry(reference_id, reference_type)	Source reference lookup
idx_ledger_idem	UNIQUE	ledger_entry(idempotency_key) WHERE idempotency_key IS NOT NULL	Prevent duplicate entries
idx_escrow_buyer	BTREE	escrow(buyer_id, status)	Buyer active escrows
idx_escrow_seller	BTREE	escrow(seller_id, status)	Seller pending receipts
idx_escrow_auto_release	BTREE	escrow(auto_release_at) WHERE status='funded'	Auto-release sweep
idx_escrow_deadline	BTREE	escrow(review_deadline) WHERE status='delivery_submitted'	Review deadline sweep
idx_escrow_idem	UNIQUE	escrow(idempotency_key)	Prevent double escrow
idx_distribution_ao	BTREE	distribution(ao_id, status)	AO distribution history
idx_dispute_order	UNIQUE	dispute(order_id)	One dispute per order

10.3 Wallet Mutation Pattern — Go Implementation

```go
// exchange-service/internal/wallet/service.go
// ALL wallet mutations MUST follow this exact pattern.

func (s *WalletService) FundEscrow(ctx context.Context, req FundEscrowReq) error {
    idem := req.IdempotencyKey
    // Check idempotency cache first
    if cached, _ := s.cache.GetIdempotentResult(ctx, idem); cached != nil {
        return nil // already processed
    }

    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        // STEP 1: Lock the wallet row — REQUIRED
        wallet, err := s.repo.GetWalletForUpdate(tx, req.BuyerID)
        if err != nil { return fmt.Errorf("lock wallet: %w", err) }

        // STEP 2: Check sufficient balance
```

```
        if wallet.FiatAvailable.LessThan(req.Amount) {
            return ErrInsufficientBalance
        }

        // STEP 3: Check KYC level for large transactions
        if req.Amount.GreaterThanOrEqual(decimal.New(1000, 0)) && wallet.KYCLevel
< 2 {
            return ErrKYCRequired
        }

        // STEP 4: Update balance fields
        wallet.FiatAvailable = wallet.FiatAvailable.Sub(req.Amount)
        wallet.FiatReserved  = wallet.FiatReserved.Add(req.Amount)
        wallet.Version++
        if err := s.repo.UpdateWallet(tx, wallet); err != nil { return err }

        // STEP 5: Write immutable ledger entry
        if err := s.repo.InsertLedgerEntry(tx, LedgerEntry{
            WalletID:      req.BuyerID,
            EntryType:     "hold",
            Amount:        req.Amount,
            BalanceBefore: wallet.FiatAvailable.Add(req.Amount),
            BalanceAfter:  wallet.FiatAvailable,
            ReferenceID:   req.EscrowID,
            ReferenceType: "escrow",
            Description:   fmt.Sprintf("Hold for escrow %s", req.EscrowID),
            IdempotencyKey: &idem,
        }); err != nil { return err }

        // STEP 6: Update escrow status
        if err := s.repo.UpdateEscrowStatus(tx, req.EscrowID, "funded",
req.EscrowVersion); err != nil {
            return err // version mismatch → 409
        }

        // STEP 7: Write outbox event
        return s.outbox.Insert(tx, "exchange.escrow.funded", EscrowFundedPayload{
            EscrowID: req.EscrowID, Amount: req.Amount, BuyerID: req.BuyerID,
        })
    })
}
```

10.4 Escrow Release — Auto and Manual

```
// exchange-service/internal/escrow/release.go

// Auto-release job (runs every 5 minutes)
func (s *EscrowService) AutoReleaseSweep(ctx context.Context) error {
    due, err := s.repo.GetDueForAutoRelease(ctx)
    if err != nil { return err }

    for _, escrow := range due {
```

```
        idem := fmt.Sprintf("auto-release-%s", escrow.ID)
        if err := s.ReleaseEscrow(ctx, ReleaseEscrowReq{
            EscrowID:       escrow.ID,
            IdempotencyKey: idem,
            AutoRelease:    true,
        }); err != nil {
            log.Error("auto-release failed", "escrow_id", escrow.ID, "err", err)
            // Don't abort sweep; continue with next
        }
    }
    return nil
}

func (s *EscrowService) ReleaseEscrow(ctx context.Context, req ReleaseEscrowReq)
error {
    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        escrow, _ := s.repo.GetEscrowForUpdate(tx, req.EscrowID)
        if escrow.Status != "delivery_submitted" && !req.AutoRelease {
            return ErrInvalidEscrowState
        }

        netAmount := escrow.Amount.Sub(escrow.PlatformFee)

        // Debit buyer reserved
        buyerWallet, _ := s.repo.GetWalletForUpdate(tx, escrow.BuyerID)
        buyerWallet.FiatReserved = buyerWallet.FiatReserved.Sub(escrow.Amount)
        buyerWallet.Version++
        s.repo.UpdateWallet(tx, buyerWallet)
        s.repo.InsertLedgerEntry(tx, ledgerHoldRelease(buyerWallet, escrow))

        // Credit seller available
        sellerWallet, _ := s.repo.GetWalletForUpdate(tx, escrow.SellerID)
        sellerWallet.FiatAvailable = sellerWallet.FiatAvailable.Add(netAmount)
        sellerWallet.Version++
        s.repo.UpdateWallet(tx, sellerWallet)
        s.repo.InsertLedgerEntry(tx, ledgerCredit(sellerWallet, netAmount,
escrow))

        s.repo.UpdateEscrowStatus(tx, req.EscrowID, "complete", escrow.Version)
        return s.outbox.Insert(tx, "exchange.escrow.released",
EscrowReleasedPayload{
            EscrowID: req.EscrowID, SellerID: escrow.SellerID,
            NetAmount: netAmount, Fee: escrow.PlatformFee,
        })
    })
}
```

10.5 Distribution Algorithm

```
// Pro-rata labor share formula (common for worker cooperatives)
func ComputeDistributionRecipients(d Distribution, members []AOMember) []Recipient
{
    switch d.Formula.Type {
```

```
    case "equal":
        perMember := d.TotalAmount.Div(decimal.New(int64(len(members)), 0))
        return mapEqual(members, perMember)

    case "pro_rata_stake":
        totalStake := sumStakes(members)
        return mapProRata(members, d.TotalAmount, func(m AOMember) decimal.Decimal
{
            return m.EquityPct.Div(totalStake)
        })

    case "pro_rata_labor":
        totalHours := sumHours(members, d.PeriodStart, d.PeriodEnd)
        return mapProRata(members, d.TotalAmount, func(m AOMember) decimal.Decimal
{
            hours := getMemberHours(m.UserID, d.PeriodStart, d.PeriodEnd)
            if totalHours == 0 { return decimal.Zero }
            return decimal.New(int64(hours), 0).Div(decimal.New(int64(totalHours),
0))
        })

    case "hybrid":
        // e.g. 50% stake-weighted + 50% labor-weighted
        stakeRecips :=
ComputeDistributionRecipients(Distribution{Formula:{Type:"pro_rata_stake"},...},
members)
        laborRecips :=
ComputeDistributionRecipients(Distribution{Formula:{Type:"pro_rata_labor"},...},
members)
        stakeWeight := d.Formula.Params["stake_weight"].(float64)
        return blendRecipients(stakeRecips, laborRecips, stakeWeight)
    }
    return nil
}
```

10.6 Exchange API

Endpoint	Method	Auth	Description
GET /api/v1/exchange/wallet	JWT	—	Get my wallet balances
GET /api/v1/exchange/wallet/transactions	JWT	—	Paginated ledger history
POST /api/v1/exchange/wallet/deposit	JWT	Idem	Initiate Stripe deposit
POST /api/v1/exchange/wallet/withdraw	JWT	Idem	Request payout to Stripe Connect
POST /api/v1/exchange/invoices	JWT	Idem	Create and send invoice
GET /api/v1/exchange/invoices	JWT	—	List my invoices
POST /api/v1/exchange/escrow	JWT	Idem	Create escrow (called by market-svc)

Endpoint	Method	Auth	Description
PUT /api/v1/exchange/escrow/:id/fund	JWT	Idem	Fund escrow from wallet
PUT /api/v1/exchange/escrow/:id/release	JWT	Idem	Manually release escrow to seller
PUT /api/v1/exchange/escrow/:id/refund	JWT	Idem	Refund escrow to buyer
GET /api/v1/exchange/escrow/:id	JWT	—	Get escrow status
POST /api/v1/exchange/distributions	JWT(AOAdmin)	Idem	Create distribution (requires governance approval)
GET /api/v1/exchange/distributions/:id	JWT	—	Get distribution detail
GET /api/v1/exchange/tax/1099	JWT	—	Download annual 1099 data (CSV)
GET /api/v1/exchange/kyc/status	JWT	—	Get KYC status
POST /api/v1/exchange/kyc/initiate	JWT	—	Initiate KYC verification (redirect to provider)

§11 · STUDIO

Idea Studio System

The Idea Studio is the innovation layer of Kogi. It captures raw ideas instantly and guides them through a structured 5-stage pipeline from raw concept to launch-ready plan, with AI synthesis, business model canvas, IP timestamping, and stakeholder validation at each stage.

11.1 Pipeline Stages

Stage	Name	Entry Criteria	Key Actions	Exit Criteria	AI Role
1	Capture	None — zero friction	Title/voice capture; auto-tag; assign portfolio	Idea exists and is saved	Auto-tag; classify domain; suggest related existing ideas
2	Concept	Stage 1 complete	Problem statement; target market; early hypothesis; competitor notes	Problem + market defined; 3 competitors noted	Generate problem synthesis; surface similar validated concepts; suggest interview questions
3	Validate	Stage 2 complete	User interview notes; signal log; demand testing; lean canvas	3+ user signals; lean canvas draft	Summarize interview patterns; score demand signals; auto-fill lean canvas from notes
4	Design	Stage 3 complete	Technical design notes; feature scope; tech stack decision; build/buy/partner matrix	MVP feature list defined; approach decided	Generate system design draft; suggest tech stack; estimate build effort
5	Launch	Stage 4 complete	Go-to-market plan; success metrics; resource allocation; linked portfolio project	GTM plan + metrics defined; linked project created	Draft launch announcement; suggest success metrics; auto-create portfolio project + WBS from scope

11.2 Database Schema

Table: idea

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	FK → account.id
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
title	VARCHAR(500)	NOT NULL	—	Short capture; Elasticsearch indexed
body	TEXT	NULL	NULL	Extended notes; Markdown
stage	SMALLINT	NOT NULL	1	1–5 (pipeline stages above)
status	VARCHAR(20)	NOT NULL	'active'	active|archived|promoted|abandoned
domain	VARCHAR(60)	NULL	NULL	Auto-tagged domain (saas|marketplace|coop|content|physical|service|research)
tags	JSONB	NOT NULL	'[]'	User-defined and AI-suggested tags
canvas	JSONB	NULL	NULL	Lean canvas / BMC: {problem,solution,uvp,channels,segments,revenue,costs,metrics,advantage}
competitors	JSONB	NOT NULL	'[]'	[{name,url,notes,threat_level:1-5}]
demand_signals	JSONB	NOT NULL	'[]'	[{type,description,strength:1-5,date}]
user_interviews	JSONB	NOT	'[]'	[{participant,date,key_findings[]}]

Column	Type	Null	Default	Description
		NULL		
tech_notes	TEXT	NULL	NULL	Architecture and technical decisions
gtm_plan	JSONB	NULL	NULL	{channels[], pricing, launch_sequence[], success_metrics[]}
linked_portfolio_id	CHAR(36)	NULL	NULL	FK → portfolio_item.id — set on stage 5 promote
linked_wbs_id	CHAR(36)	NULL	NULL	FK → wbs.id — set on promote
ip_timestamp_id	CHAR(36)	NULL	NULL	FK → ip_timestamp.id when timestamped
ai_synthesis	TEXT	NULL	NULL	AI-generated synthesis paragraph; refreshed on stage advance
ai_viability_score	DECIMAL(3,2)	NULL	NULL	AI viability signal 0.0–1.0
ai_risk_factors	JSONB	NULL	NULL	AI-identified risks []{factor,severity,mitigation?}
visibility	VARCHAR(20)	NOT NULL	'private'	private|team|public
collaborators	JSONB	NOT NULL	'[]'	[{user_id,role:viewer|editor}]
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: ip_timestamp

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
idea_id	CHAR(36)	NOT NULL	—	FK → idea.id; UNIQUE
owner_id	CHAR(36)	NOT NULL	—	—
title_hash	CHAR(64)	NOT NULL	—	SHA-256 of idea title at timestamp time
body_hash	CHAR(64)	NOT NULL	—	SHA-256 of idea body
full_hash	CHAR(64)	NOT NULL	—	SHA-256 of canonical JSON representation
timestamp_utc	TIMESTAMPTZ	NOT NULL	NOW()	Server-side only; not client-supplied
certificate_url	VARCHAR(512)	NOT NULL	—	CDN URL of generated PDF certificate
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Indexes: Studio Schema

Index Name	Type	Columns	Purpose
idx_idea_owner_status	BTREE	idea(owner_id, status, stage) WHERE deleted_at IS NULL	My ideas dashboard
idx_idea_ws_stage	BTREE	idea(workspace_id, stage) WHERE deleted_at IS NULL	Workspace pipeline view
idx_idea_tags	GIN	idea(tags)	Tag filtering

Index Name	Type	Columns	Purpose
idx_idea_canvas	GIN	idea(canvas)	Canvas JSON search
idx_ip_idea	UNIQUE	ip_timestamp(idea_id)	One timestamp per idea
idx_ip_full_hash	BTREE	ip_timestamp(full_hash)	Duplicate detection

11.3 Stage Advancement Logic

```
// studio-service/internal/service/pipeline.go

func (s *StudioService) AdvanceStage(ctx context.Context, ideaID string, userID
string) error {
    idea, err := s.repo.GetIdea(ctx, ideaID)
    if err != nil || idea.OwnerID != userID { return ErrNotFound }
    if idea.Stage >= 5 { return ErrAlreadyAtFinalStage }

    // Validate exit criteria for current stage
    if err := validateExitCriteria(idea); err != nil { return err }

    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        idea.Stage++
        idea.Version++
        if err := s.repo.UpdateIdea(tx, idea); err != nil { return err }

        // Stage 5 promotion: auto-create portfolio item and WBS
        if idea.Stage == 5 {
            return s.promoteToPortfolio(tx, idea)
        }

        return s.outbox.Insert(tx, "studio.idea.stage_advanced",
IdeaStageAdvancedPayload{
            IdeaID: ideaID, NewStage: idea.Stage, Title: idea.Title,
        })
    })
}

func validateExitCriteria(idea *Idea) error {
    switch idea.Stage {
    case 1: // → stage 2
        if len(idea.Title) < 10 { return ErrInsufficientDetail{Field:"title"} }
        return nil
    case 2: // → stage 3
        if idea.Canvas == nil || idea.Canvas.Problem == "" { return
ErrInsufficientDetail{Field:"canvas.problem"} }
        if len(idea.Competitors) < 1 { return
ErrInsufficientDetail{Field:"competitors"} }
        return nil
    case 3: // → stage 4
        if len(idea.DemandSignals) < 3 { return
ErrInsufficientDetail{Field:"demand_signals (min 3)"} }
        return nil
```

```
    case 4: // → stage 5
        if idea.TechNotes == "" { return ErrInsufficientDetail{Field:"tech_notes"}
}
        return nil
    }
    return nil
}
```

11.4 AI Synthesis — Prompt Strategy

```
// Called by ai-service when studio.idea.stage_advanced event received

func buildIdeaSynthesisPrompt(idea Idea) string {
    return fmt.Sprintf(`You are a startup strategist and product advisor.

Analyze this idea at pipeline stage %d and provide:
1. A 2-3 sentence synthesis of the core opportunity
2. A viability score (0.0-1.0) with one-line rationale
3. The top 3 risk factors as a JSON array [{factor, severity: low|medium|high,
mitigation}]
4. 3 concrete next-step recommendations for the user

Idea title: %s
Body: %s
Canvas: %s
Demand signals: %s

Respond ONLY in JSON: {synthesis, viability_score, risks, next_steps}`,
        idea.Stage, idea.Title, truncate(idea.Body, 2000),
        jsonStr(idea.Canvas), jsonStr(idea.DemandSignals))
}
```

11.5 Studio API

Endpoint	Method	Auth	Description
POST /api/v1/studio/ideas	JWT	Idem	Capture idea (title only required)
GET /api/v1/studio/ideas	JWT	—	List ideas with filters (stage, status, tag)
GET /api/v1/studio/ideas/:id	JWT	—	Get idea detail with AI synthesis
PUT /api/v1/studio/ideas/:id	JWT	Idem	Update idea fields
PUT /api/v1/studio/ideas/:id/advance	JWT	Idem	Advance to next stage (validates exit criteria)
PUT /api/v1/studio/ideas/:id/canvas	JWT	Idem	Update lean canvas / BMC
POST /api/v1/studio/ideas/:id/timestamp	JWT(Pro)	Idem	Create IP timestamp certificate

Endpoint	Method	Auth	Description
GET /api/v1/studio/ideas/:id/timestamp	JWT	—	Get IP timestamp certificate URL
POST /api/v1/studio/ideas/:id/promote	JWT	Idem	Promote to portfolio (creates project + WBS)
DELETE /api/v1/studio/ideas/:id	JWT	—	Archive idea
GET /api/v1/studio/ideas/:id/synthesis	JWT	—	Get or request fresh AI synthesis

§12 · COMMUNITY

Community & Social System

The Community System provides the social and collaborative layer of Kogi: Spaces (communities), threaded Rooms (channels), direct messages, social connections, real-time presence, WebSocket chat, and an activity feed. It uses NATS for multi-pod WebSocket fan-out.

12.1 Database Schema

Table: space

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
owner_type	VARCHAR(10)	NOT NULL	—	user|ao
name	VARCHAR(200)	NOT NULL	—	—
slug	VARCHAR(100)	NOT NULL	—	UNIQUE; URL-safe
description	TEXT	NULL	NULL	—
category	VARCHAR(60)	NOT NULL	—	cooperative|freelance|industry|project|interest|open_source
tags	JSONB	NOT NULL	'[]'	—
avatar_url	VARCHAR(512)	NULL	NULL	—

Column	Type	Null	Default	Description
banner_url	VARCHAR(512)	NULL	NULL	—
privacy	VARCHAR(20)	NOT NULL	'public'	public|invite_only|private
member_count	INTEGER	NOT NULL	0	Denormalized counter
is_verified	BOOLEAN	NOT NULL	false	Platform-verified community
linked_ao_id	CHAR(36)	NULL	NULL	FK → autonomous_org.id — governance-linked space
rules	TEXT	NULL	NULL	Community rules; Markdown
default_room_id	CHAR(36)	NULL	NULL	FK → room.id — #general equivalent
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	—

Table: space_member

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK

Column	Type	Null	Default	Description
space_id	CHAR(36)	NOT NULL	—	FK → space.id
user_id	CHAR(36)	NOT NULL	—	FK → account.id
role	VARCHAR(20)	NOT NULL	'member'	owner|moderator|member
joined_at	TIMESTAMPTZ	NOT NULL	NOW()	—
muted_until	TIMESTAMPTZ	NULL	NULL	NULL = not muted
last_read_at	TIMESTAMPTZ	NULL	NULL	For unread badge calculation

Table: room

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
space_id	CHAR(36)	NOT NULL	—	FK → space.id
name	VARCHAR(100)	NOT NULL	—	—
topic	VARCHAR(300)	NULL	NULL	—
room_type	VARCHAR(20)	NOT NULL	'text'	text|announcement|thread|voice(future)
is_default	BOOLEAN	NOT	false	Whether new members auto-join

Column	Type	Null	Default	Description
		NULL		
is_archived	BOOLEAN	NOT NULL	false	—
linked_wbs_id	CHAR(36)	NULL	NULL	FK → wbs.id — project room
linked_idea_id	CHAR(36)	NULL	NULL	FK → idea.id — idea discussion room
sort_order	FLOAT	NOT NULL	0	—
last_message_at	TIMESTAMPTZ	NULL	NULL	Updated on new message; used for ordering
message_count	INTEGER	NOT NULL	0	Denormalized
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: message

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
room_id	CHAR(36)	NOT NULL	—	FK → room.id; partitioned by room_id for performance
author_id	CHAR(36)	NOT NULL	—	FK → account.id

Column	Type	Null	Default	Description
parent_id	CHAR(36)	NULL	NULL	FK → message.id — threaded reply
content	TEXT	NOT NULL	—	Max 10,000 chars; Markdown. Required unless media_urls non-empty.
content_type	VARCHAR(20)	NOT NULL	'text'	text|ai_summary|system|announcement
media_urls	JSONB	NOT NULL	'[]'	[{url,type:image|file|video,filename,size_bytes}]
mentions	JSONB	NOT NULL	'[]'	[user_id] — extracted from @mentions
reactions	JSONB	NOT NULL	'{}'	{emoji: [user_id, ...]} — denormalized for speed
link_previews	JSONB	NULL	NULL	Auto-generated from URLs in content
is_pinned	BOOLEAN	NOT NULL	false	—
is_edited	BOOLEAN	NOT NULL	false	—
edited_at	TIMESTAMPTZ	NULL	NULL	—
deleted_at	TIMESTAMPTZ	NULL	NULL	Soft delete; content replaced with "[deleted]"
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: social_connection

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
requester_id	CHAR(36)	NOT NULL	—	FK → account.id
addressee_id	CHAR(36)	NOT NULL	—	FK → account.id
status	VARCHAR(20)	NOT NULL	'pending'	pending|connected|blocked
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: dm_thread

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
participant_ids	JSONB	NOT NULL	—	Sorted array of user IDs; max 2 for DM, more for group
participant_hash	CHAR(64)	NOT NULL	—	SHA-256 of sorted participants; UNIQUE index for lookup
last_message_at	TIMESTAMPTZ	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT	NOW()	—

Column	Type	Null	Default	Description
		NULL		

Indexes: Community Schema

Index Name	Type	Columns	Purpose
idx_space_slug	UNIQUE	space(slug)	URL routing
idx_space_category	BTREE	space(category, privacy, member_count DESC)	Discovery browse
idx_space_ao	BTREE	space(linked_ao_id) WHERE linked_ao_id IS NOT NULL	AO space lookup
idx_member_space_user	UNIQUE	space_member(space_id, user_id)	Membership check
idx_member_user	BTREE	space_member(user_id)	My spaces list
idx_room_space	BTREE	room(space_id, sort_order)	Space rooms list
idx_room_wbs	BTREE	room(linked_wbs_id) WHERE linked_wbs_id IS NOT NULL	Project room lookup
idx_msg_room_ts	BTREE	message(room_id, created_at DESC)	Room message feed
idx_msg_parent	BTREE	message(parent_id) WHERE parent_id IS NOT NULL	Thread replies
idx_msg_author	BTREE	message(author_id, created_at DESC)	User message history
idx_msg_pinned	PARTIAL	message(room_id) WHERE is_pinned=true	Pinned messages
idx_connection_pair	UNIQUE	social_connection(requester_id, addressee_id)	Duplicate prevention
idx_connection_status	BTREE	social_connection(addressee_id, status)	Pending requests
idx_dm_hash	UNIQUE	dm_thread(participant_hash)	DM thread lookup

12.2 WebSocket Hub — Multi-Pod Architecture

```
// community-service/internal/ws/hub.go
```

```go
// Each community-service pod runs one Hub instance.
// NATS bridges messages between pods.

type Hub struct {
    mu      sync.RWMutex
    clients map[string]*Client       // userID → *Client
    rooms   map[string][]*Client     // roomID → []*Client (local pod only)
    nats    *nats.Conn
    db      *sqlx.DB
}

func (h *Hub) HandleConnect(w http.ResponseWriter, r *http.Request) {
    userID := r.Context().Value(ctxKeyUserID).(string)
    conn, _ := websocket.Upgrade(w, r, nil, 4096, 4096)
    client := &Client{
        UserID: userID, Conn: conn, Send: make(chan []byte, 256),
        Rooms: map[string]bool{},
    }
    h.mu.Lock()
    h.clients[userID] = client
    h.mu.Unlock()
    h.updatePresence(userID, true)

    go client.writePump()
    client.readPump(h) // blocks until disconnect

    h.mu.Lock()
    delete(h.clients, userID)
    h.mu.Unlock()
    h.updatePresence(userID, false)
}

// Fan-out: message arrives from any pod via NATS
func (h *Hub) SubscribeNATS() {
    h.nats.Subscribe("room.>", func(msg *nats.Msg) {
        // subject: room.{roomID}.message
        roomID := extractRoomID(msg.Subject)
        h.mu.RLock()
        clients := h.rooms[roomID]
        h.mu.RUnlock()
        for _, c := range clients {
            select {
            case c.Send <- msg.Data:
            default: // slow client — skip; client will catch up on reconnect
                close(c.Send)
            }
        }
    })
}

// When a user sends a message from this pod:
func (h *Hub) BroadcastMessage(roomID string, msg ChatMessage) {
    data, _ := json.Marshal(WsEnvelope{Type:"message", Payload: msg})
    // Publish to NATS → all pods fan out to their local WebSocket clients
    h.nats.Publish(fmt.Sprintf("room.%s.message", roomID), data)
}
```

12.3 Presence System

```
// Presence stored in Redis with TTL
// Key: presence:{userID}   Value: {status,last_seen,space_id?,room_id?}   TTL:
90s

func (h *Hub) updatePresence(userID string, online bool) {
    if online {
        h.redis.Set(ctx, "presence:"+userID, PresenceData{
            Status: "online", LastSeen: time.Now(),
        }, 90*time.Second)
    } else {
        h.redis.Set(ctx, "presence:"+userID, PresenceData{
            Status: "offline", LastSeen: time.Now(),
        }, 24*time.Hour) // keep offline status for 24h
    }
    // Broadcast presence change to all rooms this user is in
    h.nats.Publish("presence."+userID, PresenceChangeEvent{
        UserID: userID, Online: online,
    })
}

// Heartbeat: client sends ping every 30s; hub updates TTL
// After 2 missed heartbeats (90s), Redis key expires → user shows as offline

// Batch presence for a list of user IDs (e.g. room member list)
func (s *CommunityService) GetPresenceBatch(ctx, userIDs []string)
map[string]PresenceData {
    keys := make([]string, len(userIDs))
    for i, id := range userIDs { keys[i] = "presence:"+id }
    results, _ := s.redis.MGet(ctx, keys...).Result()
    out := map[string]PresenceData{}
    for i, r := range results {
        if r != nil { json.Unmarshal([]byte(r.(string)), &out[userIDs[i]]) }
    }
    return out
}
```

12.4 Message Moderation

Rule	Detection Method	Action	Reversibility
Spam (duplicate messages)	> 5 identical messages in 60s from same user in same room	Mute user in space for 30 min; alert space moderator	Moderator can lift mute
Prohibited keywords	Regex match against platform-level blocklist (maintained in config)	Message delivery blocked; author warned; moderator notified	Author can edit and resubmit
Reported content	User report → AI classification	If confidence > 0.85 harmful: auto-hide	Moderator reinstates or permanently removes

Rule	Detection Method	Action	Reversibility
	(harmful|spam|ok confidence)	pending review; else: queue for moderator	
Doxxing detection	Pattern: phone/SSN/email patterns in message body	Message blocked; author warned; security team alerted if repeated	Security team review
NSFW image	AWS Rekognition label confidence > 0.90 for adult content	Image blocked; message still delivered without image; moderator alerted	—

12.5 Community API

Endpoint	Method	Auth	Description
GET /api/v1/community/spaces	JWT	—	Discover spaces (search, browse, my spaces)
POST /api/v1/community/spaces	JWT	Idem	Create space
GET /api/v1/community/spaces/:id	JWT	—	Get space detail + rooms
POST /api/v1/community/spaces/:id/join	JWT	—	Join space
DELETE /api/v1/community/spaces/:id/leave	JWT	—	Leave space
GET /api/v1/community/spaces/:id/rooms	JWT	—	List rooms
POST /api/v1/community/spaces/:id/rooms	JWT	Idem	Create room
GET /api/v1/community/rooms/:id/messages	JWT	—	Get messages (cursor-paginated)
POST /api/v1/community/rooms/:id/messages	JWT	Idem	Send message (REST fallback)
PUT /api/v1/community/messages/:id	JWT	Idem	Edit message
DELETE /api/v1/community/messages/:id	JWT	—	Delete message (soft)
POST /api/v1/community/messages/:id/react	JWT	—	Add/remove emoji reaction
POST /api/v1/community/messages/:id/report	JWT	—	Report message
GET /api/v1/community/presence	JWT	—	Get presence for a list of user IDs
POST /api/v1/community/connections	JWT	—	Send connection request

Endpoint	Method	Auth	Description
PUT /api/v1/community/connections/:id	JWT	—	Accept/decline connection
GET /api/v1/community/dms	JWT	—	List DM threads
GET /api/v1/community/dms/:threadId	JWT	—	Get DM thread messages
WS /api/v1/community/ws	JWT(WS)	—	WebSocket connection (chat + presence + board)

§13 · WORK

Work, Strategy & OKR System

The Work System manages Objectives and Key Results (OKRs), strategic goals, capacity planning, and automation rules. It integrates with WBS (story completion → OKR auto-check-in), Portfolio (item health → OKR context), and the AI Agent (OKR narrative + scoring).

13.1 Database Schema

Table: okr (Objective + Key Results as JSONB)

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
owner_type	VARCHAR(10)	NOT NULL	—	user|ao
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
parent_okr_id	CHAR(36)	NULL	NULL	FK → okr.id — company→team→individual cascade
level	VARCHAR(20)	NOT NULL	—	org|team|individual
period_type	VARCHAR(10)	NOT NULL	—	annual|quarterly|custom
period_label	VARCHAR(20)	NOT NULL	—	e.g. "Q1 2025" or "2025 Annual"
period_start	DATE	NOT	—	—

Column	Type	Null	Default	Description
		NULL		
period_end	DATE	NOT NULL	—	—
title	VARCHAR(500)	NOT NULL	—	Objective statement
description	TEXT	NULL	NULL	—
key_results	JSONB	NOT NULL	—	Array of KRs — see structure below
overall_score	DECIMAL(3,2)	NULL	NULL	0.00–1.00; weighted avg of KR scores
confidence	DECIMAL(3,2)	NULL	NULL	Subjective confidence 0.00–1.00
status	VARCHAR(20)	NOT NULL	'on_track'	on_track|at_risk|off_track|completed|cancelled
linked_portfolio_item_ids	JSONB	NOT NULL	'[]'	Portfolio items contributing to this OKR
linked_wbs_ids	JSONB	NOT NULL	'[]'	WBS IDs whose story completion auto-updates KRs
ai_narrative	TEXT	NULL	NULL	AI-generated status narrative; weekly refresh
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT	NOW()	—

Column	Type	Null	Default	Description
		NULL		

ℹ️ **Key Result JSON Structure**

- Each element in key_results array: {id (uuid), title, kr_type (numeric|boolean|milestone|percentage), target, current, unit, score (0-1), auto_track (boolean), auto_track_source (story_type[]|metric_name), notes, last_updated_at}
- KR scoring: numeric → clamp(0,1, current/target). boolean → 0 or 1. milestone → fraction complete. percentage → current/100.
- When auto_track=true and auto_track_source is story_type[], wbs.story.completed events for matching types auto-increment current and recompute score.

Table: okr_check_in

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
okr_id	CHAR(36)	NOT NULL	—	FK → okr.id; append-only
check_in_type	VARCHAR(20)	NOT NULL	—	manual|auto_story|auto_metric|ai_weekly
kr_updates	JSONB	NOT NULL	—	[{kr_id, prev_value, new_value, delta}]
overall_score	DECIMAL(3,2)	NOT NULL	—	—
confidence	DECIMAL(3,2)	NULL	NULL	—
note	TEXT	NULL	NULL	Manual or AI-generated note
actor_id	CHAR(36)	NOT NULL	—	User ID or AI agent session ID

Column	Type	Null	Default	Description
actor_type	VARCHAR(20)	NOT NULL	—	user|ai_agent|system
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: automation_rule

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
workspace_id	CHAR(36)	NOT NULL	—	FK → workspace.id
owner_id	CHAR(36)	NOT NULL	—	FK → account.id
name	VARCHAR(200)	NOT NULL	—	—
is_enabled	BOOLEAN	NOT NULL	true	—
trigger_type	VARCHAR(30)	NOT NULL	—	event|schedule|webhook
trigger_config	JSONB	NOT NULL	—	event: {event_type, filters:{}}. schedule: {cron_expr}. webhook: {url,secret}
conditions	JSONB	NOT NULL	'[]'	[{field, operator, value}] — AND logic

Column	Type	Null	Default	Description
actions	JSONB	NOT NULL	—	[{action_type, params:{}}] — see action types below
run_count	INTEGER	NOT NULL	0	—
last_run_at	TIMESTAMPTZ	NULL	NULL	—
last_run_status	VARCHAR(20)	NULL	NULL	success|failed|skipped
last_error	TEXT	NULL	NULL	—
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

ℹ️ **Automation Action Types**

- create_story — {wbs_id, story_type, title_template, priority, assignee_id?}
- update_story_status — {story_id_ref or filter, new_status}
- send_notification — {user_ids_ref, channel: in_app|email, message_template}
- search_market — {query_template, category?, max_results}
- create_portfolio_item — {portfolio_id, item_type, name_template}
- update_okr_kr — {okr_id, kr_id, delta_value or set_value}
- run_ai_synthesis — {target_type: idea|story|okr, target_id_ref}
- webhook_call — {url, method, headers, body_template}
- add_tag — {target_type, target_id_ref, tags[]}
- create_dm — {recipient_id, message_template}

Indexes: Work Schema

Index Name	Type	Columns	Purpose
idx_okr_owner_period	BTREE	okr(owner_id, period_type, period_label)	Dashboard OKR list
idx_okr_parent	BTREE	okr(parent_okr_id) WHERE parent_okr_id IS NOT NULL	Cascade scoring
idx_okr_status	BTREE	okr(status, period_end)	At-risk OKRs sweep
idx_checkin_okr_ts	BTREE	okr_check_in(okr_id, created_at DESC)	Check-in history
idx_automation_ws	BTREE	automation_rule(workspace_id, is_enabled)	Active rules list
idx_automation_trigger	BTREE	automation_rule(trigger_type, is_enabled)	Event-triggered rule lookup

13.2 OKR Auto-Scoring — Cascade Algorithm

```
// work-service/internal/okr/scorer.go

// Called when: story completed, manual check-in, weekly AI job
func (s *OKRScorer) ScoreOKRCascade(ctx context.Context, okrID string) error {
    okr, _ := s.repo.GetOKR(ctx, okrID)

    // 1. Score each key result
    for i, kr := range okr.KeyResults {
        if kr.AutoTrack && kr.AutoTrackSource != nil {
            // Recount from WBS stories
            current, _ := s.countAutoTrackProgress(ctx, okr, kr)
            okr.KeyResults[i].Current = current
        }
        okr.KeyResults[i].Score = computeKRScore(kr)
    }

    // 2. Weighted average (equal weights unless kr.weight set)
    totalWeight, weightedSum := 0.0, 0.0
    for _, kr := range okr.KeyResults {
        w := kr.Weight
        if w == 0 { w = 1.0 }
        weightedSum += float64(kr.Score) * w
        totalWeight += w
    }
    if totalWeight > 0 { okr.OverallScore = weightedSum / totalWeight }

    // 3. Update status based on score and days remaining
    okr.Status = deriveStatus(okr.OverallScore, okr.PeriodEnd)

    // 4. Write check-in record and update okr row
    s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        s.repo.UpdateOKR(tx, okr)
        s.repo.InsertCheckIn(tx, OKRCheckIn{
            OKRID: okrID, CheckInType: "auto_story",
```

```
            OverallScore: okr.OverallScore, ActorType: "system",
        })
        return s.outbox.Insert(tx, "work.okr.scored", OKRScoredPayload{
            OKRID: okrID, Score: okr.OverallScore, Status: okr.Status,
        })
    })

    // 5. Cascade to parent OKR if exists
    if okr.ParentOKRID != nil {
        return s.ScoreOKRCascade(ctx, *okr.ParentOKRID)
    }
    return nil
}

func deriveStatus(score float64, end time.Time) string {
    daysRemaining := time.Until(end).Hours() / 24
    if daysRemaining <= 0 {
        if score >= 0.70 { return "completed" }
        return "off_track"
    }
    timeElapsed := 1.0 - (daysRemaining / float64(90)) // assume quarterly
    expectedProgress := timeElapsed
    if score >= expectedProgress*0.8 { return "on_track" }
    if score >= expectedProgress*0.5 { return "at_risk" }
    return "off_track"
}
```

13.3 Automation Engine

```
// work-service/internal/automation/engine.go

type AutomationEngine struct {
    repo    AutomationRepo
    bus     *nats.Conn
    actions map[string]ActionHandler
}

// Triggered by NATS subscription on "platform.>""
func (e *AutomationEngine) HandleEvent(ctx context.Context, event PlatformEvent)
error {
    // Find all enabled rules whose trigger matches this event type
    rules, _ := e.repo.GetRulesForTrigger(ctx, event.EventType)
    for _, rule := range rules {
        if !evaluateConditions(rule.Conditions, event.Payload) { continue }

        ctx2 := withRuleContext(ctx, rule)
        for _, action := range rule.Actions {
            handler, ok := e.actions[action.ActionType]
            if !ok { continue }
            if err := handler.Execute(ctx2, action.Params, event); err != nil {
                e.repo.RecordRunFailure(ctx, rule.ID, err)
                break // stop action chain on failure
            }
```

```
        }
        e.repo.RecordRunSuccess(ctx, rule.ID)
    }
    return nil
}

// Condition evaluation: simple DSL
// Supported operators: eq, neq, gt, lt, gte, lte, contains, starts_with, in,
not_in
func evaluateConditions(conditions []Condition, payload map[string]interface{})
bool {
    for _, c := range conditions {
        val := extractPath(payload, c.Field)
        if !evalOperator(c.Operator, val, c.Value) { return false }
    }
    return true // all conditions passed (AND logic)
}
```

13.4 Work Service API

Endpoint	Method	Auth	Description
POST /api/v1/work/okrs	JWT	Idem	Create OKR
GET /api/v1/work/okrs	JWT	—	List OKRs (filtered by period, status, owner)
GET /api/v1/work/okrs/:id	JWT	—	Get OKR detail with check-ins
PUT /api/v1/work/okrs/:id	JWT	Idem	Update OKR fields
POST /api/v1/work/okrs/:id/checkin	JWT	Idem	Submit manual check-in with progress update
PUT /api/v1/work/okrs/:id/score	JWT	—	Trigger immediate rescore
GET /api/v1/work/okrs/:id/checkins	JWT	—	OKR check-in history
DELETE /api/v1/work/okrs/:id	JWT	—	Cancel OKR
POST /api/v1/work/automations	JWT(Pro)	Idem	Create automation rule
GET /api/v1/work/automations	JWT(Pro)	—	List automation rules
PUT /api/v1/work/automations/:id	JWT(Pro)	Idem	Update rule
PUT /api/v1/work/automations/:id/toggle	JWT(Pro)	—	Enable/disable rule
POST /api/v1/work/automations/:id/test	JWT(Pro)	—	Test rule with sample payload
GET /api/v1/work/automations/:id/runs	JWT(Pro)	—	Automation run history

Endpoint	Method	Auth	Description
DELETE /api/v1/work/automations/:id	JWT(Pro)	—	Delete rule
GET /api/v1/work/capacity	JWT	—	Team capacity view (hours vs story commitments)

§14 · FUND

Crowdfunding & Capital System

The Fund System manages crowdfunding campaigns (reward, donation, and equity-lite pledge) and cooperative capital pools. It integrates with the Exchange System for fund movement and the AO Governance System for spending proposals.

14.1 Database Schema

Table: campaign

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
owner_id	CHAR(36)	NOT NULL	—	User or AO UUID
owner_type	VARCHAR(10)	NOT NULL	—	user|ao
workspace_id	CHAR(36)	NOT NULL	—	—
title	VARCHAR(300)	NOT NULL	—	ES indexed
description	TEXT	NOT NULL	—	Markdown; max 50KB
campaign_type	VARCHAR(20)	NOT NULL	—	reward|donation|pledge
status	VARCHAR(20)	NOT NULL	'draft'	draft|live|funded|failed|cancelled|closed

Column	Type	Null	Default	Description
funding_goal	DECIMAL(18,2)	NOT NULL	—	Target amount in USD
min_funding	DECIMAL(18,2)	NULL	NULL	NULL = all-or-nothing at goal; set for flexible
funding_model	VARCHAR(20)	NOT NULL	'all_or_nothing'	all_or_nothing|flexible
currency	CHAR(3)	NOT NULL	'USD'	—
start_date	DATE	NULL	NULL	NULL = starts immediately on launch
end_date	DATE	NOT NULL	—	—
current_amount	DECIMAL(18,2)	NOT NULL	0.00	Denormalized; updated on each pledge
backer_count	INTEGER	NOT NULL	0	Denormalized
media	JSONB	NOT NULL	'[]'	[{url,type,caption}]
rewards	JSONB	NOT NULL	'[]'	Reward tiers — see structure below
faq	JSONB	NOT NULL	'[]'	[{question,answer}]
updates	JSONB	NOT NULL	'[]'	Campaign updates [{date,title,body}]
linked_portfolio_id	CHAR(36)	NULL	NULL	FK → portfolio_item.id

Column	Type	Null	Default	Description
escrow_id	CHAR(36)	NULL	NULL	FK → escrow.id (funds held until goal met)
capital_pool_id	CHAR(36)	NULL	NULL	FK → capital_pool.id — where funds go on success
risk_disclosure	TEXT	NULL	NULL	Required for pledge campaigns
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: pledge

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
campaign_id	CHAR(36)	NOT NULL	—	FK → campaign.id
backer_id	CHAR(36)	NOT NULL	—	FK → account.id
amount	DECIMAL(18,2)	NOT NULL	—	In campaign currency
reward_tier_idx	SMALLINT	NULL	NULL	Index into campaign.rewards array; NULL = donation
status	VARCHAR(20)	NOT NULL	'pledged'	pledged|captured|refunded|cancelled

Column	Type	Null	Default	Description
payment_intent_id	VARCHAR(100)	NULL	NULL	Stripe Payment Intent; captured on goal met
anonymous	BOOLEAN	NOT NULL	false	—
note	VARCHAR(500)	NULL	NULL	Backer message to creator
reward_fulfilled	BOOLEAN	NOT NULL	false	Set by campaign creator on reward delivery
idempotency_key	VARCHAR(64)	NOT NULL	—	UNIQUE
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: capital_pool

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id; UNIQUE
wallet_id	CHAR(36)	NOT NULL	—	FK → wallet.id — AO-owned wallet
name	VARCHAR(200)	NOT NULL	—	—

Column	Type	Null	Default	Description
description	TEXT	NULL	NULL	—
balance_usd	DECIMAL(18,2)	NOT NULL	0.00	Materialized from ledger; reconciled nightly
reserved_usd	DECIMAL(18,2)	NOT NULL	0.00	In active spending proposals
contribution_rules	JSONB	NOT NULL	'{}'	{min_contribution, max_contribution, contribution_period, require_governance_for_withdrawal_over}
total_contributed	DECIMAL(18,2)	NOT NULL	0.00	Lifetime contributed; for member ROI calculations
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: pool_contribution

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
capital_pool_id	CHAR(36)	NOT NULL	—	FK → capital_pool.id
contributor_id	CHAR(36)	NOT	—	FK → account.id

Column	Type	Null	Default	Description
		NULL		
amount	DECIMAL(18,2)	NOT NULL	—	—
contribution_type	VARCHAR(20)	NOT NULL	—	cash|service_credit|asset_transfer|pledge_fulfillment
note	VARCHAR(500)	NULL	NULL	—
escrow_id	CHAR(36)	NULL	NULL	FK → escrow.id — if contribution held in escrow pending governance
ledger_entry_id	CHAR(36)	NOT NULL	—	FK → ledger_entry.id — immutable record
idempotency_key	VARCHAR(64)	NOT NULL	—	UNIQUE
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

14.2 Campaign State Machine

State	Entry Actions	Valid Transitions	Guard Conditions
DRAFT	Owner editing; no public visibility; preview available	→ LIVE (manual launch)→ CANCELLED	All required fields complete; risk_disclosure set for pledge campaigns
LIVE	Publicly visible; accepting pledges; progress tracking	→ FUNDED (goal met before end_date)→ FAILED (end_date passed, goal not met, all_or_nothing model)→ CANCELLED (owner cancels)	end_date in future; owner verified; at least 1 reward tier (if reward type)
FUNDED	Goal met; capture all pledges; transfer to capital_pool; notify backers	(Terminal — but update thread continues)	current_amount >= funding_goal (or >= min_funding for flexible model)
FAILED	Goal not met (all-or-nothing); refund all	(Terminal)	end_date passed; current_amount <

State	Entry Actions	Valid Transitions	Guard Conditions
	pledges to backer wallets; notify		funding_goal; funding_model = all_or_nothing
CANCELLED	Owner cancelled; refund all captured pledges; notify backers; log reason	(Terminal)	cancellation_reason required; if pledges already captured, refund sequence triggered
CLOSED	Flexible-funded but ended; final capture; distribute above minimum; close	(Terminal)	end_date passed; current_amount >= min_funding; funding_model = flexible

14.3 All-or-Nothing Funding Logic

```go
// fund-service/internal/campaign/funding.go

func (s *FundService) FundingDeadlineSweep(ctx context.Context) error {
    campaigns, _ := s.repo.GetExpiredLiveCampaigns(ctx)
    for _, c := range campaigns {
        if c.CurrentAmount >= c.FundingGoal {
            s.fundCampaign(ctx, c)
        } else if c.FundingModel == "all_or_nothing" {
            s.failCampaign(ctx, c)
        } else { // flexible
            if c.CurrentAmount >= c.MinFunding.ValueOrZero() {
                s.closeCampaign(ctx, c)
            } else {
                s.failCampaign(ctx, c)
            }
        }
    }
    return nil
}

func (s *FundService) failCampaign(ctx context.Context, c Campaign) {
    pledges, _ := s.repo.GetActivePledges(ctx, c.ID)
    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
        // Refund each pledge
        for _, p := range pledges {
            s.walletSvc.RefundToWallet(tx, p.BackerID, p.Amount, p.ID)
        }
        s.repo.UpdateCampaignStatus(tx, c.ID, "failed", c.Version)
        return s.outbox.Insert(tx, "fund.campaign.failed", CampaignFailedPayload{
            CampaignID: c.ID, RefundedPledges: len(pledges),
        })
    })
}
```

14.4 Fund System API

Endpoint	Method	Auth	Description
POST /api/v1/fund/campaigns	JWT	Idem	Create campaign
GET /api/v1/fund/campaigns	Public	—	Browse campaigns (public only)
GET /api/v1/fund/campaigns/:id	Public	—	Get campaign detail
PUT /api/v1/fund/campaigns/:id	JWT	Idem	Update draft campaign
PUT /api/v1/fund/campaigns/:id/launch	JWT	Idem	Launch campaign (draft → live)
PUT /api/v1/fund/campaigns/:id/cancel	JWT	Idem	Cancel campaign
POST /api/v1/fund/campaigns/:id/pledge	JWT	Idem	Back a campaign
DELETE /api/v1/fund/campaigns/:id/pledge	JWT	—	Cancel your pledge (if campaign still live)
POST /api/v1/fund/campaigns/:id/update	JWT	Idem	Post campaign update
GET /api/v1/fund/capital-pools/:aoId	JWT(Member)	—	Get AO capital pool status
POST /api/v1/fund/capital-pools/:aoId/contribute	JWT(Member)	Idem	Contribute to capital pool

§15 · GOVERNANCE

Autonomous Organization Governance System

The AO System provides the complete governance infrastructure for autonomous organizations: cooperatives, investment clubs, open-source foundations, and any group requiring formal democratic decision-making, charter management, member management, and auditable records.

15.1 Database Schema

Table: autonomous_org

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
founder_id	CHAR(36)	NOT NULL	—	FK → account.id
name	VARCHAR(300)	NOT NULL	—	ES indexed
slug	VARCHAR(100)	NOT NULL	—	UNIQUE URL-safe
ao_type	VARCHAR(30)	NOT NULL	—	worker_coop|consumer_coop|investment_club|open_source|dao|custom
status	VARCHAR(20)	NOT NULL	'forming'	forming|active|suspended|dissolved
charter	JSONB	NOT NULL	—	Full charter — see structure below
member_count	INTEGER	NOT NULL	0	Denormalized

Column	Type	Null	Default	Description
total_equity	DECIMAL(18,2)	NOT NULL	0.00	Sum of all member equity_pct (should = 100.00)
wallet_id	CHAR(36)	NULL	NULL	FK → wallet.id — AO-owned wallet
space_id	CHAR(36)	NULL	NULL	FK → space.id — AO community space
portfolio_id	CHAR(36)	NULL	NULL	FK → portfolio.id — AO portfolio
is_public	BOOLEAN	NOT NULL	true	—
logo_url	VARCHAR(512)	NULL	NULL	—
website_url	VARCHAR(512)	NULL	NULL	—
version	INTEGER	NOT NULL	0	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—
dissolved_at	TIMESTAMPTZ	NULL	NULL	—

ℹ️ Charter JSON Structure

- {governance_model: "one_member_one_vote"|"stake_weighted"|"hybrid"|"role_based",
- quorum_pct: decimal (e.g. 0.51 = simple majority),
- supermajority_pct: decimal (e.g. 0.67 — for charter amendments and dissolution),
- voting_window_days: integer (default 7),
- proposal_types: [{type, title, required_approver_role?, auto_execute, auto_execute_threshold?}],
- member_roles: [{name, can_submit_proposals, can_vote, can_receive_distributions, default_equity_pct?}],
- distribution_rules: {default_formula, require_governance, approval_threshold_usd},
- charter_version: integer, ratified_at: timestamptz, ratified_by: [user_id]}

Table: ao_member

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id
user_id	CHAR(36)	NOT NULL	—	FK → account.id
role	VARCHAR(30)	NOT NULL	—	Matches charter.member_roles
equity_pct	DECIMAL(10,6)	NOT NULL	0	Ownership percentage; sum across ao must = 100.00
labor_hours	DECIMAL(18,2)	NOT NULL	0.00	Cumulative labor hours contributed (for labor-share distributions)
capital_contributed	DECIMAL(18,2)	NOT NULL	0.00	Cumulative capital contributed
status	VARCHAR(20)	NOT NULL	'active'	active|suspended|resigned|removed
joined_via	VARCHAR(20)	NOT NULL	—	founder|proposal|invitation
proposal_id	CHAR(36)	NULL	NULL	FK → proposal.id that admitted this member
joined_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Column	Type	Null	Default	Description
exited_at	TIMESTAMPTZ	NULL	NULL	—

Table: proposal ← APPEND-ONLY — NO DELETES EVER

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id
proposer_id	CHAR(36)	NOT NULL	—	FK → account.id
proposal_type	VARCHAR(40)	NOT NULL	—	member_admission|member_removal|capital_allocation|charter_amendment|distribution|dissolution|custom
title	VARCHAR(300)	NOT NULL	—	—
description	TEXT	NOT NULL	—	Markdown; full rationale
payload	JSONB	NOT NULL	—	Type-specific: {user_id} for admission; {amount, destination} for capital; etc.
status	VARCHAR(20)	NOT NULL	'open'	open|quorum_met|passed|failed|withdrawn|executed
voting_model	VARCHAR(30)	NOT NULL	—	From charter at time of submission — frozen at submit
quorum_required	DECIMAL(5,4)	NOT	—	Frozen from charter at submission

Column	Type	Null	Default	Description
		NULL		
pass_threshold	DECIMAL(5,4)	NOT NULL	—	Frozen from charter at submission
voting_opens_at	TIMESTAMPTZ	NOT NULL	—	—
voting_closes_at	TIMESTAMPTZ	NOT NULL	—	—
total_eligible	INTEGER	NOT NULL	—	Eligible voter count at open time — frozen
votes_yes	INTEGER	NOT NULL	0	Denormalized
votes_no	INTEGER	NOT NULL	0	—
votes_abstain	INTEGER	NOT NULL	0	—
quorum_reached	BOOLEAN	NOT NULL	false	—
executed_at	TIMESTAMPTZ	NULL	NULL	When downstream action was executed
execution_log	TEXT	NULL	NULL	Result of execution — success or error details
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: vote ← IMMUTABLE — NO UPDATES, NO DELETES

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
proposal_id	CHAR(36)	NOT NULL	—	FK → proposal.id
voter_id	CHAR(36)	NOT NULL	—	FK → account.id; UNIQUE with proposal_id
vote	VARCHAR(10)	NOT NULL	—	yes|no|abstain
weight	DECIMAL(10,6)	NOT NULL	1.0	From ao_member at time of vote — frozen
cast_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: governance_log ← APPEND-ONLY — NO UPDATES, NO DELETES — LEGAL AUDIT RECORD

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
ao_id	CHAR(36)	NOT NULL	—	FK → autonomous_org.id
event_type	VARCHAR(40)	NOT NULL	—	ao_formed|member_joined|member_exited|proposal_submitted|vote_cast|proposal_passed|proposal_failed|charter_amended|distribution_executed|dissolved

Column	Type	Null	Default	Description
actor_id	CHAR(36)	NOT NULL	—	User ID or system
reference_id	CHAR(36)	NULL	NULL	Proposal ID / distribution ID / etc.
payload	JSONB	NOT NULL	—	Full snapshot of relevant state at event time
created_at	TIMESTAMPTZ	NOT NULL	NOW()	— Server-side ONLY

⚠️ **Governance Audit Log Immutability Enforcement**

- DB trigger: BEFORE UPDATE OR DELETE ON governance_log → RAISE EXCEPTION 'governance_log is immutable — RFC-005 INV-GOV-1'
- DB trigger: BEFORE UPDATE OR DELETE ON vote → RAISE EXCEPTION 'vote is immutable — INV-GOV-2'
- DB trigger: BEFORE UPDATE OR DELETE ON proposal → RAISE EXCEPTION if status = passed|failed|executed
- Application layer: AO service exposes NO endpoint that modifies or deletes governance records.
- Quarterly automated report: export governance log for each active AO as signed JSON; store in S3.

Indexes: AO Governance Schema

Index Name	Type	Columns	Purpose
idx_ao_slug	UNIQUE	autonomous_org(slug)	URL routing
idx_ao_type_status	BTREE	autonomous_org(ao_type, status)	Discovery / analytics
idx_member_ao_user	UNIQUE	ao_member(ao_id, user_id)	Membership check
idx_member_user	BTREE	ao_member(user_id, status)	My AOs list
idx_proposal_ao_status	BTREE	proposal(ao_id, status, voting_closes_at)	Active proposals list
idx_proposal_close	BTREE	proposal(voting_closes_at) WHERE status='open'	Voting deadline sweep
idx_vote_proposal	UNIQUE	vote(proposal_id, voter_id)	One vote per member per proposal

Index Name	Type	Columns	Purpose
idx_govlog_ao_ts	BTREE	governance_log(ao_id, created_at DESC)	AO audit trail
idx_govlog_type	BTREE	governance_log(event_type, created_at DESC)	Event type analytics

15.2 Proposal Execution Engine

```
// ao-service/internal/governance/executor.go

type ProposalExecutor struct {
    db          *sqlx.DB
    outbox      OutboxService
    exchangeSvc ExchangeServiceClient  // gRPC
    portfolioSvc PortfolioServiceClient
    userSvc     UserServiceClient
}

func (e *ProposalExecutor) Execute(ctx context.Context, proposal Proposal) error {
    if proposal.Status != "passed" { return ErrProposalNotPassed }

    var execErr error
    switch proposal.ProposalType {

    case "capital_allocation":
        payload := proposal.Payload.(CapitalAllocationPayload)
        // Transfer from AO capital pool wallet to destination
        execErr = e.exchangeSvc.TransferFunds(ctx, TransferFundsReq{
            FromWalletID: payload.FromCapitalPoolWalletID,
            ToWalletID:   payload.ToWalletID,
            Amount:       payload.Amount,
            Description:  fmt.Sprintf("Approved by proposal %s", proposal.ID),
            IdempotencyKey: "proposal-exec-"+proposal.ID,
        })

    case "member_admission":
        payload := proposal.Payload.(MemberAdmissionPayload)
        execErr = e.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
            return e.memberRepo.AddMember(tx, proposal.AOID, payload.UserID,
payload.Role, payload.EquityPct)
        })

    case "member_removal":
        payload := proposal.Payload.(MemberRemovalPayload)
        execErr = e.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
            return e.memberRepo.SetMemberStatus(tx, payload.MemberID, "removed")
        })

    case "charter_amendment":
        payload := proposal.Payload.(CharterAmendmentPayload)
        // Write new charter version; governance_log records old + new charter
        execErr = e.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
```

```
            return e.aoRepo.AmendCharter(tx, proposal.AOID, payload.NewCharter)
        })

    case "dissolution":
        execErr = e.dissolveAO(ctx, proposal)
    }

    // Log execution result (success or error) — append-only governance log
    e.appendGovernanceLog(ctx, proposal.AOID, "proposal_executed", proposal.ID,
execErr)
    if execErr != nil {
        e.updateProposalExecutionLog(ctx, proposal.ID, "failed: "+execErr.Error())
        return execErr
    }
    e.updateProposalStatus(ctx, proposal.ID, "executed")
    return nil
}
```

15.3 Quorum and Vote Tally

```
// Called by voting_deadline_sweep job every 5 minutes
func (s *ProposalService) TallyAndClose(ctx context.Context, proposalID string)
error {
    proposal, _ := s.repo.GetProposal(ctx, proposalID)
    if proposal.Status != "open" { return nil }
    if time.Now().Before(proposal.VotingClosesAt) { return nil }

    totalVoteWeight := 0.0
    yesWeight := 0.0
    votes, _ := s.repo.GetVotes(ctx, proposalID)
    for _, v := range votes {
        totalVoteWeight += float64(v.Weight)
        if v.Vote == "yes" { yesWeight += float64(v.Weight) }
    }

    // Check quorum
    eligibleWeight := float64(proposal.TotalEligible) // for 1M1V, = member count
    quorumReached := totalVoteWeight/eligibleWeight >=
float64(proposal.QuorumRequired)

    var status string
    if !quorumReached {
        status = "failed" // no quorum = failed
    } else {
        passPct := yesWeight / totalVoteWeight
        if passPct >= float64(proposal.PassThreshold) {
            status = "passed"
        } else {
            status = "failed"
        }
    }

    return s.db.RunInTx(ctx, func(tx *sqlx.Tx) error {
```

```
        s.repo.UpdateProposalStatus(tx, proposalID, status)
        s.appendGovernanceLog(tx, proposal.AOID, "proposal_"+status, proposalID)
        eventType := "ao.proposal.passed"
        if status == "failed" { eventType = "ao.proposal.failed" }
        return s.outbox.Insert(tx, eventType, ProposalClosedPayload{
            ProposalID: proposalID, AOID: proposal.AOID, Status: status,
            QuorumReached: quorumReached, YesPct: yesWeight/totalVoteWeight,
        })
    })
}
```

15.4 AO Governance API

Endpoint	Method	Auth	Description
POST /api/v1/ao	JWT	Idem	Form new AO
GET /api/v1/ao/:id	Public	—	Get AO public info
PUT /api/v1/ao/:id	JWT(AOAdmin)	Idem	Update AO settings
GET /api/v1/ao/:id/members	JWT(Member)	—	List AO members + equity
POST /api/v1/ao/:id/invitations	JWT(AOAdmin)	Idem	Invite new member
POST /api/v1/ao/:id/proposals	JWT(Member)	Idem	Submit proposal
GET /api/v1/ao/:id/proposals	JWT(Member)	—	List proposals (open/history)
GET /api/v1/ao/:id/proposals/:pid	JWT(Member)	—	Get proposal + votes
POST /api/v1/ao/:id/proposals/:pid/vote	JWT(Member)	Idem	Cast vote
PUT /api/v1/ao/:id/proposals/:pid/withdraw	JWT(Proposer)	—	Withdraw open proposal
GET /api/v1/ao/:id/governance-log	JWT(Member)	—	Full governance audit log

Endpoint	Method	Auth	Description
GET /api/v1/ao/:id/governance-log/export	JWT(AOAdmin)	—	Export full history (JSON/CSV)
GET /api/v1/ao/:id/capital	JWT(Member)	—	Capital pool status + ledger
POST /api/v1/ao/:id/capital/contribute	JWT(Member)	Idem	Contribute to capital pool
DELETE /api/v1/ao/:id/members/me	JWT(Member)	—	Resign from AO

§16 · AI AGENT

AI Agent System

The AI Agent System is the contextual intelligence layer of Kogi. It provides a portfolio-aware conversational agent, autonomous action capabilities, proactive alerting, background synthesis tasks, and the tool-routing infrastructure that connects LLM outputs to Kogi module actions.

16.1 System Architecture

Component	Responsibility	Implementation
Session Manager	Manages per-user conversation sessions with sliding-window history	PostgreSQL session table + Redis hot cache for active sessions
Context Builder	Constructs system prompt with user portfolio context before each LLM call	Parallel gRPC calls to portfolio, wbs, work, exchange services; context assembly < 500ms
LLM Gateway	Sends formatted messages to Claude API; handles streaming; manages token budget	Go HTTP client with retry; SSE streaming to client via gin SSE writer
Tool Router	Parses tool_call blocks from LLM response; dispatches to module action handlers	Registry pattern: tool_name → ActionHandler; each action validated before execution
Action Log	Immutable record of every action taken by AI Agent on behalf of user	ai_action_log table; append-only
Synthesis Worker	Background Goroutine pool consuming ai.* NATS events for scheduled synthesis tasks	Work queue via NATS; 4-worker goroutine pool per pod
Alert Engine	Monitors portfolio/OKR/sprint state; generates proactive notifications	Triggered by domain events and nightly schedule sweep

16.2 Database Schema

Table: agent_session

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK

Column	Type	Null	Default	Description
user_id	CHAR(36)	NOT NULL	—	FK → account.id
workspace_id	CHAR(36)	NOT NULL	—	—
title	VARCHAR(300)	NULL	NULL	AI-generated summary title; set after first exchange
status	VARCHAR(20)	NOT NULL	'active'	active|ended
message_count	INTEGER	NOT NULL	0	—
token_count	INTEGER	NOT NULL	0	Running total of tokens used
context_snapshot	JSONB	NULL	NULL	Portfolio context used in this session
memory_refs	JSONB	NOT NULL	'[]'	Memory item IDs injected into context
model_used	VARCHAR(50)	NOT NULL	—	e.g. claude-sonnet-4-20250514
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
ended_at	TIMESTAMPTZ	NULL	NULL	—
last_message_at	TIMESTAMPTZ	NULL	NULL	—

Table: agent_message

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
session_id	CHAR(36)	NOT NULL	—	FK → agent_session.id
role	VARCHAR(20)	NOT NULL	—	user|assistant|tool_result
content	TEXT	NOT NULL	—	Message text or tool call JSON
tool_calls	JSONB	NULL	NULL	Tool calls issued by assistant in this turn
tool_results	JSONB	NULL	NULL	Tool execution results for tool_result role
token_count	INTEGER	NOT NULL	0	Tokens in this message
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: agent_action_log ← APPEND-ONLY

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
session_id	CHAR(36)	NOT NULL	—	FK → agent_session.id
user_id	CHAR(36)	NOT NULL	—	—

Column	Type	Null	Default	Description
action_type	VARCHAR(40)	NOT NULL	—	See action type table below
action_params	JSONB	NOT NULL	—	Input params to action
result	JSONB	NULL	NULL	Action result (truncated to 1KB for log)
status	VARCHAR(20)	NOT NULL	—	success|failed|skipped
error_message	TEXT	NULL	NULL	—
duration_ms	INTEGER	NULL	NULL	—
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—

Table: agent_memory

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
user_id	CHAR(36)	NOT NULL	—	FK → account.id
memory_type	VARCHAR(30)	NOT NULL	—	preference|goal|context|person|fact
content	TEXT	NOT NULL	—	Plain text memory description
embedding	vector(1536)	NULL	NULL	pgvector embedding for semantic search

Column	Type	Null	Default	Description
source_session_id	CHAR(36)	NULL	NULL	Session this memory was extracted from
confidence	DECIMAL(3,2)	NOT NULL	1.00	—
expires_at	TIMESTAMPTZ	NULL	NULL	NULL = permanent; time-limited for temp context
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
deleted_at	TIMESTAMPTZ	NULL	NULL	User-deletable (right to erasure)

16.3 Tool Catalog

Tool Name	Description	Inputs	Side Effects	Auth Gate
search_portfolio	Retrieve portfolio items matching a query	query, item_types?, status?	None — read only	user scope only
get_blocked_stories	List all currently blocked stories across WBSes	wbs_ids? (default: all)	None	user scope
search_market	Search marketplace listings	query, category?, max_results	None	user scope
create_story	Create a new WBS story	wbs_id, title, story_type, priority	Creates DB record; emits event	Requires user confirmation if wbs not in user workspace
update_story_status	Change a story status	story_id, new_status, note?	Updates story; emits event	Story must belong to user
run_sprint_plan	Generate AI sprint plan	wbs_id, sprint_id	Sets sprint plan; emits event	Pro+ only
get_okr_status	Retrieve OKR scores and status	okr_ids? or owner_id	None	user scope
update_okr_checkin	Record OKR check-in	okr_id, kr_updates[], note?	Creates check-in record	okr must belong to user

Tool Name	Description	Inputs	Side Effects	Auth Gate
get_wallet_summary	Retrieve wallet balance overview	—	None	user scope — no raw ledger
create_invoice	Create and optionally send invoice	client_details, line_items[], send?	Creates invoice; optionally sends email	Pro+ only
search_market_for_blocked_story	Find providers who can unblock a story	story_id (reads blocker_note)	None	user scope
get_portfolio_health	Get health score breakdown	portfolio_id	None	user scope
generate_standup	Generate daily standup summary from stories	wbs_id, lookback_hours?	None — generates text only	user scope
draft_proposal	Draft an AO governance proposal	ao_id, proposal_type, details	None — returns draft text only	member of AO
advance_idea_stage	Advance an idea to next pipeline stage	idea_id	Advances stage; emits event	Requires exit criteria met

16.4 Context Builder — Go Implementation

```go
// ai-service/internal/context/builder.go

type ContextBuilder struct {
    portfolioSvc PortfolioClient
    wbsSvc       WBSClient
    workSvc      WorkClient
    exchangeSvc  ExchangeClient
}

func (b *ContextBuilder) Build(ctx context.Context, userID, workspaceID string)
(*AgentContext, error) {
    // Fire all gRPC calls in parallel
    var (wg sync.WaitGroup
    var portfolio  PortfolioSummary
    var blocked    []Story
    var okrs       []OKRSummary
    var wallet     WalletSummary
    var recentActs []AgentAction)

    for _, fn := range []func(){
        func() { portfolio, _ = b.portfolioSvc.GetSummary(ctx, userID) },
        func() { blocked, _ = b.wbsSvc.GetBlockedStories(ctx, workspaceID, 10) },
        func() { okrs, _ = b.workSvc.GetActiveOKRs(ctx, userID) },
        func() { wallet, _ = b.exchangeSvc.GetWalletSummary(ctx, userID) },
    } {
        wg.Add(1)
        go func(f func()) { defer wg.Done(); f() }(fn)
```

```
    }
    wg.Wait()

    return &AgentContext{
        UserID:      userID,
        Portfolio:   portfolio,
        Blocked:     blocked,
        OKRs:        okrs,
        Wallet:      wallet,
        TokenBudget: 2500,
    }, nil
}

func (b *ContextBuilder) BuildSystemPrompt(ac *AgentContext, userPrefs UserPrefs)
string {
    return fmt.Sprintf(`You are Kogi, a personal AI assistant for independent
worker %s.

PORTFOLIO CONTEXT (as of %s):
- Active projects: %d (health score: %.0f/100)
- Blocked stories: %d needing resolution
- Active OKRs: %d (avg score: %.2f)
- Wallet balance: $%.2f available

CAPABILITIES: You can search the portfolio, create and update stories, search the
marketplace for services, generate sprint plans, check OKR status, and more.
Use tools when the user asks you to take action. Ask for confirmation before
irreversible actions. Show your reasoning.

COMMUNICATION STYLE: %s. Concise. Factual. Worker-first.

RECENT ACTIONS (last 5): %s`,
        ac.UserID, time.Now().Format("2006-01-02 15:04"),
        len(ac.Portfolio.Items), ac.Portfolio.HealthScore,
        len(ac.Blocked), len(ac.OKRs), avgOKRScore(ac.OKRs),
        ac.Wallet.FiatAvailable,
        userPrefs.CommunicationStyle, formatRecentActions(ac.RecentActions))
}
```

16.5 Streaming Response — SSE

```
// ai-service/internal/handler/chat.go

func (h *ChatHandler) StreamChat(c *gin.Context) {
    userID := c.GetString("user_id")
    sessionID := c.Param("sessionID")
    var req ChatRequest
    c.ShouldBindJSON(&req)

    // Build context and system prompt
    ac, _ := h.contextBuilder.Build(c.Request.Context(), userID, req.WorkspaceID)
    sysPrompt := h.contextBuilder.BuildSystemPrompt(ac, req.UserPrefs)
```

```
    // Retrieve conversation history (sliding window)
    history, _ := h.sessionRepo.GetMessageHistory(c.Request.Context(), sessionID,
20)

    // Build messages array
    messages := buildMessages(sysPrompt, history, req.Content)

    // Set SSE headers
    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("X-Accel-Buffering", "no")

    // Call Claude API with streaming
    stream, _ := h.llm.StreamMessages(c.Request.Context(), StreamRequest{
        Model:    "claude-sonnet-4-20250514",
        Messages: messages,
        Tools:    h.toolRegistry.Definitions(),
        MaxTokens: 4096,
    })

    var fullContent strings.Builder
    flusher := c.Writer.(http.Flusher)
    for delta := range stream {
        switch delta.Type {
        case "text_delta":
            fullContent.WriteString(delta.Text)
            c.SSEvent("text_delta", delta.Text)
            flusher.Flush()
        case "tool_use":
            result, err := h.toolRouter.Execute(c.Request.Context(), userID,
delta.ToolCall)
            c.SSEvent("tool_result", ToolResultEvent{Tool: delta.ToolCall.Name,
Result: result, Error: errStr(err)})
            flusher.Flush()
        case "message_stop":
            c.SSEvent("complete", CompletedEvent{SessionID: sessionID})
            flusher.Flush()
        }
    }
    // Persist assistant message
    h.sessionRepo.AppendMessage(sessionID, "assistant", fullContent.String())
}
```

16.6 AI Agent API

Endpoint	Method	Auth	Description
POST /api/v1/ai/sessions	JWT	Idem	Create AI agent session
GET /api/v1/ai/sessions	JWT	—	List sessions (paginated, most recent first)
GET /api/v1/ai/sessions/:id	JWT	—	Get session with message history

Endpoint	Method	Auth	Description
POST /api/v1/ai/sessions/:id/messages	JWT	Idem	Send message (SSE stream response)
GET /api/v1/ai/sessions/:id/messages	JWT	—	Get full message history
DELETE /api/v1/ai/sessions/:id	JWT	—	End session
GET /api/v1/ai/actions	JWT	—	List AI actions taken on my account
GET /api/v1/ai/memory	JWT	—	List AI memories for my account
DELETE /api/v1/ai/memory/:id	JWT	—	Delete specific memory (right to erasure)
DELETE /api/v1/ai/memory	JWT	—	Delete ALL memories (full right to erasure)
POST /api/v1/ai/synthesize/idea/:id	JWT	—	Trigger manual idea synthesis
POST /api/v1/ai/synthesize/okr/:id	JWT	—	Trigger manual OKR narrative generation
GET /api/v1/ai/credits	JWT	—	Get AI credit balance and usage this month

16.7 Proactive Alert Engine

Alert Type	Trigger Condition	Detection Method	Action
Blocked story escalation	Story blocked for > 24h without resolution	Nightly sweep on story WHERE status=blocked AND blocked_since < NOW()-24h	Push notification + in-app; AI searches Market for providers; suggestion card in session context
Sprint at-risk	Active sprint: < 40% stories complete with < 30% time remaining, OR > 2 blocked stories	Sprint status event + hourly sweep on active sprints	Team notification; AI generates recovery options
OKR off-track	OKR score < 50% with < 30% of period remaining	Weekly OKR scoring job result	Email summary + in-app alert; AI generates narrative on path to recovery
Escrow approaching auto-release	Escrow review_deadline within 24h with no buyer response	Hourly sweep on escrow WHERE status=delivery_submitted	Buyer push notification + email; reminder every 6h
Invoice overdue	Invoice not paid 7+ days after due_date	Nightly sweep on open invoices	Email to issuer: payment status; suggested follow-up message generated by AI
Portfolio health decline	portfolio.health_score drops > 10 points in 7 days	health_score_updated event with delta check	AI alert with breakdown of declined factors + suggested actions

Alert Type	Trigger Condition	Detection Method	Action
Governance vote closing	Proposal voting_closes_at within 24h, member has not voted	Hourly sweep on open proposals	Push notification to all members who haven't voted

§17 · SEARCH

Search & Discovery System

The Search System provides full-text, faceted, and semantic search across all platform entities. It uses Elasticsearch 8.12 with dense vector (kNN) search for semantic capabilities, and a NATS-driven indexing pipeline that keeps ES in sync with PostgreSQL.

17.1 Index Catalog

Index Name	Source Service	Primary Use	Key Analyzers	Semantic?
listings	market-svc	Marketplace search, recommendations	standard + ngram + lowercase	Yes — 1536-dim Claude embedding
profiles	user-svc	User/provider discovery	standard + lowercase, edge-ngram for handle prefix	Yes — 1536-dim
spaces	community-svc	Community discovery	standard + lowercase	No
ideas	studio-svc	Idea discovery (public ideas only)	standard + lowercase	Yes — 1536-dim
portfolios	portfolio-svc	Public portfolio discovery	standard	No
stories	wbs-svc	Cross-project story search	standard + lowercase	No
ao	ao-svc	AO discovery	standard + lowercase + synonym analyzer	No

17.2 Indexing Pipeline

```
// search-service/internal/indexer/pipeline.go
// Event-driven indexing: NATS event → ES doc upsert

// Indexable events and their ES operations:
// market.listing.created|updated  → upsert listings/{id}
// market.listing.deleted          → delete listings/{id}
// user.profile.updated            → upsert profiles/{account_id}
// community.space.created|updated → upsert spaces/{id}
// studio.idea.updated (public)    → upsert ideas/{id}

func (p *IndexPipeline) HandleEvent(ctx context.Context, event PlatformEvent)
error {
    doc, indexName, docID, op, err := p.buildESDoc(ctx, event)
```

```
    if err != nil { return fmt.Errorf("build doc: %w", err) }

    // Embed if needed (listings, profiles, ideas)
    if requiresEmbedding(indexName) {
        text := buildEmbeddingText(indexName, doc)
        embedding, err := p.embedder.Embed(ctx, text)
        if err != nil {
            log.Warn("embedding failed, indexing without vector", "err", err)
        } else {
            doc["embedding"] = embedding
        }
    }

    switch op {
    case "upsert":
        _, err = p.es.Index(indexName, strings.NewReader(toJSON(doc)),
p.es.Index.WithDocumentID(docID))
    case "delete":
        _, err = p.es.Delete(indexName, docID)
    }
    return err
}

// Backfill job: runs on service restart if es_indexed_at IS NULL
func (p *IndexPipeline) Backfill(ctx context.Context, indexName string) error {
    var page int
    for {
        rows, done, _ := p.fetchUnindexed(ctx, indexName, page, 500)
        for _, row := range rows { p.HandleEvent(ctx, syntheticEvent(indexName,
row)) }
        if done { break }
        page++
        time.Sleep(100 * time.Millisecond) // rate-limit backfill
    }
    return nil
}
```

17.3 Search Query Architecture

```
// Composite listing search query (ES 8.12 bool query)
func buildListingQuery(req SearchRequest) map[string]interface{} {
    must := []map[string]interface{}{}
    filter := []map[string]interface{}{
        {"term": {"status": "active"}},
    }

    // Full-text or semantic
    if req.Query != "" {
        if req.Semantic && req.QueryEmbedding != nil {
            // kNN semantic search
            must = append(must, map[string]interface{}{
                "knn": map[string]interface{}{
                    "field": "embedding", "query_vector": req.QueryEmbedding,
```

```
                    "k": 50, "num_candidates": 200,
                },
            })
        } else {
            must = append(must, map[string]interface{}{
                "multi_match": map[string]interface{}{
                    "query":  req.Query,
                    "fields": []string{"title^3", "title.ngram^1",
"description^2", "tags^2"},
                    "type":   "best_fields", "fuzziness": "AUTO",
                },
            })
        }
    }

    // Facet filters
    if req.Category != "" { filter = append(filter, termFilter("category",
req.Category)) }
    if req.Availability != "" { filter = append(filter,
termFilter("availability_status", req.Availability)) }
    if req.PriceMin > 0 || req.PriceMax > 0 {
        filter = append(filter, rangeFilter("pricing_min", req.PriceMin,
req.PriceMax))
    }
    if req.RadiusKM > 0 && req.Lat != 0 {
        filter = append(filter, geoFilter("location_geo", req.Lat, req.Lon,
req.RadiusKM))
    }

    return map[string]interface{}{
        "query": map[string]interface{}{"bool": map[string]interface{}{"must":
must, "filter": filter}},
        "sort":  buildSort(req.SortBy, req.SortDir),
        "aggs":  buildFacetAggs(),
        "from":  req.Page * req.PageSize, "size": req.PageSize,
        "highlight": map[string]interface{}{"fields":
map[string]interface{}{"title": map[string]interface{}{"fragment_size": 100}}},
    }
}
```

17.4 Search API

Endpoint	Method	Auth	Description
GET /api/v1/search	JWT	—	Global search across all entity types
GET /api/v1/search/listings	Public	—	Listing search with facets (ES-backed)
GET /api/v1/search/listings/semantic	JWT	—	Semantic listing search (vector kNN)
GET /api/v1/search/profiles	JWT	—	Profile/provider search
GET /api/v1/search/spaces	Public	—	Community space search

Endpoint	Method	Auth	Description
GET /api/v1/search/stories	JWT	—	WBS story search across all user workspaces
GET /api/v1/search/ideas	JWT	—	Idea search (private scope + public ideas)
GET /api/v1/search/ao	Public	—	AO / cooperative search
GET /api/v1/search/suggestions	JWT	—	Autocomplete suggestions (Redis prefix cache)

§18 · NOTIFICATIONS

Notification & Alert System

The Notification System fans out domain events to users via multiple channels: in-app (SSE), email, push (mobile), and Slack integration. It respects per-user delivery preferences, rate limits per channel, and batching for digest delivery.

18.1 Database Schema

Table: notification

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
user_id	CHAR(36)	NOT NULL	—	FK → account.id
notification_type	VARCHAR(60)	NOT NULL	—	Type enum — see catalog below
title	VARCHAR(300)	NOT NULL	—	—
body	TEXT	NOT NULL	—	—
action_url	VARCHAR(512)	NULL	NULL	Deep link URL for action
icon_type	VARCHAR(30)	NOT NULL	—	story|order|escrow|okr|proposal|general
reference_id	CHAR(36)	NULL	NULL	Related entity UUID
reference_type	VARCHAR(30)	NULL	NULL	Entity type string
priority	VARCHAR(10)	NOT	'normal'	critical|high|normal|low

Column	Type	Null	Default	Description
		NULL		
is_read	BOOLEAN	NOT NULL	false	—
read_at	TIMESTAMPTZ	NULL	NULL	—
channels_sent	JSONB	NOT NULL	'[]'	[{channel, status, sent_at, failure_reason?}]
metadata	JSONB	NOT NULL	'{}'	Extra context for UI rendering
created_at	TIMESTAMPTZ	NOT NULL	NOW()	—
expires_at	TIMESTAMPTZ	NULL	NULL	Auto-hide after expiry (e.g. review reminders)

Table: notification_preference

Column	Type	Null	Default	Description
id	CHAR(36)	NOT NULL	gen_uuid()	PK
user_id	CHAR(36)	NOT NULL	—	FK → account.id; UNIQUE with category
category	VARCHAR(40)	NOT NULL	—	See preference categories below
in_app_enabled	BOOLEAN	NOT NULL	true	—
email_enabled	BOOLEAN	NOT	true	—

Column	Type	Null	Default	Description
		NULL		
push_enabled	BOOLEAN	NOT NULL	true	—
digest_mode	VARCHAR(10)	NOT NULL	'realtime'	realtime|daily|weekly|off
quiet_hours_start	SMALLINT	NULL	NULL	0–23 (hour of day, user local time)
quiet_hours_end	SMALLINT	NULL	NULL	—
updated_at	TIMESTAMPTZ	NOT NULL	NOW()	—

18.2 Notification Type Catalog

Category	Notification Types	Default Channels	Priority
WBS / Work	story_blocked, story_assigned, story_deadline_approaching, sprint_at_risk, sprint_started, sprint_closed	in-app + email	story_blocked=high; sprint_at_risk=high; rest=normal
Market	order_received, order_accepted, order_delivered, order_completed, order_disputed, review_received	in-app + email	order_received=high; all others=normal
Exchange	payment_received, escrow_funded, escrow_auto_release_24h, invoice_paid, invoice_overdue, distribution_completed	in-app + email	escrow_auto_release=critical; payment_received=high; rest=normal
Portfolio	health_score_declined, item_status_changed, collaborator_added	in-app	health_score_declined=normal
Community	connection_request, new_dm,	in-app only (email for DM only)	connection_request=normal; mention=normal

Category	Notification Types	Default Channels	Priority
	space_mention, reply_to_post		
AI Agent	ai_blocked_story_resolved, ai_okr_off_track, ai_invoice_overdue, ai_sprint_risk	in-app	varies by alert type
Governance	proposal_submitted, vote_reminder, proposal_passed, proposal_failed, member_joined, charter_amended	in-app + email	vote_reminder=high; proposal_passed=high; rest=normal
Account	login_from_new_location, password_changed, plan_changed, mfa_disabled	email only (security)	critical
Crowdfunding	campaign_funded, campaign_failed, new_backer, campaign_ending_24h	in-app + email	campaign_funded=high; rest=normal

18.3 Fanout Architecture

```
// notification-service/internal/fanout/processor.go

// NATS subscription on all platform events
func (p *FanoutProcessor) Start(ctx context.Context) {
    p.nats.QueueSubscribe("platform.>", "notification-workers", func(msg
*nats.Msg) {
        event := parseEvent(msg.Data)
        recipients, notif := p.buildNotification(ctx, event)
        if notif == nil { return } // event has no notification mapping

        for _, userID := range recipients {
            prefs, _ := p.prefsRepo.Get(ctx, userID, notif.Category)
            if !shouldDeliver(prefs, notif) { continue }

            // Persist notification
            n := *notif
            n.ID = uuid.New().String()
            n.UserID = userID
            p.notifRepo.Insert(ctx, &n)

            // Deliver to each enabled channel
            if prefs.InAppEnabled { p.sendInApp(ctx, userID, &n) }
            if prefs.EmailEnabled && notif.Priority != "low" { p.sendEmail(ctx,
userID, &n) }
            if prefs.PushEnabled && notif.Priority == "critical" { p.sendPush(ctx,
userID, &n) }
```

```
        }
    })
}

// In-app delivery: push to SSE connection if active
func (p *FanoutProcessor) sendInApp(ctx context.Context, userID string, n
*Notification) {
    // Publish to NATS SSE subject; SSE handler in api-gateway picks up
    p.nats.Publish(fmt.Sprintf("sse.user.%s", userID), toJSON(SSEEvent{
        Type: "notification", Payload: n,
    }))
}

// Rate limiting: max 10 emails/day per user, 50 in-app/day
func shouldDeliver(prefs NotifPrefs, n *Notification) bool {
    if n.Priority == "critical" { return true } // critical always through
    key := fmt.Sprintf("notif-rate:%s:%s", prefs.UserID, n.Channel)
    count, _ := p.redis.Incr(ctx, key).Result()
    if count == 1 { p.redis.Expire(ctx, key, 24*time.Hour) }
    limits := map[string]int64{"in_app": 50, "email": 10, "push": 20}
    return count <= limits[n.Channel]
}
```

18.4 Email Delivery

Template	Trigger	SLA	Provider
email_verification	user.registered	< 1 min	AWS SES
welcome_series_1	user.email_verified (+ 1h)	< 1h	AWS SES
password_reset	password reset requested	< 1 min (priority queue)	AWS SES
order_received	market.order.placed	< 2 min	AWS SES
payment_received	exchange.escrow.released	< 5 min	AWS SES
invoice_overdue	nightly sweep (3 days overdue)	< 1h	AWS SES
governance_vote_reminder	proposal vote closing in 24h	< 1h	AWS SES
campaign_funded	fund.campaign.funded	< 5 min	AWS SES
security_alert	anomalous_login, mfa_disabled, password_changed	< 1 min (critical)	AWS SES
weekly_digest	Every Sunday 09:00 user local time	< 1h	AWS SES

18.5 Notification API

Endpoint	Method	Auth	Description
GET /api/v1/notifications	JWT	—	List notifications (paginated, unread first)
GET /api/v1/notifications/unread-count	JWT	—	Get unread count (cached in Redis, O(1))
PUT /api/v1/notifications/:id/read	JWT	—	Mark notification as read
PUT /api/v1/notifications/read-all	JWT	—	Mark all as read
DELETE /api/v1/notifications/:id	JWT	—	Dismiss notification
GET /api/v1/notifications/preferences	JWT	—	Get delivery preferences
PUT /api/v1/notifications/preferences	JWT	—	Update delivery preferences
SSE /api/v1/notifications/stream	JWT(SSE)	—	Real-time notification stream (Server-Sent Events)

§19 · ANALYTICS

Analytics & Reporting System

The Analytics System ingests all platform events via NATS, stores enriched data in ClickHouse, and serves dashboards via the analytics-service (Scala/Akka). It provides user-facing portfolio analytics, operator platform metrics, and AI-readable aggregate signals for the context builder and alert engine.

19.1 ClickHouse Schema

Table: events (raw event stream)

```
-- ClickHouse DDL: events table (raw ingestion)
CREATE TABLE events (
    event_id       UUID,
    event_type     LowCardinality(String),
    source_module  LowCardinality(String),
    payload        String,        -- JSON
    user_id        Nullable(UUID),
    workspace_id   Nullable(UUID),
    created_at     DateTime64(3, 'UTC')
) ENGINE = MergeTree()
PARTITION BY toYYYYMM(created_at)
ORDER BY (event_type, created_at)
TTL created_at + INTERVAL 2 YEAR;

-- Aggregated: daily_user_activity (materialized view)
CREATE MATERIALIZED VIEW daily_user_activity
ENGINE = AggregatingMergeTree()
PARTITION BY toYYYYMM(date)
ORDER BY (user_id, date)
AS SELECT
    toDate(created_at)            AS date,
    user_id,
    countState()                  AS event_count,
    uniqState(source_module)      AS modules_used,
    minState(created_at)          AS first_event,
    maxState(created_at)          AS last_event
FROM events WHERE user_id IS NOT NULL
GROUP BY date, user_id;
```

Table: story_metrics (pre-aggregated WBS analytics)

```
CREATE TABLE story_metrics (
    date           Date,
    wbs_id         UUID,
    user_id        UUID,
    story_type     LowCardinality(String),
    from_status    LowCardinality(String),
    to_status      LowCardinality(String),
```

```
    count          UInt32,
    sum_points     UInt32,
    avg_duration_min Float32
) ENGINE = SummingMergeTree(count, sum_points)
ORDER BY (wbs_id, date, story_type, from_status, to_status);

-- Used for: burndown, velocity charts, cycle time by status, throughput
```

Table: market_metrics

```
CREATE TABLE market_metrics (
    date           Date,
    category       LowCardinality(String),
    listing_type   LowCardinality(String),
    event_type     LowCardinality(String),    -- view|save|order|complete|dispute
    provider_id    UUID,
    amount_usd     Decimal(18,2),
    count          UInt32
) ENGINE = SummingMergeTree(count, amount_usd)
ORDER BY (date, category, event_type, provider_id);
```

Table: exchange_metrics

```
CREATE TABLE exchange_metrics (
    date           Date,
    wallet_id      UUID,
    entry_type     LowCardinality(String),
    amount_usd     Decimal(18,2),
    count          UInt32
) ENGINE = SummingMergeTree(count, amount_usd)
ORDER BY (date, wallet_id, entry_type);

-- Monthly income/expense summary per user:
-- SELECT toStartOfMonth(date) as month,
--     sumIf(amount_usd, entry_type IN ('credit','hold_release')) as income,
--     sumIf(amount_usd, entry_type IN ('fee','debit')) as fees
-- FROM exchange_metrics WHERE wallet_id = ? GROUP BY month
```

19.2 Analytics Service Architecture (Scala/Akka)

```
// analytics-service/src/main/scala/io/kogi/analytics/AnalyticsService.scala

object AnalyticsService extends App {
  implicit val system: ActorSystem[_] = ActorSystem(Behaviors.empty, "analytics")

  // Akka Streams pipeline: NATS → parse → enrich → ClickHouse bulk insert
```

```
  val natsSource: Source[ConsumerRecord, _] = NATSSource.create("PLATFORM",
"analytics-worker")

  val pipeline: RunnableGraph[_] = natsSource
    .map(record => parseEvent(record.data))
    .filter(event => event.isDefined)
    .map(_.get)
    .map(enrichEvent)                          // add user plan, workspace type
from cache
    .groupedWithin(500, 5.seconds)             // batch for bulk insert
    .mapAsync(4)(batch => clickHouse.bulkInsert("events", batch))
    .toMat(Sink.ignore)(Keep.right)

  pipeline.run()

  // HTTP server: serve analytics queries to analytics-service REST endpoint
  val routes = AnalyticsRoutes(clickHouse, redisCache)
  Http().newServerAt("0.0.0.0", 8081).bind(routes)
}

// Cache layer: analytics queries cached in Redis by (query_hash, time_range)
// TTL: hourly metrics → 5min cache; daily → 30min; monthly → 2h
```

19.3 User-Facing Analytics Endpoints

Endpoint	Method	Auth	Description
GET /api/v1/analytics/portfolio/health-trend	JWT	—	Health score history (last 90d)
GET /api/v1/analytics/portfolio/activity	JWT	—	Activity heatmap (contributions per day)
GET /api/v1/analytics/wbs/velocity	JWT	—	Sprint velocity chart (last N sprints)
GET /api/v1/analytics/wbs/burndown/:sprintId	JWT	—	Active sprint burndown chart
GET /api/v1/analytics/wbs/cycle-time	JWT	—	Average cycle time per status
GET /api/v1/analytics/wbs/throughput	JWT	—	Stories completed per week (last 12w)
GET /api/v1/analytics/market/earnings	JWT	—	Earnings trend by month
GET /api/v1/analytics/market/order-funnel	JWT	—	Orders by status: viewed→inquired→ordered→completed
GET /api/v1/analytics/exchange/income-summary	JWT	—	Monthly income/expense P&L summary
GET /api/v1/analytics/exchange/invoices-aging	JWT	—	Outstanding invoice aging buckets

Endpoint	Method	Auth	Description
GET /api/v1/analytics/okr/progress	JWT	—	OKR progress over time for period
GET /api/v1/analytics/ai/usage	JWT	—	AI credit usage by month + breakdown by tool

19.4 Platform Operator Metrics

Metric	Source	Aggregation	Dashboard Panel
MAU / DAU / WAU	ClickHouse daily_user_activity	Count distinct user_id by time window	Growth overview
MRR / ARR	PostgreSQL account plan + Stripe events	Sum of plan prices by active subscription	Revenue overview
GMV (Gross Marketplace Volume)	ClickHouse exchange_metrics WHERE entry_type=credit	Sum(amount_usd) by period	Market health
New registrations	ClickHouse events WHERE event_type=user.registered	Count by period + plan breakdown	Acquisition
D7 / D30 retention	ClickHouse: cohort analysis on daily_user_activity	Users active on day 7/30 / users registered in cohort	Retention
Plan distribution	PostgreSQL account	Count by plan, live	Revenue mix
AI credits consumed	ClickHouse events WHERE event_type=ai.session.*	Sum token_count by period	AI costs
P95 API latency	Prometheus histogram	histogram_quantile(0.95, …)	Platform health
Error rate	Prometheus counter	rate(http_requests_total{status=~"5.."}[5m])	Platform health
NATS consumer lag	NATS monitoring API	Max lag across all consumer groups	Event bus health

§20 · SECURITY

Security Architecture

Security is layered across all platform levels: network (TLS/mTLS/WAF), service (JWT/RBAC/rate-limiting), data (encryption-at-rest/KMS/field-level), and operational (secret rotation/audit logs/pen testing). This section defines the authoritative security design.

20.1 Encryption Architecture

Layer	Algorithm	Key Management	Rotation
TLS (transit)	TLS 1.3 minimum; TLS 1.2 rejected	AWS ACM — auto-renewed	Automatic on cert expiry (90d)
mTLS (service mesh)	TLS 1.3 via Istio	Istio Citadel — SPIFFE SVID certs	Every 24h automatically by Istio
JWT signing	HMAC-SHA256, 256-bit key	AWS Secrets Manager	Manual rotation every 90d; 24h overlap window
Passwords	Argon2id: time=3 mem=65536 threads=4 keylen=32	Not stored — hash only	N/A — never recover, only reset
Data at rest (RDS)	AES-256 (AWS managed)	AWS KMS — CMK per environment	Annual KMS key rotation (automatic)
Field-level encryption (PII, KYC)	AES-256-GCM	AWS KMS envelope encryption: DEK wrapped by CMK	DEK rotated quarterly; KMS CMK annually
OAuth tokens stored	AES-256-GCM	Per-row DEK, CMK in KMS	On OAuth re-auth (user action)
MFA secrets (TOTP)	AES-256-GCM	Per-record DEK	On MFA re-enrollment
S3 at rest	AES-256-SSE-S3	AWS S3-managed	N/A

20.2 API Gateway Security Middleware Stack

```
// api-gateway/internal/middleware/chain.go
// Middleware executes in this exact order for every request:

// 1. HTTPS redirect (HTTP → HTTPS 308 redirect)
// 2. CORS: allow-list of permitted origins; preflight cache 24h
// 3. Request ID: inject X-Request-ID header if absent
// 4. Rate limiter (token bucket per IP for public; per user_id for JWT)
//    Limits:
//    - Unauthenticated: 60 req/min per IP (hard), 10 req/min per IP on /auth/*
(strict)
```

```
//    - Authenticated: 300 req/min per user (normal), 30 req/min on /exchange/*
(financial)
//    - AI endpoints: 60 req/hour per user (credits-based secondary check in
ai-svc)
// 5. JWT validation:
//    a. Parse header — reject alg=none, reject non-HS256
//    b. Verify HMAC signature (current key, then previous key in rotation window)
//    c. Verify exp > now()
//    d. Check Redis revocation set: SISMEMBER revoked {jti} → 0 required
//    e. Extract claims into context
// 6. Idempotency key check (mutating requests only):
//    - Check Redis: GET idempotency:{key} → if HIT return cached 200/response
//    - Continue if MISS; set key after handler returns (TTL = 25h)
// 7. Request size limit: 10MB max body (except /media endpoints: 100MB)
// 8. Content-Type validation (POST/PUT: must be application/json)
// 9. Forward to service via gRPC (secure channel via Istio mTLS)
```

20.3 Input Validation

```
// All request structs use validator tags + custom business validators
// Example: Create Story request

type CreateStoryRequest struct {
    WBSID       string         `json:"wbs_id"       validate:"required,uuid4"`
    Title       string         `json:"title"
validate:"required,min=1,max=500"`
    StoryType   string         `json:"story_type"
validate:"required,oneof=feature bug capability..."`
    Priority    string         `json:"priority"
validate:"required,oneof=critical high medium low"`
    Points      *int           `json:"story_points"
validate:"omitempty,min=0,max=1000"`
    Description *string        `json:"description"
validate:"omitempty,max=102400"`
    DueDate     *time.Time     `json:"due_date"     validate:"omitempty,gttoday"`
    Labels      []string       `json:"labels"
validate:"omitempty,max=20,dive,min=1,max=50"`
}

// Sanitization rules applied AFTER validation:
// - Title: trim whitespace; strip HTML (no rich text allowed in story titles)
// - Description: Markdown sanitized via bluemonday to allow only safe markdown
//   No <script>, no event handlers, no external image URLs
// - Labels: lowercase; remove duplicates

// SQL injection: impossible — ALL DB interactions via pgx parameterized queries.
//   Raw query strings are prohibited in code review. ESLint/golangci-lint
enforces this.
```

20.4 Three-Layer Authorization

```
// Layer 1: Platform-level (in JWT)
// - roles claim: ["user"] → standard access
// - roles claim: ["user","moderator"] → can moderate community content
// - roles claim: ["user","admin"] → platform admin; can access /admin/* endpoints

// Layer 2: Workspace-level (checked by service from DB)
// - workspace_member row with role: owner|admin|member|guest
// - Fetched from DB on first request; cached in Redis for 5 minutes:
ws-role:{ws_id}:{user_id}

// Layer 3: Resource-level (checked by service for each resource)
// Examples:
// - Portfolio: portfolio.owner_id = user_id OR portfolio_collaborator row exists
// - Story: story.wbs.portfolio_item.portfolio.workspace_id = user workspace_id
// - AO proposal: ao_member.ao_id = proposal.ao_id AND ao_member.user_id = user_id
AND role allows voting
// - Escrow: escrow.buyer_id = user_id OR escrow.seller_id = user_id

// Authorization helper (every service handler calls this)
func requireOwnerOrCollaborator(ctx context.Context, portfolioID, userID string,
minRole string) error {
    cacheKey := fmt.Sprintf("portfolio-access:%s:%s", portfolioID, userID)
    if cached, _ := redis.Get(ctx, cacheKey).Result(); cached != "" {
        role := cached
        if !roleAllows(role, minRole) { return ErrForbidden }
        return nil
    }
    role, err := portfolioRepo.GetUserRole(ctx, portfolioID, userID)
    if err != nil { return ErrForbidden }
    redis.Set(ctx, cacheKey, role, 5*time.Minute)
    if !roleAllows(role, minRole) { return ErrForbidden }
    return nil
}
```

20.5 Security Event Monitoring

Event	Detection	Automated Response	Human Alert
Brute-force login attempt	> 10 failed logins in 15min per account	Lock account for 15min; increment lock duration	Account security team paged if ≥ 5 accounts hit in same hour
Credential stuffing	> 100 failed logins in 5min across different accounts from same IP/ASN	WAF rate-limit IP block (1h); CloudFront rule update	Security team alerted immediately
JWT replay after revocation	Redis SISMEMBER returns 1 on revoked jti	Request rejected 401; incident log written	Security team paged; investigate leaked token
Anomalous financial transaction	Amount > $5,000 from account with 0 prior transactions; or > 3 failed escrow attempts	Hold transaction for manual review; notify account owner	Finance team review within 1h

Event	Detection	Automated Response	Human Alert
Insider threat (admin)	Admin action on financial data outside business hours	Action logged with elevated retention; secondary admin notified	Security team review of all admin actions quarterly
Secret exposed in logs	Regex patterns for credit card, SSN, JWT secrets in log stream (Loki alert rule)	Log line quarantined; alert immediate	Security team P1 response

20.6 Secret Management

```
// No secrets in source code, environment files, or Kubernetes manifests.
// All secrets stored in AWS Secrets Manager.
// Services access secrets via IAM Role (IRSA — IAM Roles for Service Accounts).

// Secret categories and rotation:
// kogi/prod/db/{service}        → PostgreSQL password; rotated every 90d by
Lambda
// kogi/prod/nats/password       → NATS client password; rotated every 90d
// kogi/prod/jwt/signing-key     → JWT HMAC key; manually rotated every 90d
// kogi/prod/jwt/previous-key    → Previous key (24h grace window)
// kogi/prod/stripe/secret-key   → Stripe API key; manually rotated annually
// kogi/prod/anthropic/api-key   → Claude API key; manually rotated annually
// kogi/prod/ses/smtp-password   → AWS SES SMTP; auto-rotated 90d
// kogi/prod/redis/auth-token    → ElastiCache auth token; rotated 180d
// kogi/prod/kms/field-enc-key   → Field encryption master key; annual rotation

// Secret injection at runtime (not build time):
// 1. Pod starts with IAM role annotation (IRSA)
// 2. Init container calls secrets-init binary → fetches secrets → writes to
/run/secrets/
// 3. Application reads from /run/secrets/ at startup
// 4. Secrets Manager change triggers K8s rolling restart via
external-secrets-operator
```

§21 · PERFORMANCE

Performance & Caching Architecture

The Caching Architecture uses a 3-level hierarchy: Redis L1 (hot objects, < 1ms), read-replica PostgreSQL L2 (recent queries, < 10ms), and ClickHouse L3 (aggregations, < 500ms). Cache invalidation is event-driven via NATS subscriptions.

21.1 Redis Cache Key Catalog

Key Pattern	TTL	Value	Invalidated By
portfolio:{id}	5 min	Portfolio JSON with health score	portfolio.created|updated|deleted
portfolio-items:{id}	2 min	Flat item array for portfolio	portfolio.item.created|updated|deleted
portfolio-tree:{id}	2 min	Nested item tree JSON	portfolio.item.moved|created|deleted
portfolio-health:{id}	24h	Health score float + factors	portfolio.health_score_updated
profile:{account_id}	5 min	Public profile JSON	user.profile.updated
profile-handle:{handle}	5 min	Profile ID	user.profile.updated (handle change)
ws-role:{ws_id}:{user_id}	5 min	Role string	workspace_member.updated
portfolio-access:{pid}:{uid}	5 min	Role string	portfolio_collaborator.updated|deleted
wbs-board:{wbs_id}	30 sec	Board state JSON (all columns + stories)	wbs.story.status_changed|created|deleted|moved
sprint-active:{wbs_id}	1 min	Active sprint JSON	wbs.sprint.started|closed
market-listing:{id}	5 min	Listing JSON	market.listing.updated|deleted
listing-views:{id}	1 hr	Integer view count (increment, sync nightly)	N/A — sync job, not event-driven
wallet:{user_id}	30 sec	Wallet balances JSON	exchange.escrow.funded|released|refunded; deposit; withdrawal
notif-unread:{user_id}	5 min	Integer unread count	notification.created; notification.read_all
okr-status:{okr_id}	10 min	OKR score + status	work.okr.scored

Key Pattern	TTL	Value	Invalidated By
agent-session-hot:{session_id}	30 min	Last 10 messages + context (active sessions only)	On each new message
presence:{user_id}	90 sec	Presence JSON + last_seen	WS heartbeat (refresh); disconnect (set offline)
search-suggest:{prefix}	10 min	Autocomplete suggestion array	ES index update for prefix
ao-charter:{ao_id}	10 min	Charter JSON	ao.proposal.passed (charter_amendment)
revoked:{jti}	exp - now	"1" (set on revocation)	auth.session_revoked

21.2 Cache Strategy by Operation Type

Operation Type	Strategy	Rationale	Consistency Window
Portfolio item tree	Read-through + write-invalidate	Tree requires full rebuild on any item change; no point in partial update	< 2 min
WBS board state	Write-invalidate + aggressive refresh (30s TTL)	High mutation rate; board is always nearly current; short TTL is safety net	< 30 sec
Wallet balances	Write-through on all balance changes + low TTL	Financial data correctness is paramount; cache always updated in same TX	< 1 sec (write-through)
User profile	Read-through + event invalidation	Low mutation rate; read-heavy (profile viewed on every listing)	< 5 min
Market listing detail	Read-through + event invalidation	Very low mutation; read by many clients	< 5 min
View counts (listing)	Buffered increment (Redis INCR) + sync to DB nightly	High-frequency increments cannot hit DB on every view	Up to 24h for DB; real-time in Redis
Notification unread count	Increment/decrement (Redis INCRBY/DECRBY) + periodic full count sync	Avoid count query on every nav render	Near real-time
JWT revocation check	Redis SET (not read-through)	Must be O(1); no DB lookup on every request	Synchronous — revocation is immediate
Aggregated analytics	ClickHouse query + Redis cache with long TTL	Analytics queries are expensive; hourly or daily staleness acceptable	5 min to 2h depending on range

21.3 Database Query Optimization

```
-- EXPLAIN ANALYZE example: story board query (most frequent DB read)
-- BEFORE optimization (Seq Scan, 850ms on 500K stories):
-- SELECT * FROM story WHERE wbs_id=$1 AND deleted_at IS NULL
--
-- AFTER optimization (Index Scan, 2ms):
-- Same query, with partial index: idx_story_wbs_status on (wbs_id, status)

-- Critical query: portfolio tree (recursive CTE)
-- Use: explain (analyze, buffers) to identify shared_hit vs read
-- Target: all blocks from shared_hits (cache), 0 from disk for hot portfolios

-- Connection pool query: PgBouncer transaction mode
-- SHOW POOLS: monitor avg_query_count; alert if > 200/s per service
-- SHOW DATABASES: monitor pool_size vs max_connections; alert if > 85% utilized

-- Read replica routing in Go:
func (r *Repo) ReadDB() *sqlx.DB {
    // Round-robin across read replicas; exclude replica if lag > 30s
    for i := 0; i < len(r.replicas); i++ {
        idx := atomic.AddUint64(&r.rIdx, 1) % uint64(len(r.replicas))
        replica := r.replicas[idx]
        lag, _ := r.replicaLag(replica)
        if lag < 30 { return replica }
    }
    return r.primary // fallback to primary if all replicas lagged
}
```

21.4 Performance SLOs

Endpoint Category	p50 Target	p95 Target	p99 Target	Error Rate
GET /api/v1/portfolios/:id	< 20ms	< 80ms	< 200ms	< 0.01%
GET /api/v1/wbs/:id/board	< 30ms	< 100ms	< 300ms	< 0.01%
GET /api/v1/market/listings	< 50ms	< 150ms	< 400ms	< 0.05%
POST /api/v1/market/orders	< 100ms	< 300ms	< 800ms	< 0.01%
POST /api/v1/exchange/escrow (fund)	< 200ms	< 500ms	< 1s	< 0.001%

Endpoint Category	p50 Target	p95 Target	p99 Target	Error Rate
WS first message delivery	< 100ms	< 300ms	< 800ms	< 0.01%
SSE first token (AI chat)	< 1s	< 3s	< 6s	< 0.1%
POST /api/v1/auth/login	< 200ms	< 500ms	< 1s	< 0.01%
GET /api/v1/analytics/*	< 200ms	< 800ms	< 2s	< 0.1%

21.5 N+1 Query Prevention

```
// DataLoader pattern for batch fetching — prevents N+1 on API Gateway
aggregations

type UserLoader struct {
    db    *sqlx.DB
    cache *redis.Client
    mu    sync.Mutex
    keys  []string
    batch chan struct{}
}

// Batch loads: accumulate requests for 10ms, then single query
func (l *UserLoader) Load(ctx context.Context, userID string) (*Profile, error) {
    l.mu.Lock()
    l.keys = append(l.keys, userID)
    if len(l.keys) == 1 {
        // Schedule flush in 10ms
        go func() {
            time.Sleep(10 * time.Millisecond)
            l.flush(ctx)
        }()
    }
    l.mu.Unlock()
    // Wait for batch result
    return <-l.results[userID]
}

// All story list endpoints include author profiles in a single batched DB query
// All notification list endpoints include related entity snapshots in single
query
// Rule: no query inside a loop. Batch loaders are code-review gate.
```

§22 · OBSERVABILITY

Observability, Logging & Tracing

The Observability System follows the three pillars: Metrics (Prometheus), Logs (Loki + structured JSON), and Traces (OpenTelemetry → Tempo). All three are queryable from Grafana dashboards and alert rules.

22.1 Structured Log Schema

```
// Every log line is structured JSON. Fields are standardized across all services.
// PII RULE: user_id only — never log email, name, payment data, or token values.

{
  "ts":           "2025-01-15T14:32:10.123Z",
  "level":        "info",        // trace|debug|info|warn|error|fatal
  "service":      "wbs-service",
  "version":      "1.4.2",
  "trace_id":     "abc123...",   // OpenTelemetry trace ID
  "span_id":      "def456...",
  "request_id":   "req-uuid",
  "user_id":      "user-uuid",   // PII-safe: UUID only, never email
  "workspace_id": "ws-uuid",
  "operation":    "CreateStory",
  "status_code":  201,
  "duration_ms":  45,
  "msg":          "story created",
  "story_id":     "story-uuid",
  "wbs_id":       "wbs-uuid",
  "error":        null           // Only present on error level
}

// Log levels by use case:
// TRACE: per-query SQL with params (DEV + staging only; NEVER production)
// DEBUG: function entry/exit, cache hit/miss, N+1 detection (DEV only)
// INFO:  request complete, background job started/completed, event published
// WARN:  slow query (>500ms), cache miss cascade, near rate-limit, retry attempt
// ERROR: unexpected errors, failed DB operations, failed event publish (with
retry)
// FATAL: unrecoverable errors that cause service restart
```

22.2 Metrics Catalog

Metric Name	Type	Labels	Alert Threshold
http_request_duration_seconds	Histogram	service, method, route, status_code	p95 > 500ms for 5min
http_requests_total	Counter	service, method, route, status_code	error_rate > 1% for 5min

Metric Name	Type	Labels	Alert Threshold
grpc_server_duration_seconds	Histogram	service, method, code	p95 > 300ms for 5min
db_query_duration_seconds	Histogram	service, query_name, result	p95 > 100ms for 5min (warn); > 500ms (alert)
db_connections_active	Gauge	service	> 90% of pool_size for 5min
nats_consumer_lag	Gauge	service, consumer, stream	> 10,000 messages for 10min
redis_hit_rate	Gauge	service, cache_prefix	< 70% for 10min
escrow_pending_count	Gauge	—	> 100 pending for 60min (financial health)
ai_token_cost_usd_total	Counter	model, operation	Weekly budget threshold alert
job_duration_seconds	Histogram	job_name, status	> 2× average for job type
wallet_reconciliation_errors	Gauge	—	> 0 at any time (P0 alert)
ws_connected_clients	Gauge	pod	> 2500 per pod (scale-out trigger)
spam_blocked_total	Counter	channel	spike > 5× baseline (abuse detection)
kubernetes_pod_restart_count	Counter	namespace, pod	> 3 restarts in 15min

22.3 OpenTelemetry Instrumentation

```
// Every Go service is instrumented with OTEL SDK
// Spans are created for: HTTP handlers, gRPC calls, DB queries, NATS
publish/subscribe

// HTTP handler auto-instrumented via otelgin middleware:
router.Use(otelgin.Middleware("wbs-service"))

// DB query spans (custom):
func (r *StoryRepo) GetStory(ctx context.Context, id string) (*Story, error) {
    ctx, span := otel.Tracer("wbs").Start(ctx, "StoryRepo.GetStory")
    defer span.End()
    span.SetAttributes(attribute.String("db.story_id", id))

    story := &Story{}
    if err := r.db.GetContext(ctx, story, "SELECT * FROM story WHERE id=$1", id);
err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, err.Error())
        return nil, err
```

```
    }
    return story, nil
}

// NATS publish span (custom):
func (o *OutboxRelay) publishEvent(ctx context.Context, e OutboxEvent) error {
    ctx, span := otel.Tracer("outbox").Start(ctx, "OutboxRelay.Publish")
    defer span.End()
    span.SetAttributes(
        attribute.String("messaging.system", "nats"),
        attribute.String("messaging.destination", e.EventType),
    )
    return o.nats.PublishMsgWithContext(ctx, &nats.Msg{Subject: e.EventType, Data:
e.Payload})
}

// OTEL Collector configuration:
// Traces → Tempo (S3-backed, 7d retention)
// Metrics → Prometheus
// Logs → (not via OTEL; Loki gets logs via Promtail directly)
// Sampling: 100% for errors and warnings; 1% for successful requests in
production
```

22.4 Grafana Dashboards

Dashboard	Key Panels	Primary Audience	Refresh Rate
Platform Overview	MAU, DAU, MRR, GMV, error rate, p95 latency, active users map	CTO, Product, Engineering Leads	5 min
Service Health	Per-service: RPS, error rate, p95, DB pool utilization, cache hit rate	On-call Engineers	15 sec
Database Health	Per-DB: query volume, slow queries, replica lag, connections, disk utilization	DBAs, Platform Eng	1 min
NATS Health	Per-stream: message rate, consumer lag, unacked messages, DLQ depth	Platform Eng	30 sec
Financial Health	Active escrows, pending distributions, reconciliation errors, GMV trend	Finance, On-call	1 min
AI Platform	Token usage by model/operation, cost/day, AI session count, error rate	AI Eng Lead, Finance	5 min

Dashboard	Key Panels	Primary Audience	Refresh Rate
Security Events	Failed logins, rate-limit hits, WAF blocks, anomaly alerts	Security Team	1 min
Background Jobs	Job duration, success/failure rates, DLQ depth, last run time	Platform Eng	5 min

22.5 Alerting Runbooks

Alert	Severity	Immediate Action	Runbook Link
wallet_reconciliation_errors > 0	P0	Page financial oncall; freeze all wallet mutations; run manual reconciliation	runbooks/financial/reconciliation.md
http_error_rate > 1% for 5min	P1	Page oncall; check recent deploys; prepare rollback	runbooks/platform/high-error-rate.md
nats_consumer_lag > 10K for 10min	P1	Page oncall; check consumer logs; scale notification/analytics pods	runbooks/platform/nats-lag.md
db_connections_active > 90%	P1	Scale service pods if possible; check PgBouncer; prepare emergency connection limit increase	runbooks/database/connection-exhaustion.md
escrow_pending > 100 for 60min	P2	Check auto-release job; verify no stuck escrows; contact support for affected users if needed	runbooks/financial/stuck-escrows.md
ai_token_cost_usd_total exceeds budget	P2	Alert AI team; analyze usage spike; consider rate limiting AI endpoints	runbooks/ai/budget-exceeded.md
kubernetes_pod_restart > 3 in 15min	P2	Check pod logs; identify OOM or crash loop; page oncall	runbooks/platform/pod-crash-loop.md

§23 · JOBS

Background Job System

All recurring background work is registered in the Zig Kernel Scheduler and executed via Go worker goroutines listening on NATS. Job execution uses a distributed lock (Redis SETNX) to guarantee single execution across pods.

23.1 Distributed Lock Pattern

```
// shared/go/job/runner.go

func RunWithLock(ctx context.Context, redis *redis.Client, jobName string, ttl
time.Duration, fn func(ctx context.Context) error) error {
    lockKey := fmt.Sprintf("job-lock:%s", jobName)
    lockVal := uuid.New().String()

    // Try to acquire lock (SETNX equivalent with NX + EX)
    acquired, err := redis.SetNX(ctx, lockKey, lockVal, ttl).Result()
    if err != nil { return fmt.Errorf("redis lock error: %w", err) }
    if !acquired {
        log.Info("job already running on another pod, skipping", "job", jobName)
        return nil
    }

    // Ensure lock release even on panic
    defer func() {
        // Only delete if our lock value (prevent releasing another pod's lock)
        pipe := redis.TxPipeline()
        cur, _ := redis.Get(ctx, lockKey).Result()
        if cur == lockVal { redis.Del(ctx, lockKey) }
        pipe.Exec(ctx)
    }()

    return fn(ctx)
}
```

23.2 Job Catalog

Job Name	Schedule	Service	Timeout	Idempotency	Description
portfolio_health_refresh	0 2 * * *	portfolio-svc	30 min	portfolio_id + date	Recompute health scores for all active portfolios
reputation_score_refresh	0 3 * * *	market-svc	45 min	user_id + date	Recompute reputation score for all active providers
story_blocked_escalation	0 * * * *	wbs-svc	10 min	sweep timestamp	Alert on stories blocked > 24h; trigger AI Market search

Job Name	Schedule	Service	Timeout	Idempotency	Description
sprint_velocity_update	*/30 * * * *	wbs-svc	5 min	sprint_id	Update sprint velocity on closed sprints
auto_release_escrow_sweep	*/5 * * * *	exchange-svc	10 min	escrow_id	Release escrows past review_deadline or auto_release_at
invoice_overdue_sweep	0 9 * * *	exchange-svc	10 min	invoice_id + date	Mark invoices overdue + send reminder notifications
wallet_reconciliation	0 4 * * *	exchange-svc	30 min	date	Verify balance = Σ ledger entries for ALL wallets; alert on mismatch
ledger_audit	0 5 * * 0	exchange-svc	60 min	week_start	Weekly full ledger audit; export to S3 signed JSON
okr_weekly_score	0 8 * * 1	work-svc	20 min	okr_id + week	Rescore all active OKRs; generate AI narrative for off-track ones
ai_weekly_digest	0 9 * * 0	ai-svc	60 min	user_id + week	Generate personalized weekly AI digest per active Pro+ user
notification_digest	0 9 * * *	notification-svc	30 min	user_id + date	Send daily digest to users with digest_mode=daily
campaign_deadline_sweep	*/15 * * * *	fund-svc	15 min	campaign_id	Fund or fail campaigns past end_date
governance_vote_deadline	*/5 * * * *	ao-svc	10 min	proposal_id	Tally and close proposals past voting_closes_at
governance_log_export	0 6 1 * *	ao-svc	60 min	ao_id + month	Monthly governance log export to S3 (signed JSON per AO)
es_index_sync	*/10 * * * *	search-svc	10 min	entity_id	Index any entities with es_doc_id=NULL (backfill stale docs)
session_cleanup	0 1 * * *	auth-svc	10 min	date	Soft-delete sessions past inactivity timeout; purge expired revoked JTIs
deleted_data_purge	0 0 * * 0	all-svcs	120 min	date	Permanently purge records with deleted_at > 30d
analytics_rollup	5 0 * * *	analytics-svc	60 min	date	ClickHouse materialized view refresh; daily aggregate tables
s3_lifecycle_sweep	0 2 * * 0	cdn-svc	30 min	date	Move S3 objects to IA/Glacier per lifecycle policy
1099_threshold_check	0 10 1 * *	exchange-svc	30 min	user_id + month	Check ytd_received threshold for 1099 generation; notify approaching $600

§24 · API DESIGN

API Design Patterns & Contracts

All Kogi APIs follow a consistent design language: envelope response format, standardized error format, cursor-based pagination, idempotency keys on mutations, and semantic versioning via URL path prefix.

24.1 Standard Response Envelope

```
// Success response (single resource)
{
  "ok": true,
  "data": {
    "id": "...",
    // resource fields
  },
  "meta": {
    "request_id": "req-uuid",
    "version": "1.0",
    "timestamp": "2025-01-15T14:32:10Z"
  }
}

// Success response (collection)
{
  "ok": true,
  "data": [...],
  "pagination": {
    "cursor": "opaque-base64-encoded-cursor",
    "has_more": true,
    "total_count": 1247    // only present if ?include_count=true
  },
  "meta": { "request_id": "...", "timestamp": "..." }
}

// Error response
{
  "ok": false,
  "error": {
    "code": "STORY_STATUS_INVALID_TRANSITION",
    "message": "Story cannot transition from DONE to IN_PROGRESS",
    "field": "status",
    "details": { "current_status": "done", "requested_status": "in_progress" },
    "request_id": "req-uuid"
  }
}
```

24.2 Pagination — Cursor-Based

```
// All list endpoints use cursor pagination (NOT page/offset)
// Cursor = base64-encoded JSON: {id: last_seen_id, ts: last_seen_created_at}

// Query parameters:
// ?cursor=<opaque>   // omit for first page
// ?limit=25          // default 25; max 100
// ?sort=asc|desc     // default: desc (newest first)

// Implementation (Go):
func paginateQuery(baseQuery string, cursor *Cursor, limit int, sort string)
(string, []interface{}) {
    params := []interface{}{}
    if cursor != nil {
        direction := ">"
        if sort == "desc" { direction = "<" }
        baseQuery += fmt.Sprintf(" AND (created_at, id) %s ($%d, $%d)",
            direction, len(params)+1, len(params)+2)
        params = append(params, cursor.Timestamp, cursor.ID)
    }
    baseQuery += fmt.Sprintf(" ORDER BY created_at %s, id %s LIMIT $%d",
        strings.ToUpper(sort), strings.ToUpper(sort), len(params)+1)
    params = append(params, limit+1) // fetch one extra to determine has_more
    return baseQuery, params
}
```

24.3 Idempotency Protocol

```
// All POST (create) and PUT (update) endpoints require Idempotency-Key header.
// PATCH (partial update) and DELETE do not require idempotency key.

// Request:
// POST /api/v1/market/orders
// Idempotency-Key: client-generated-uuid-v4
// Content-Type: application/json
// { "listing_id": "...", "amount": 500.00 }

// On first call:
//   1. Check Redis: GET idempotency:{key} → MISS
//   2. Execute request
//   3. Store result: SET idempotency:{key} {response_json} EX 90000 (25h)
//   4. Return response

// On duplicate call (same key):
//   1. Check Redis: GET idempotency:{key} → HIT
//   2. Return cached response with X-Idempotency-Replayed: true header
//   3. Do NOT execute request again

// Idempotency keys must be:
//   - Client-generated UUID v4
//   - Unique per operation (not per session)
//   - Never reused across different request bodies (if body differs → 422)
```

```
// Missing Idempotency-Key on mutable endpoint:
//   → 400 Bad Request: { "error": { "code": "IDEMPOTENCY_KEY_REQUIRED" } }
```

24.4 Versioning Strategy

```
// URL-based versioning: /api/v1/... → /api/v2/...
// v1 will be maintained for 18 months after v2 ships
// Deprecation communicated via:
//   - Sunset: {date} response header on deprecated endpoints
//   - Deprecation: "endpoint deprecated, use v2/xxx instead" response header
//   - Email notification to API key owners 90 days before retirement

// Non-breaking changes (no version bump required):
//   - Adding new optional fields to responses
//   - Adding new optional query parameters
//   - Adding new endpoints
//   - Adding new error codes

// Breaking changes (require new version):
//   - Removing or renaming fields
//   - Changing field types
//   - Changing endpoint paths
//   - Changing required parameters
//   - Changing auth model
```

24.5 gRPC Internal Contracts

```
// proto/portfolio/v1/portfolio.proto
// All inter-service calls use gRPC with protobuf

syntax = "proto3";
package portfolio.v1;

service PortfolioService {
    rpc GetPortfolioSummary(GetSummaryRequest) returns (PortfolioSummary);
    rpc GetPortfolioItems(GetItemsRequest) returns (GetItemsResponse);
    rpc GetPortfolioItem(GetItemRequest) returns (PortfolioItem);
    rpc GetPortfolioHealthScore(GetHealthRequest) returns (HealthScore);
    rpc GetUserPortfolioContext(GetContextRequest) returns (PortfolioContext);
}

message GetSummaryRequest { string user_id = 1; string workspace_id = 2; }
message PortfolioSummary {
    string portfolio_id = 1;
    int32 item_count = 2;
    float health_score = 3;
    repeated PortfolioItem active_items = 4;  // top 5 by recency
    google.protobuf.Timestamp updated_at = 5;
}
```

```
// Timeout configuration per call type:
// GetSummary: 500ms  GetItems: 1s  GetContext: 500ms
// All gRPC calls have: retry(max=3, backoff=100ms), circuit breaker via Istio
```

§25 · ERRORS

Error Handling & Taxonomy

25.1 Error Code Taxonomy

Error Code	HTTP Status	Retryable?	Category	Description
VALIDATION_ERROR	400	No	Input	Request body fails schema validation; field + message in details
IDEMPOTENCY_KEY_REQUIRED	400	No	Protocol	Mutating endpoint missing Idempotency-Key header
IDEMPOTENCY_BODY_MISMATCH	422	No	Protocol	Idempotency key reused with different request body
UNAUTHENTICATED	401	No	Auth	Missing, expired, or invalid JWT
SESSION_REVOKED	401	No	Auth	JWT jti found in revocation set
FORBIDDEN	403	No	Auth	Authenticated but insufficient permission
PLAN_LIMIT_REACHED	402	No	Billing	Action blocked by current plan tier; upgrade link in details
NOT_FOUND	404	No	Resource	Resource does not exist or access denied (not distinguished)
VERSION_CONFLICT	409	Yes	Concurrency	Optimistic lock failure; client should re-fetch and retry
DUPLICATE_REQUEST	409	No	Protocol	Request body is identical to a recent conflicting request
RATE_LIMIT_EXCEEDED	429	Yes	Protocol	Rate limit hit; Retry-After header set
STORY_STATUS_INVALID_TRANSITION	422	No	Domain	Story state machine: invalid from→to transition
SPRINT_ALREADY_ACTIVE	422	No	Domain	Cannot start sprint when another sprint is active
INSUFFICIENT_BALANCE	402	No	Financial	Wallet balance insufficient for operation
KYC_REQUIRED	402	No	Financial	Transaction requires higher KYC level
ESCROW_INVALID_STATE	422	No	Financial	Escrow state machine: invalid operation for current state
ESCROW_ALREADY_DISPUTED	409	No	Financial	Cannot dispute an already-disputed escrow
PROPOSAL_VOTING_CLOSED	422	No	Governance	Voting window has closed for this proposal

Error Code	HTTP Status	Retryable?	Category	Description
ALREADY_VOTED	409	No	Governance	Member has already cast a vote on this proposal
AI_CREDITS_EXHAUSTED	402	No	AI	Monthly AI credit quota depleted; upgrade or wait for reset
AI_CONTEXT_BUILD_TIMEOUT	503	Yes	AI	Portfolio context fetch timed out; retryable
IDEA_STAGE_EXIT_CRITERIA_UNMET	422	No	Domain	Cannot advance stage; missing required fields; details has list
CAMPAIGN_NOT_LIVE	422	No	Domain	Cannot pledge to a non-live campaign
SERVICE_UNAVAILABLE	503	Yes	Infrastructure	Downstream service unavailable; Retry-After set
INTERNAL_ERROR	500	Yes	Infrastructure	Unexpected server error; incident reference in response

25.2 Go Error Handling Pattern

```go
// shared/go/errors/errors.go

type KogiError struct {
    Code       string
    Message    string
    HTTPStatus int
    Field      string
    Details    map[string]interface{}
    Cause      error
    Retryable  bool
}

func (e *KogiError) Error() string { return fmt.Sprintf("[%s] %s", e.Code,
e.Message) }
func (e *KogiError) Unwrap() error { return e.Cause }

// Constructor functions per error type:
func ErrValidation(field, msg string) *KogiError {
    return &KogiError{Code: "VALIDATION_ERROR", HTTPStatus: 400, Field: field,
Message: msg}
}

func ErrInsufficientBalance() *KogiError {
    return &KogiError{Code: "INSUFFICIENT_BALANCE", HTTPStatus: 402, Retryable:
false,
        Message: "Wallet balance insufficient for this operation."}
}

func ErrVersionConflict(current int) *KogiError {
    return &KogiError{Code: "VERSION_CONFLICT", HTTPStatus: 409, Retryable: true,
```

```
        Message: "Resource was modified by another request. Re-fetch and retry.",
        Details: map[string]interface{}{"current_version": current}}
}

// Middleware: convert *KogiError to JSON envelope response
func ErrorHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Next()
        if len(c.Errors) == 0 { return }
        err := c.Errors.Last().Err
        var ke *KogiError
        if errors.As(err, &ke) {
            if ke.Retryable { c.Header("Retry-After", "5") }
            c.JSON(ke.HTTPStatus, errorEnvelope(ke, c.GetString("request_id")))
        } else {
            log.Error("unhandled error", "err", err, "trace_id", getTraceID(c))
            c.JSON(500, errorEnvelope(&KogiError{Code:"INTERNAL_ERROR",...}, ...))
        }
    }
}
```

25.3 Circuit Breaker Configuration

```
// Istio DestinationRule circuit breaker (per-service)
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: exchange-service-circuit-breaker
spec:
  host: exchange-service.kogi-services.svc.cluster.local
  trafficPolicy:
    outlierDetection:
      consecutive5xxErrors: 5           # 5 consecutive 5xx → eject
      interval: 10s                     # Evaluation window
      baseEjectionTime: 30s             # Initial ejection duration
      maxEjectionPercent: 50            # Max % of hosts ejected (HA)
    connectionPool:
      http:
        http1MaxPendingRequests: 100
        http2MaxRequests: 1000
        retries:
          attempts: 3
          perTryTimeout: 5s
          retryOn: "5xx,connect-failure,retriable-4xx"
      tcp:
        maxConnections: 100
        connectTimeout: 3s
```

§26 · TESTING

Test Strategy & Test Catalog

The Kogi test strategy follows the Testing Pyramid: many unit tests, targeted integration tests, and critical end-to-end smoke tests. All tests run in CI via Bazel. Code coverage gate: 80% for all services.

26.1 Test Pyramid

Layer	Framework	Count Target	Coverage Target	Run In
Unit Tests	Go testing + testify	> 500 per service	> 80% line coverage	Every PR; < 5 min
Integration Tests	Go testing + Docker Compose (Postgres + Redis + NATS)	> 50 per service	Critical paths + happy path	PR merge to main; < 15 min
Contract Tests	pact-go (consumer-driven)	1 per gRPC pair	All gRPC interfaces	PR merge; < 5 min
End-to-End (Smoke)	Playwright (TypeScript)	> 20 critical flows	P0 user journeys	Post-deploy to staging; < 10 min
Performance Tests	k6 (TypeScript)	5 load scenarios	SLO compliance	Weekly on staging; < 30 min
Security Tests	OWASP ZAP + gosec	Full OWASP Top 10	API endpoints	Weekly on staging; < 30 min

26.2 Critical Test Catalog

Test ID	System	Test Description	Type
AUTH-001	Auth	Register → verify email → login → get JWT → validate JWT claims	Integration
AUTH-002	Auth	10 failed logins → account locked → unlock after timeout	Integration
AUTH-003	Auth	Login → logout → reuse refresh token → expect 401	Integration
AUTH-004	Auth	TOTP enrollment → scan QR → verify code → MFA login → disable MFA	Integration
PORT-001	Portfolio	Create portfolio → add 3 items → compute health score → verify components	Integration
PORT-002	Portfolio	Add 1000 items → fetch tree → verify depth + sort order	Performance

Test ID	System	Test Description	Type
WBS-001	WBS	Create story → transition through all valid states → verify history	Integration
WBS-002	WBS	Transition DONE → IN_PROGRESS → expect STORY_STATUS_INVALID_TRANSITION	Unit
WBS-003	WBS	Complete story linked to OKR → verify OKR auto-check-in created	Integration
WBS-004	WBS	Sprint planning → pack stories → verify capacity constraint respected	Unit
MKT-001	Market	Create listing → place order → accept → deliver → approve → verify escrow released	E2E
MKT-002	Market	Place order → dispute → mediator resolves → verify refund	Integration
MKT-003	Market	Recommendation engine → verify top listing has cosine similarity ≥ 0.7 with query	Unit
EXC-001	Exchange	Concurrent wallet debit (100 goroutines) → verify no negative balance, no lost updates	Integration (concurrency)
EXC-002	Exchange	Duplicate FundEscrow call with same idempotency key → verify single debit	Integration
EXC-003	Exchange	Escrow review deadline sweep → verify auto-release fires exactly once	Integration
EXC-004	Exchange	Nightly reconciliation → introduce artificial ledger error → verify alert fires	Integration
AO-001	AO Governance	Create AO → invite 3 members → submit proposal → all vote yes → verify auto-execution	E2E
AO-002	AO Governance	Proposal reaches quorum → vote → verify governance_log record is immutable (no DELETE)	Integration
AO-003	AO Governance	Distribution formula: pro-rata stake → 3 members with 50/30/20% → verify amounts	Unit
AI-001	AI Agent	Chat session → ask about blocked stories → verify context includes actual blocked stories	Integration
AI-002	AI Agent	AI action: create_story → verify story created in correct WBS → verify action_log entry	Integration
JOB-001	Jobs	wallet_reconciliation job → all wallets balanced → 0 errors	Integration
JOB-002	Jobs	Two pods start wallet_reconciliation simultaneously → verify only one executes (Redis lock)	Integration
SEC-001	Security	Attempt SQL injection in story title → verify parameterized query blocks it	Security

Test ID	System	Test Description	Type
SEC-002	Security	Access portfolio of user B while authenticated as user A → expect 404	Security
SEC-003	Security	Submit JWT with alg:none → expect 401	Security
SEC-004	Security	Revoke session → attempt request with old JWT → expect 401	Security

26.3 Financial Test Suite — Special Requirements

⚠️ **Financial Testing Invariants**

- EXC-001 MUST run under maximum concurrency (100 goroutines) to expose race conditions. Must pass 1000 consecutive runs.
- EXC-002 MUST verify exactly ONE ledger entry created for duplicate request (check count, not just balance).
- EXC-003 MUST use real time (not mocked) for sweep test to verify cron expression is correct.
- EXC-004 runs in staging weekly with synthetic ledger errors injected — NEVER in production.
- All financial tests run against a dedicated test DB with no shared state between test cases.
- Financial test suite is a blocking gate before any production deploy involving exchange-svc or fund-svc.

§27 · MIGRATION

Data Migration & Schema Versioning

All database schema changes are managed via golang-migrate with numbered SQL migration files. Migrations are applied automatically in CI before integration tests and in production via the deployment pipeline. Zero-downtime migrations are required for all production changes.

27.1 Migration Strategy

Change Type	Strategy	Downtime?	Example
Add nullable column	Single migration: ALTER TABLE ADD COLUMN nullable	None	ADD COLUMN ai_summary TEXT NULL
Add NOT NULL column	Phase 1: add nullable. Phase 2 (after code deploy): add DEFAULT. Phase 3: add NOT NULL constraint	None (3 deploys)	ADD COLUMN status VARCHAR(20) NULL → backfill → ALTER ADD CONSTRAINT
Add index	CREATE INDEX CONCURRENTLY (non-blocking on Postgres 11+)	None	CREATE INDEX CONCURRENTLY idx_name ON table(col)
Drop column	Phase 1: remove all code references. Phase 2 (next deploy): DROP COLUMN	None (2 deploys)	Remove all ORM references first
Rename column	Phase 1: add new_col, write to both. Phase 2: migrate reads to new_col. Phase 3: drop old_col	None (3 deploys)	Never use ALTER COLUMN RENAME in a single deploy
Add unique constraint	Deduplicate data first; then add UNIQUE constraint in separate migration	None if data is clean	Dedupe script + migration in same PR
Change column type (widening)	ALTER COLUMN TYPE to wider type (e.g. INT → BIGINT) — usually safe	None	ALTER COLUMN points TYPE BIGINT
Change column type (narrowing)	Dangerous — requires data audit; new column + backfill + drop old	None (risky)	Always audit data first; CI test required
Partition large table	Create partitioned table; migrate data in batches; swap at maintenance window	Maintenance window required	message table partitioned by room_id

27.2 Migration File Convention

```
-- Migration naming: {service}/migrations/NNNNNN_description.up.sql
-- Rollback:         {service}/migrations/NNNNNN_description.down.sql

-- Example: 000043_add_story_ai_summary.up.sql
ALTER TABLE story ADD COLUMN ai_summary TEXT NULL;
ALTER TABLE story ADD COLUMN ai_suggestions JSONB NULL;

CREATE INDEX CONCURRENTLY idx_story_ai_summary_null
    ON story(id) WHERE ai_summary IS NULL AND status != 'cancelled' AND deleted_at
IS NULL;

-- Comments required: WHY is this change happening?
-- Example: 000043_add_story_ai_summary.down.sql
DROP INDEX CONCURRENTLY IF EXISTS idx_story_ai_summary_null;
ALTER TABLE story DROP COLUMN IF EXISTS ai_suggestions;
ALTER TABLE story DROP COLUMN IF EXISTS ai_summary;

-- Migration testing requirement:
-- 1. Run .up.sql on clean test DB
-- 2. Run .down.sql
-- 3. Run .up.sql again (idempotency check)
-- 4. All three steps must succeed without error
```

27.3 Schema Version Table

```
-- golang-migrate tracks applied migrations in schema_migrations table
-- Per-service: each service has its own schema_migrations in its own DB

SELECT version, dirty, applied_at
FROM schema_migrations
ORDER BY version DESC
LIMIT 5;

-- version  | dirty | applied_at
-- ---------+-------+---------------------------
--   000043  | false | 2025-01-10 02:31:44.123 UTC
--   000042  | false | 2024-12-15 01:00:12.456 UTC
--   000041  | false | 2024-11-20 03:15:55.789 UTC

-- dirty=true indicates a failed migration — this is a P1 incident.
-- Never modify a migration file after it has been applied to staging or
production.
-- If a migration needs correction, create a new numbered migration.
```

§28 · GLOSSARY

Glossary & Reference

Term	Definition
AO (Autonomous Organization)	A Kogi entity with its own charter, member registry, capital pool, and governance system. Supertype for cooperatives, investment clubs, open-source foundations, and custom collective organizations.
Argon2id	Password hashing algorithm used for all user passwords. Parameters: time=3, memory=65536, threads=4, keylen=32. OWASP recommended for 2024.
Backfill	A bulk re-indexing or re-computation job that processes existing records to populate a new column or index. Always run with rate limiting to avoid production impact.
BMC (Business Model Canvas)	Nine-block visual framework for describing a business model. Stored as JSONB in idea.canvas.
CQRS (Command Query Responsibility Segregation)	Architectural pattern: write operations (commands) go through the service write path; read operations (queries) are served from optimized projections or caches.
Cursor Pagination	Pagination method using an opaque cursor encoding the last-seen record position. Stable across concurrent inserts/deletes; preferred over offset-based pagination.
DataLoader	Batching pattern: collects individual resource fetch requests for 10ms, then issues a single batch DB query. Prevents N+1 query problems.
DEK (Data Encryption Key)	A symmetric key used to encrypt a specific data record. Wrapped by a CMK (Customer Master Key) from AWS KMS — envelope encryption.
Distributed Lock	Redis SETNX-based mechanism ensuring only one pod executes a background job at a time. Released by TTL expiry or explicit DEL.
DLQ (Dead Letter Queue)	A NATS subject where events are written after all delivery retry attempts are exhausted. Triggers on-call alert.
DSL (Domain-Specific Language)	In automation context: the expression language used in automation rule conditions. Supports operators: eq, neq, gt, lt, contains, in, not_in.
Envelope Encryption	AWS KMS pattern: each sensitive field encrypted with a per-record DEK; the DEK is itself encrypted by a CMK and stored alongside the ciphertext.
FFI (Foreign Function Interface)	The mechanism by which the Go API Gateway calls Zig Kernel functions via C-ABI exported functions. Uses cgo on the Go side.
Fractional Index	Sort order stored as a floating-point number (e.g. 1000, 2000, 1500) allowing insertion between existing positions without re-numbering.
IRSA (IAM Roles for Service Accounts)	AWS EKS mechanism: Kubernetes service accounts are annotated with an IAM role ARN; pods using that account automatically receive IAM credentials for AWS API access without static credentials.
JetStream	NATS persistence and delivery-guarantee layer. Provides at-least-once delivery, consumer groups, and replay from any point in the stream.

Term	Definition
JTBD (Jobs to Be Done)	Product framework: understand what job a user "hires" a product to do (functional, emotional, and social dimensions).
KNN (K-Nearest Neighbors)	Elasticsearch 8 dense vector search: finds listings whose embedding vector is most similar (cosine similarity) to a query embedding. Used for semantic search.
KYC (Know Your Customer)	Identity verification process. Level 0: email only. Level 1: email + phone. Level 2: government ID. Level 3: enhanced due diligence (for high-value financial operations).
mTLS (Mutual TLS)	TLS where BOTH parties present certificates. Used between all Kogi services via Istio service mesh. Prevents unauthorized inter-service communication.
Outbox Pattern	Database design pattern for guaranteed event delivery: write domain state + event to outbox_events table in same ACID transaction. A relay process reads outbox and publishes to NATS.
PITR (Point-in-Time Recovery)	RDS backup feature: restore database to any second within the retention window (90 days for financial DBs, 30 days for others).
pgvector	PostgreSQL extension providing a vector(n) column type and cosine/L2/IP similarity operators. Used in agent_memory for semantic memory retrieval.
Progressive Disclosure	UX principle: show only what is needed for the current task; reveal more complexity as the user demonstrates readiness.
Saga Pattern	Distributed transaction pattern using a sequence of local transactions each with a compensating action. Used in Zig Kernel Provisioner for new-user setup.
SELECT FOR UPDATE	PostgreSQL row-level lock: blocks other transactions from reading/writing the locked row until the current transaction commits. Required for all wallet mutations (INV-F1).
SPIFFE SVID	Standard identity document used in Istio mTLS. Each service pod has a SPIFFE-formatted identity (spiffe://cluster.local/ns/kogi-services/sa/wbs-service).
SSE (Server-Sent Events)	HTTP/2 feature: server pushes a stream of text events to a client over a single persistent HTTP connection. Unidirectional (server→client). Used for notifications and AI streaming.
WAPV (Weekly Active Portfolio Value)	Kogi North Star Metric: total economic value (projects, assets, marketplace GMV) actively managed by weekly active users.
WBS (Work Breakdown Structure)	Hierarchical decomposition of a project into: Work Package → Theme → Initiative → Epic → Story → Task. The Kogi WBS mirrors standard PM methodology.

ℹ️ **Document End**

- This document covers all 28 sections of the Kogi Platform LLSDD v1.0.0.
- For the latest version of any section, see the internal engineering wiki at eng.kogi.internal/llsdd.
- Questions, corrections, or proposed additions: open an engineering issue tagged [llsdd].

Kogi Platform — Low-Level Software Design Document

Version 1.0.0 · Platform Engineering · Confidential